

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 17 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127 #include <cuda/ptx>
128
129 #if defined(NOTES_V2_ENABLE_CUDNN)
130 #pragma nv_diag_suppress 128
131 #include <cuda/cudnn_frontend.h>
132 #pragma nv_diag_default 128
133 namespace fe = cudnn_frontend;
134 #endif
135
136 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
137 #error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
138 #endif
139
140 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
141 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
142 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
143
144 static constexpr int kWarpSize = 32;
145
146 // =====
147 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
148 // =====
149
150 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
151 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
152 // 复杂度 O(logN), N=32 时只需 5 步
153 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
154 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
155 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
156 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
157 //
158 // 初始: 每个 lane 持有自己的值 v0..v7
159 // lane: 0 1 2 3 4 5 6 7
160 // val: v0 v1 v2 v3 v4 v5 v6 v7
161 //
162 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
163 //
164 //      ┌──────────┐
165 // 对: (0,4) (1,5) (2,6) (3,7)
166 //
167 // lane: 0 1 2 3 4 5 6 7
168 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
169 //
170 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
171 //
172 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
173 // 对: (0,2) (1,3) (4,6) (5,7)
174 //      ─── 前4个一组 ───      ─── 后4个一组 ───
175 //
176 // lane: 0 1 2 3 4 5 6 7 (每 lane 持有前一轮2个值的和再加本轮配对)
177 // val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}
178 //      = v0+v2+v4+v6 ... (逐步归约)
179 //
180 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
181 //
182 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
183 // 对: (0,1) (2,3) (4,5) (6,7)
184 //
185 // lane: 0 1 2 3 4 5 6 7
186 // val: Σall Σall Σall Σall Σall Σall Σall Σall ← 所有 lane 拥有全归约结果!
187 //
188 // mask=0: 循环终止, 归约完成。
189 //
190 // 关键性质:
191 // - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i

```

```

189 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
190 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
191 // - 0(log2 N) 步完成, 无需 shared memory, 纯寄存器操作
192 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
193 // 当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
194 template <const int kWarpWidth = kWarpSize, typename T = float>
195 __device__ __forceinline__ T warp_reduce_sum(T val) {
196 #pragma unroll
197     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
198         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
199     }
200     return val;
201 }
202
203 // Warp Reduce Max — generic
204 template <const int kWarpWidth = kWarpSize, typename T = float>
205 __device__ __forceinline__ T warp_reduce_max(T val) {
206 #pragma unroll
207     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
208         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
209     }
210     return val;
211 }
212
213 // =====
214 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
215 // =====
216
217 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
218 // 两级归约流程:
219 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
220 // 2. warp leader (lane=0) 写入 shared memory
221 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
222 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
223 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
224 template <const int kNumThreads = 256>
225 __device__ float block_reduce_sum(float val) {
226     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
227     int warp = threadIdx.x / kWarpSize;
228     int lane = threadIdx.x % kWarpSize;
229     __shared__ float shared[kNumWarps];
230
231     float value = warp_reduce_sum<kWarpSize>(val);
232     if (lane == 0)
233         shared[warp] = value;
234     __syncthreads();
235     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
236     value = warp_reduce_sum<kNumWarps>(value);
237     // 关键: broadcast 结果到所有线程, 后续用 result
238     // 做除法等操作时所有线程都能拿到
239     value = __shfl_sync(0xffffffff, value, 0, 32);
240     return value;
241 }
242
243 // Block Reduce Max — FP32 (增强版, 带 broadcast)
244 template <const int kNumThreads = 256>
245 __device__ float block_reduce_max(float val) {
246     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
247     int warp = threadIdx.x / kWarpSize;
248     int lane = threadIdx.x % kWarpSize;
249     __shared__ float shared[kNumWarps];
250
251     float value = warp_reduce_max<kWarpSize>(val);

```

```

252     if (lane == 0)
253         shared[warp] = value;
254     __syncthreads();
255     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
256     value = warp_reduce_max<kNumWarps>(value);
257     value = __shfl_sync(0xffffffff, value, 0, 32);
258     return value;
259 }
260
261 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
262 // 多 block 各自做 warp->smem->warp0 reduce, 然后 atomicAdd 到全局 y
263 // 跨 block 求和的常见模式, 适合 N 较大时使用;
264 // Grid: ((N + 255) / 256, 1, 1)
265 // Block: (256, 1, 1)
266 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
267 template <const int kNumThreads = 256>
268 __global__ void block_reduce_all(float *a, float *y, int N) {
269     int tid = threadIdx.x;
270     int idx = blockIdx.x * kNumThreads + tid;
271     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
272     __shared__ float shared[kNumWarps];
273
274     float val = (idx < N) ? a[idx] : 0.0f;
275     int warp = tid / kWarpSize;
276     int lane = tid % kWarpSize;
277
278     val = warp_reduce_sum<kWarpSize>(val);
279     if (lane == 0)
280         shared[warp] = val;
281     __syncthreads();
282
283     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
284     if (warp == 0)
285         val = warp_reduce_sum<kNumWarps>(val);
286     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
287         atomicAdd(y, val);
288 }
289
290 // ---- Dot Product: y = sum(a[i] * b[i]) ----
291 // 核心模式: elementwise 乘法 -> block reduce -> atomicAdd 全局累加
292 // Grid: ((N + 255) / 256, 1, 1)
293 // Block: (256, 1, 1)
294 // source: LeetCUDA/kernels/dot-product/dot_product.cu
295 template <const int kNumThreads = 256>
296 __global__ void dot(float *a, float *b, float *y, int N) {
297     int tid = threadIdx.x;
298     int idx = blockIdx.x * kNumThreads + tid;
299     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
300     __shared__ float shared[kNumWarps];
301
302     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
303     int warp = tid / kWarpSize;
304     int lane = tid % kWarpSize;
305
306     prod = warp_reduce_sum<kWarpSize>(prod);
307     if (lane == 0)
308         shared[warp] = prod;
309     __syncthreads();
310
311     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
312     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
313         prod = warp_reduce_sum<kNumWarps>(prod);
314     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加

```

```

315     atomicAdd(y, prod);
316 }
317
318 // Dot Product + float4
319 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
320 // Block: (64, 1, 1), 256/4=64
321 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
322 // source: LeetCUDA/kernels/dot-product/dot_product.cu
323 template <const int kNumThreads = 256 / 4>
324 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
325     int tid = threadIdx.x;
326     int idx = (blockIdx.x * kNumThreads + tid) * 4;
327     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
328     __shared__ float shared[kNumWarps];
329
330     float4 reg_a = FLOAT4(a[idx]);
331     float4 reg_b = FLOAT4(b[idx]);
332     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
333                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
334                       : 0.0f;
335     int warp = tid / kWarpSize;
336     int lane = tid % kWarpSize;
337
338     prod = warp_reduce_sum<kWarpSize>(prod);
339     if (lane == 0)
340         shared[warp] = prod;
341     __syncthreads();
342
343     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
344     if (warp == 0)
345         prod = warp_reduce_sum<kNumWarps>(prod);
346     if (tid == 0)
347         atomicAdd(y, prod);
348 }
349
350
351 // =====
352 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
353 // =====
354 // 面试要点:
355 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
356 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
357 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
358
359 // ---- ReLU: y = max(0, x) ----
360 // Grid: ((N + 255) / 256, 1, 1)
361 // Block: (256, 1, 1)
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu(float *x, float *y, int N) {
364     int idx = blockIdx.x * blockDim.x + threadIdx.x;
365     if (idx < N)
366         y[idx] = fmaxf(0.0f, x[idx]);
367 }
368
369 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
370 // block(64)x4(float4)=256 元素/block, 与基础版吞吐相同
371 // Grid: ((N + 255) / 256, 1, 1)
372 // Block: (64, 1, 1)
373 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
374 // source: LeetCUDA/kernels/relu/relu.cu
375 __global__ void relu_vec4(float *x, float *y, int N) {
376     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
377     if (idx < N) {

```

```

378     float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
379     float4 reg_y;
380     reg_y.x = fmaxf(0.0f, reg_x.x);
381     reg_y.y = fmaxf(0.0f, reg_x.y);
382     reg_y.z = fmaxf(0.0f, reg_x.z);
383     reg_y.w = fmaxf(0.0f, reg_x.w);
384     FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
385 }
386 }
387
388 // ---- Elementwise Add: c = a + b ----
389 // Grid: ((N + 255) / 256, 1, 1)
390 // Block: (256, 1, 1)
391 // source: LeetCUDA/kernels/elementwise/elementwise.cu
392 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
393     int idx = blockIdx.x * blockDim.x + threadIdx.x;
394     if (idx < N)
395         c[idx] = a[idx] + b[idx];
396 }
397
398 // Elementwise Add + float4 向量化
399 // Grid: ((N + 255) / 256, 1, 1)
400 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
401 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
402 // source: LeetCUDA/kernels/elementwise/elementwise.cu
403 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
404     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
405     if ((idx + 3) < N) {
406         float4 reg_a = FLOAT4(a[idx]);
407         float4 reg_b = FLOAT4(b[idx]);
408         float4 reg_c;
409         reg_c.x = reg_a.x + reg_b.x;
410         reg_c.y = reg_a.y + reg_b.y;
411         reg_c.z = reg_a.z + reg_b.z;
412         reg_c.w = reg_a.w + reg_b.w;
413         FLOAT4(c[idx]) = reg_c;
414     } else if (idx < N) {
415         for (int i = 0; (idx + i) < N; i++) {
416             c[idx + i] = a[idx + i] + b[idx + i];
417         }
418     }
419 }
420
421 // ---- Histogram: y[a[i]]++ ----
422 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
423 // Grid: ((N + 255) / 256, 1, 1)
424 // Block: (256, 1, 1)
425 // source: LeetCUDA/kernels/histogram/histogram.cu
426 __global__ void histogram(int *a, int *y, int N) {
427     int idx = blockIdx.x * blockDim.x + threadIdx.x;
428     if (idx < N)
429         atomicAdd(&(y[a[idx]]), 1);
430 }
431
432 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
433 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
434 //
435 // 面试要点:
436 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
437 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
438 //   - 数学公式 (5 步推导):
439 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
440 //         L_max = max(LSE_1, LSE_2)

```

```

441 // Step 2 — 还原指数权重 (un-normalized) :
442 //      w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
443 // Step 3 — 归一化为混合比例:
444 //      alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
445 //      满足 alpha + beta = 1
446 // Step 4 — 逐元素加权合并:
447 //      O = alpha * O_1 + beta * O_2
448 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention 惯例一致,
449 //   lse[head_idx][token_idx])
450 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
451 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
452 // -  $-\infty$  LSE  $\rightarrow -\infty$ : 空 attention 段 (causal mask 等导致全部 score
453 //   为  $-\infty$ ) 的 LSE 可能为  $+\infty$ , 替换后  $\exp(-\infty - L_{\max}) = 0$ ,
454 //   该段权重退化为 0
455 // - 128-bit 向量化: uint4 = 4  $\times$  float, 每线程处理 4 个输出元素,
456 //   pack load  $\rightarrow$  FMA  $\rightarrow$  pack store 一条流水线
457 // - AI  $\approx$  3 FLOP / 20 bytes  $\approx$  0.15 FLOPS/Byte  $\rightarrow$  severely memory-bound
458 //
459 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
460 // kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
461 // threads_per_head = head_size / 4 = 2
462 // total_threads = num_tokens * num_heads * threads_per_head = 8
463 //
464 // global_idx:      0      1      2      3      4      5      6      7
465 // token_head_idx:  0      0      1      1      2      2      3      3
466 //   (第几个 (token,head) 对, 行优先)
467 // pack_idx:        0      1      0      1      0      1      0      1
468 //   (该 (token,head) 对内第几个 pack)
469 // token_idx:        0      0      0      0      1      1      1      1
470 //   (token_head_idx / num_heads, token 变化慢)
471 // head_idx:         0      0      1      1      0      0      1      1
472 //   (token_head_idx % num_heads, head 变化快)
473 // pack_offset:      0      4      0      4      0      4      0      4
474 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
475 // head_offset:      0      0      8      8      16     16     24     24
476 //   (token_idx * num_heads * head_size + head_idx * head_size,
477 //   该 (token,head) 对在 output 展平数组中的起始偏移)
478 //
479 // Grid: ((total_threads + 127) / 128, 1, 1)
480 // Block: (128, 1, 1)
481 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
482 __global__ void merge_attn_states(
483     float *output,           // [num_tokens, num_heads, head_size]
484     const float *prefix_output, // [num_tokens, num_heads, head_size]
485     const float *prefix_lse,   // [num_heads, num_tokens]
486     const float *suffix_output, // [num_tokens, num_heads, head_size]
487     const float *suffix_lse,   // [num_heads, num_tokens]
488     int num_tokens, int num_heads, int head_size) {
489
490     constexpr int kNumThreads = 128;
491     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
492     using pack_t = uint4;
493
494     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
495     const int threads_per_head = head_size / kPackSize;
496     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
497     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
498     const int total_threads = num_tokens * num_heads * threads_per_head;
499
500     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
501     if (global_idx >= total_threads)
502         return;
503

```

```

504 // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
505 // token_head_idx: 第几个 (token, head) 对, 行优先展平
506 const int token_head_idx = global_idx / threads_per_head;
507 // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
508 const int pack_idx = global_idx % threads_per_head;
509
510 // token_head_idx 分解为 (token_idx, head_idx)
511 const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
512 const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
513
514 // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
515 const int pack_offset = pack_idx * kPackSize;
516 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
517 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
518
519 // 定位到当前 (token, head) 的输出段起始
520 const float *prefix_head = prefix_output + head_offset;
521 const float *suffix_head = suffix_output + head_offset;
522 float *output_head = output + head_offset;
523
524 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
525 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
526 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
527
528 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
529 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
530 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
531
532 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
533 const float max_lse = fmaxf(p_lse, s_lse);
534 p_lse -= max_lse;
535 s_lse -= max_lse;
536
537 // Step 2-3: 指数还原 → 混合比例 alpha, beta
538 const float p_se = expf(p_lse);
539 const float s_se = expf(s_lse);
540 const float out_se = p_se + s_se;
541 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
542 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
543
544 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
545 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
546 if (pack_offset < head_size) {
547     pack_t p_pack =
548         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
549     pack_t s_pack =
550         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
551     pack_t o_pack;
552
553     #pragma unroll
554     for (int i = 0; i < kPackSize; ++i) {
555         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
556         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
557         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
558         reinterpret_cast<float *>(&o_pack)[i] = o_v;
559     }
560
561     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
562 }
563 }
564
565 // =====
566 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm

```

```

567 // =====
568 // 面试要点:
569 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
570 //   online (1-pass, 增量更新)
571 //   - Online Softmax 是 FlashAttention 的数学基础
572 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
573 //   + variance)
574 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
575
576 // =====
577 // Phase 3a: Softmax — 三级递进 (面试核心考点)
578 // =====
579 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
580 // 回答线索: naive(溢出) → safe → online
581
582 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
583 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
584 // 问题: x 值很大时 exp(x) 溢出为 inf
585 template <const int kNumThreads = 256>
586 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
587 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
588 // source: LeetCUDA/kernels/softmax/softmax.cu
589 __global__ void softmax_per_token(float *x, float *y, int N) {
590     const int tid = threadIdx.x;
591     const int idx = blockIdx.x * blockDim.x + tid;
592
593     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
594     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
595     if (idx < N)
596         y[idx] = exp_val / exp_sum;
597 }
598
599 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
600 // 面试重点: 为什么 Softmax 需要 Safe?
601 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
602 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
603 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
604 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
605 // Grid: (S, 1, 1)
606 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
607 // source: LeetCUDA/kernels/softmax/softmax.cu
608 template <const int kNumThreads = 256>
609 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
610     const int tid = threadIdx.x;
611     const int idx = blockIdx.x * blockDim.x + tid;
612
613     // Pass 1: block reduce max — 找最大值
614     float val = (idx < N) ? x[idx] : -FLT_MAX;
615     float max_val = block_reduce_max<kNumThreads>(val);
616
617     // Pass 2: exp(x - max) → block reduce sum
618     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
619     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
620
621     if (idx < N)
622         y[idx] = exp_val / exp_sum;
623 }
624
625 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
626 // 面试重点:
627 // 两种使用场景 (公式不同, 但结果等价):
628 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
629 //       m_new = max(m_old, x_i)

```

```

630 //      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
631 //  2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
632 //      m = max(m1, m2) safe softmax用的max值
633 //      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
634 //      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
635 //  算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
636 //  Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
637 //
638 //  不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
639 //  warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
640 //  (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
641 //  (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
642 //
643 //  合并公式 (对称, 不依赖"新/旧"概念) :
644 //  m = max(m1, m2)
645 //  d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
646 //
647 //  该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
648 struct __align__(8) MD {
649     float m; // running max
650     float d; // running denominator (sum of exp(x - max))
651 };
652
653 template <const int kWarpWidth = kWarpSize>
654 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
655     #pragma unroll
656     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
657         MD md2;
658         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
659         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
660
661         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
662         MD s_m = (md1.m > md2.m) ? md2 : md1;
663
664         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
665         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
666         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
667         md1.m = b_m.m;
668     }
669     return md1;
670 }
671
672 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
673 // Grid: (S, 1, 1)
674 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
675 // source: LeetCUDA/kernels/softmax/softmax.cu
676 template <const int kNumThreads = 256>
677 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
678     int tid = threadIdx.x;
679     int idx = blockIdx.x * kNumThreads + threadIdx.x;
680     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
681     __shared__ MD shared[kNumWarps];
682
683     float val = (idx < N) ? x[idx] : -FLT_MAX;
684     int warp = tid / kWarpSize;
685     int lane = tid % kWarpSize;
686
687     // 初始化: 每个线程持有一个 (max, denom) 对
688     MD md;
689     md.m = idx < N ? val : -FLT_MAX;
690     md.d = idx < N ? 1.0f : 0.0f;
691
692     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d

```

```

693 md = warp_reduce_md<kWarpSize>(md);
694
695 if (lane == 0)
696     shared[warp] = md;
697 __syncthreads();
698
699 // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
700 md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
701 md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
702
703 // 用全局 max 和 denom 做最终 softmax
704 float d_inv = __fdividef(1.0f, md.d);
705 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
706 if (idx < N) {
707     y[idx] = __expf(val - md.m) * d_inv;
708 }
709 }
710
711 // =====
712 // Phase 3b: RMS Normalization (1-pass reduce)
713 // =====
714 // 面试要点:
715 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
716 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
717 //   - Llama 系列使用 RMS Norm
718 //   - grid(N, K/K), block(K): 一行一个 block
719 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
720 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
721 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
722 template <const int kNumThreads = 128>
723 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
724     int tid = threadIdx.x;
725     int idx = blockIdx.x * blockDim.x + threadIdx.x;
726     const float epsilon = 1e-5f;
727
728     __shared__ float s_variance;
729     float value = (idx < N * K) ? x[idx] : 0.0f;
730     float variance = value * value;
731     variance = block_reduce_sum<kNumThreads>(variance);
732     if (tid == 0)
733         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
734     __syncthreads();
735     if (idx < N * K)
736         y[idx] = (value * s_variance) * g;
737 }
738
739 // RMS Norm + float4
740 // Grid: (N, 1, 1)
741 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
742 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
743 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
744 template <const int kNumThreads = 128 / 4>
745 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
746     int tid = threadIdx.x;
747     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
748     const float epsilon = 1e-5f;
749
750     __shared__ float s_variance;
751     float4 reg_x = FLOAT4(x[idx]);
752     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
753                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
754                                     : 0.0f;
755     variance = block_reduce_sum<kNumThreads>(variance);

```

```

756     if (tid == 0)
757         s_variance = rsqrtf(variance / (float)K + epsilon);
758     __syncthreads();
759     float4 reg_y;
760     reg_y.x = reg_x.x * s_variance * g;
761     reg_y.y = reg_x.y * s_variance * g;
762     reg_y.z = reg_x.z * s_variance * g;
763     reg_y.w = reg_x.w * s_variance * g;
764     if (idx < N * K)
765         FLOAT4(y[idx]) = reg_y;
766 }
767
768 // =====
769 // Phase 3c: Layer Normalization (2-pass reduce)
770 // =====
771 // 面试要点:
772 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
773 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
774 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
775 // Grid: (N, 1, 1), 一行一个 block
776 // Block: (128, 1, 1), kNumThreads=K=128
777 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
778 template <const int kNumThreads = 128>
779 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
780     int tid = threadIdx.x;
781     int idx = blockIdx.x * blockDim.x + threadIdx.x;
782     const float epsilon = 1e-5f;
783
784     __shared__ float s_mean;
785     __shared__ float s_variance;
786     float value = (idx < N * K) ? x[idx] : 0.0f;
787
788     // Pass 1: compute mean
789     float sum = block_reduce_sum<kNumThreads>(value);
790     if (tid == 0)
791         s_mean = sum / (float)K;
792     __syncthreads(); // 必须等待 s_mean 对所有线程可见
793
794     // Pass 2: compute variance = (x - mean)2
795     float variance = (value - s_mean) * (value - s_mean);
796     variance = block_reduce_sum<kNumThreads>(variance);
797     if (tid == 0)
798         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
799     __syncthreads(); // 必须等待 s_variance 对所有线程可见
800
801     if (idx < N * K)
802         y[idx] = ((value - s_mean) * s_variance) * g + b;
803 }
804
805 // Layer Norm + float4
806 // Grid: (N, 1, 1)
807 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
808 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
809 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
810 template <const int kNumThreads = 128 / 4>
811 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
812     int tid = threadIdx.x;
813     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
814     const float epsilon = 1e-5f;
815
816     __shared__ float s_mean;
817     __shared__ float s_variance;
818     float4 reg_x = FLOAT4(x[idx]);

```

```

819 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
820
821 float sum = block_reduce_sum<kNumThreads>(value);
822 if (tid == 0)
823     s_mean = sum / (float)K;
824 __syncthreads();
825
826 float4 reg_x_hat;
827 reg_x_hat.x = reg_x.x - s_mean;
828 reg_x_hat.y = reg_x.y - s_mean;
829 reg_x_hat.z = reg_x.z - s_mean;
830 reg_x_hat.w = reg_x.w - s_mean;
831 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
832                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
833 variance = block_reduce_sum<kNumThreads>(variance);
834 if (tid == 0)
835     s_variance = rsqrtf(variance / (float)K + epsilon);
836 __syncthreads();
837
838 float4 reg_y;
839 reg_y.x = reg_x_hat.x * s_variance * g + b;
840 reg_y.y = reg_x_hat.y * s_variance * g + b;
841 reg_y.z = reg_x_hat.z * s_variance * g + b;
842 reg_y.w = reg_x_hat.w * s_variance * g + b;
843 if (idx < N * K)
844     FLOAT4(y[idx]) = reg_y;
845 }
846
847 // =====
848 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
849 // =====
850 // 面试要点:
851 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
852 //     [x1']   [cos(θ)  -sin(θ)] [x1]
853 //     [x2'] = [sin(θ)   cos(θ)] [x2]
854 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
855 //   - token_pos = idx / N: token 在序列中的位置
856 //   - token_idx = idx % N: token 内的维度对索引
857 //   - 输入 [seq_len, hidden_size], 输出同形状
858
859 // Grid: ((seq_len * N + 255) / 256, 1, 1)
860 // Block: (256, 1, 1)
861 // source: LeetCUDA/kernels/rope/rope.cu
862 __global__ void rope(float *x, float *out, int seq_len, int N) {
863     int idx = blockIdx.x * blockDim.x + threadIdx.x;
864     float x1 = x[idx * 2];
865     float x2 = x[idx * 2 + 1];
866     int token_pos = idx / N; // 序列位置
867     int token_idx = idx % N; // 维度对索引
868
869     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
870     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
871     float sin_v = sinf(token_pos * exp_v);
872     float cos_v = cosf(token_pos * exp_v);
873
874     // 2D 旋转
875     float out1 = x1 * cos_v - x2 * sin_v;
876     float out2 = x1 * sin_v + x2 * cos_v;
877     out[idx * 2] = out1;
878     out[idx * 2 + 1] = out2;
879 }
880
881 // =====

```

```

882 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
883 // =====
884 // 面试要点 (Bank Conflict 专题) :
885 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
886 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
887 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
888 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
889 //
890 // 转置的四步演进:
891 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
892 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
893 //   BCF: smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
894 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
895 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
896
897 // 每个线程处理 1 个元素, block(16,16)
898 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
899 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
900 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
901 // Block: (16, 16, 1)
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
904     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
905     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
906     if (r < row && c < col) {
907         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
908         y[c * row + r] = x[r * col + c];
909     }
910 }
911
912 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
913 // Block: (16, 16, 1)
914 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
915 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
916 __global__ void mat_transpose_padded(
917     float *x, float *y, const int row, const int col) {
918     const int tx = threadIdx.x;
919     const int ty = threadIdx.y;
920
921     constexpr int TILE = 16;
922     constexpr int PAD = 1;
923     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
924     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
925
926     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
927     const int x_c = blockIdx.x * TILE + tx;
928     const int x_r = blockIdx.y * TILE + ty;
929
930     if (x_r * 4 < row && x_c < col) {
931         // Step 1: 4 rows × 1 col → smem (row-major);
932 #pragma unroll
933         for (int i = 0; i < 4; ++i) {
934             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
935         }
936         __syncthreads();
937
938         // Step 2: Transposed read from smem
939         float4 reg_trans;
940         reg_trans.x = tile[tx * 4 + 0][ty];
941         reg_trans.y = tile[tx * 4 + 1][ty];
942         reg_trans.z = tile[tx * 4 + 2][ty];
943         reg_trans.w = tile[tx * 4 + 3][ty];
944

```

```

945 // y 空间坐标: y_r = x 的列, y_c = x 的行
946 const int y_r = blockIdx.x * TILE + ty; // = x col
947 const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
948
949 // Coalesced write: y[y_r][y_c .. y_c+3]
950 FLOAT4(y[y_r * row + y_c]) = reg_trans;
951 }
952 }
953
954 // =====
955 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
956 // =====
957 // 面试要点:
958 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
959 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
960 // - 不同 K 值对应不同分块策略:
961 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
962 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
963 //   K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
964 //
965 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
966
967 // ---- SGEMV K32: 基础 warp-per-row ----
968 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
969 // grid(M/4), 每个 warp 负责一行
970 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
971 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
972 // Block: (32, 4, 1), 每行 1 warp
973 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
974 // source: LeetCode/kernels/sgemv/sgemv.cu
975 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
976     int tx = threadIdx.x; // 0~31
977     int ty = threadIdx.y; // 0~3
978     int lane = tx % kWarpSize; // 0~31
979     int m = blockIdx.x * blockDim.y + ty; // 全局行号
980     if (m < M) {
981         float sum = 0.0f;
982         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
983         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
984         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
985 #pragma unroll
986         for (int w = 0; w < NUM_ITERS; ++w) {
987             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
988             int k = w * kWarpSize + lane;
989             sum += a[m * K + k] * x[k];
990         }
991         sum = warp_reduce_sum<kWarpSize>(sum);
992         // 每个 warp 处理一行, lane 0 写回结果
993         if (lane == 0)
994             y[m] = sum;
995     }
996 }
997
998 // ---- SGEMV K128: float4 向量化 ----
999 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
1000 // Grid: ((M + 3) / 4, 1, 1)
1001 // Block: (32, 4, 1)
1002 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
1003 // source: LeetCode/kernels/sgemv/sgemv.cu
1004 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
1005     int tx = threadIdx.x; // 0~31
1006     int ty = threadIdx.y; // 0~3
1007     int lane = tx % kWarpSize; // 0~31

```

```

1008 int m = blockDim.y * blockIdx.x + ty;
1009
1010 if (m < M) {
1011     float sum = 0.0f;
1012     // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1013     const int NUM_ITERS = ((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1014 #pragma unroll
1015     for (int w = 0; w < NUM_ITERS; ++w) {
1016         int k = (w * kWarpSize + lane) * 4;
1017         float4 reg_x = FLOAT4(x[k]);
1018         float4 reg_a = FLOAT4(a[m * K + k]);
1019         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1020                reg_a.w * reg_x.w);
1021     }
1022     sum = warp_reduce_sum<kWarpSize>(sum);
1023     if (lane == 0)
1024         y[m] = sum;
1025 }
1026 }
1027
1028 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1029 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1030 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1031 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1032 // Block: (32, 4, 1)
1033 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1034 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1035 template <const int kRowPerWarp = 2>
1036 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1037     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1038     int tx = threadIdx.x;
1039     int ty = threadIdx.y;
1040     int lane = tx % kWarpSize;
1041     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1042     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1043     if (m < M) {
1044         float sum = A[m * K + k] * x[k];
1045         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1046         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1047         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1048         if (k == 0)
1049             y[m] = sum;
1050     }
1051 }
1052
1053 // =====
1054 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1055 // =====
1056 // 面试要点 (GEMM 优化五层金字塔):
1057 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1058 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1059 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1060 //   load/store 指令数 Level 4 — Tensor Core (MMA
1061 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1062 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1063 //
1064 // 计算密度递进:
1065 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1066 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1067 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1068
1069 // =====
1070 // Phase 7a: SGEMM (非 Tensor Core 路径)

```

```

1071 // =====
1072
1073 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
1074 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1075 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1076 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1077 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1078 // Block: (32, 32, 1), 1024 线程
1079 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1080 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1081     constexpr int BM = 32; // vec 版: 32x4 = 128
1082     constexpr int BN = 32; // vec 版: 32x4 = 128
1083     constexpr int BK = 32;
1084     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1085
1086     int bx = blockIdx.x;
1087     int by = blockIdx.y;
1088     int tx = threadIdx.x;
1089     int tid = threadIdx.y * blockDim.x + tx;
1090
1091     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1092     // 技巧: 一般来说 “/” 表示线程不是连续排布的, “%” 表示线程是连续排布的
1093     // 因此, 在需要考虑连续访问的维度使用 “%”, 比如, 连续的线程访问列方向连续的元素
1094     // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
1095     // 线程不连续访问 M 维度 → 用 /;
1096     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1097     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1098     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1099     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1100     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1101     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1102
1103     float sum = 0.f; // 遍历完整的K, slice K;
1104     // 这里不用pragma unroll, 因为K不是编译器常量, 编译器无法展开循环
1105     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1106         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1107         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1108         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1109         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1110         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1111         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1112         __syncthreads(); // 确保整个 smem tile 加载完毕
1113
1114     #pragma unroll
1115     for (int k = 0; k < BK; ++k) {
1116         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1117         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1118         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1119     }
1120     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1121 }
1122 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1123 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1124 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1125 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1126 }
1127
1128 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1129 // 在 Level 1 基础上引入两层优化:
1130 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1131 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1132 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1133 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major

```

```

1134 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4×4=16 个 C 元素
1135 // 1024 × 16 = 16384 = 128 × 128 ✓
1136 //
1137 // 线程到 4×4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1138 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1139 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1140 //
1141 // 加载映射 (每线程 4 个元素, float4):
1142 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8×4=32 列 ✓
1143 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32×4=128 列 ✓
1144 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1145 //
1146 // △ Bank Conflict 提示 (面试加分点):
1147 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1148 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1149 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1150 //
1151 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1152 // Block: (32, 32, 1), 1024 线程
1153 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1154 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1155 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1156     constexpr int BM = 128;
1157     constexpr int BN = 128;
1158     constexpr int BK = 32;
1159     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1160     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1161
1162     int bx = blockIdx.x;
1163     int by = blockIdx.y;
1164     int tx = threadIdx.x;
1165     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1166
1167     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1168     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1169     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1170     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1171     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1172     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1173
1174     int load_gmem_a_m = by * BM + load_smem_a_m;
1175     int load_gmem_b_n = bx * BN + load_smem_b_n;
1176
1177     // 4×4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1178     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1179     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1180     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1181     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1182     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1183
1184     float sum[4][4] = {0.f};
1185     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1186         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1187         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1188         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1189         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1190         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1191         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1192         __syncthreads();
1193     }
1194     #pragma unroll
1195     for (int k = 0; k < BK; ++k) {
1196         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4×4=16 次 FMA

```

```

1197     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1198                       s_a[comp_smem_a_m_base + 1][k],
1199                       s_a[comp_smem_a_m_base + 2][k],
1200                       s_a[comp_smem_a_m_base + 3][k]};
1201     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1202                       s_b[k][comp_smem_b_n_base + 1],
1203                       s_b[k][comp_smem_b_n_base + 2],
1204                       s_b[k][comp_smem_b_n_base + 3]};
1205 #pragma unroll
1206     for (int i = 0; i < 4; ++i) {
1207 #pragma unroll
1208         for (int j = 0; j < 4; ++j) {
1209             sum[i][j] += a_vals[i] * b_vals[j];
1210         }
1211     }
1212 }
1213 __syncthreads();
1214 }
1215
1216 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1217 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1218 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1219 #pragma unroll
1220 for (int i = 0; i < 4; ++i) {
1221     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1222     float4 reg_c;
1223     reg_c.x = sum[i][0];
1224     reg_c.y = sum[i][1];
1225     reg_c.z = sum[i][2];
1226     reg_c.w = sum[i][3];
1227     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1228 }
1229 }
1230
1231 // =====
1232 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1233 // =====
1234 // 面试要点:
1235 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1236 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1237 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1238 //     寄存器)
1239 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1240 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1241 //
1242 // ★ TN 布局详解 (面试高频考点):
1243 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1244 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1245 //   N = Normal (列优先, BLAS 原生格式)
1246 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1247 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1248 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1249 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1250 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1251 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1252 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1253 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1254 //
1255 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1256 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1257 //   N = col-major (列优先, BLAS native = normal)
1258 //
1259 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]

```

```

1260 // - A: row-major [M, K], B: row-major [K, N]
1261 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1262 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1263 //
1264 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1265 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1266 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1267 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1268 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1269 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1270 //
1271 // WGMMA (Warp Group MMA, Hopper+):
1272 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1273 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1274 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1275 // threads) 做计算
1276 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1277 //
1278 // =====
1279 // Phase 7b-1: MMA PTX 宏定义
1280 // =====
1281 //
1282 // ---- gmem → smem: cp.async ----
1283 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1284 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1285 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1286 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1287 // - wait_all: 等价于 commit_group + wait_group 0。
1288 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1289 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1290 #define CP_ASYNC_WAIT_GROUP(n) \
1291     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1292 //
1293 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1294 #define CP_ASYNC_CG(dst, src, bytes) \
1295     asm volatile( \
1296         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1297         "l"(src), "n"(bytes))
1298 //
1299 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1300 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1301 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1302 // aligned: 要求 128-bit 对齐, trans: 转置加载
1303 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1304     asm volatile( \
1305         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1306         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1307         : "r"(addr))
1308 //
1309 #define LDMATRIX_X2(R0, R1, addr) \
1310     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1311         : "=r"(R0), "=r"(R1) \
1312         : "r"(addr))
1313 //
1314 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1315 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1316 #define LDMATRIX_X2_T(R0, R1, addr) \
1317     asm volatile( \
1318         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1319         : "=r"(R0), "=r"(R1) \
1320         : "r"(addr))
1321 //
1322 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----

```

```

1323 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1324 // row.col: A 是 row-major, B 是 col-major
1325 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1326 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1327 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1328 #define MMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1329     asm volatile( \
1330         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1331         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1332         : "=r"(RD0), "=r"(RD1) \
1333         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1334         "r"(RC1))
1335
1336 // ---- MMA 辅助函数 ----
1337 // div_ceil: 整数除法向上取整
1338 #define HOST_DEVICE_INLINE __device__ __host__ inline
1339 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1340     return (a % b != 0) ? (a / b + 1) : (a / b);
1341 }
1342
1343 // =====
1344 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1345 // =====
1346 // 面试重点 — Tile Hierarchy:
1347 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1348 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1349 //   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1350 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1351 //           → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1352 //
1353 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1354 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1355 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1356 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1357 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1358 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1359 //
1360 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1361 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1362 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1363 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1364 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1365 //
1366 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1367 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1368 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1369 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1370 //
1371 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1372 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1373 //
1374 //       N / bx → 0           1           2           3           ...
1375 //       M ↓
1376 //       by = 0   | SM0 | SM1 | SM2 | SM3 | ...
1377 //                | A 0,B0 | A 0,B1 | A 0,B2 | A 0,B3 |
1378 //
1379 //
1380 //   同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1381 //   推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1382 //   L2 更容易被其他 B tile 挤占。
1383 //
1384 //   Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1385 //   下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:

```

```

1386 //
1387 //      N / local x → 0      1
1388 //      M ↓
1389 //      by = 0      

|     |     |
|-----|-----|
| SM0 | SM1 |
| SM2 | SM3 |

 → A 0; B^T 0, B^T 1
1390 //
1391 //      by = 1      

|     |     |
|-----|-----|
| SM2 | SM3 |
|-----|-----|

 → A 1; B^T 0, B^T 1
1392 //
1393 //
1394 // 对于这个 2x2 窗口：同一行横向复用 A (SM0/SM1、SM2/SM3)，同一列纵向复用 B
1395 // (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2，但缩小它会缩短再次访问相同 bx 的距离。
1396 // 此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会，不保证 CTA 的 z 顺序、
1397 // 缓存驻留或 L2 hit。
1398 //
1399 // Launch (kBlockSwizzle=false, 当前默认路径)：
1400 // grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1401 // Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid)：
1402 // tiles_n = ceil(N / BN), S = ceil(N / 2048)
1403 // grid = (ceil(tiles_n / S), ceil(M / BM), S)
1404 // bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1405 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1406 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1407 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1408 // 对齐; ceil 只保证逻辑 tile 编号不遗漏, 并不使部分尾 tile 变为安全。
1409 // Block: (256, 1, 1), 8 warps
1410 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1411 // =====
1412 template <const int kMmaM = 16,          // MMA atom M dim (m16n8k16)
1413           const int kMmaN = 8,          // MMA atom N dim
1414           const int kMmaK = 16,         // MMA atom K dim, also BK tile = kMmaK
1415           const int kMmaTileM = 2,      // warps along M, 2 → warp tile M = 32
1416           const int kMmaTileN = 4,      // warps along N, 4 → warp tile N = 32
1417           const int kValTileM = 4,      // value-repeat along M, BM = 16*2*4 = 128
1418           const int kValTileN = 4,      // value-repeat along N, BN = 8*4*4 = 128
1419           const int kStages = 3,        // cp.async pipeline depth
1420           const int kBlockSwizzle = 0>  // 1 enables 3D grid swizzle for L2 locality
1421 __global__ void __launch_bounds__(256)
1422   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1423   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1424   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1425   // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1426   // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1427   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1428   const int by = blockIdx.y;
1429   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1430   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1431   constexpr int BK = kMmaK; // 16
1432
1433   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1434   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1435   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1436   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1437   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1438   extern __shared__ half smem[];
1439   half *s_a = smem;
1440   half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1441   constexpr int s_a_stage_offset = BM * BK; // 128*16
1442   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1443   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1444   const int warp_id = tid / kWarpSize; // 0~7
1445   const int lane_id = tid % kWarpSize; // 0~31
1446   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1447   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0-MMA7的
1448   //      N direction (warp_n)

```

```

1449 //          0      1      2      3
1450 // M direction (warp_m) 0 MMA0   MMA2   MMA4   MMA6
1451 //          1 MMA1   MMA3   MMA5   MMA7
1452 // MMA的排布方式不是唯一的，现在这样排是因为结合MMA/VAL Tile之后，刚好能覆盖整个
1453 // C Tile[128,128]。只要调整MMA/VAL Tile，这里的排布方式也可以跟着调整。要注意的
1454 // 是，MMA0-7逻辑上是可以认为是并行执行的，各自的计算结果累计加到对应的C Tile位置上。
1455 const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1456 const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1457
1458 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1459 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1460 int load_smem_a_m = tid / 2; // 0~127
1461 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1462 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1463 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1464 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1465 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1466 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1467     return;
1468
1469 // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1470 // CUDA中每个block能放的线程数是有上限的 (warp数量有限)，一般为4或者8个warps。为了能处理更大的C tile，
1471 // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
1472 // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
1473 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1474
1475 // CVTA: 一次转换 smem 基地址，避免每次 cp.async 都做转换
1476 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1477 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1478
1479 // 预加载前 (kStages-1) 个 stage
1480 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1481 #pragma unroll
1482 for (int k = 0; k < (kStages - 1); ++k) {
1483     int load_gmem_a_k = k * BK + load_smem_a_k;
1484     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1485     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1486         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1487     );
1488     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1489
1490     int load_gmem_b_k = k * BK + load_smem_b_k;
1491     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1492     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1493     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1494         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1495     );
1496     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1497
1498     CP_ASYNC_COMMIT_GROUP();
1499 }
1500
1501 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1502 __syncthreads();
1503
1504 // 统一循环: k 从 0 开始，每次迭代负责 tile k (加载 + 计算合并为单循环)
1505 const int NUM_K_TILES = div_ceil(K, BK);
1506 // 此处不用pragma unroll，因为 K 不是编译期常量，因此NUM_K_TILES 也不是编译期常量
1507 for (int k = 0; k < NUM_K_TILES; ++k) {
1508     int smem_sel = k % kStages; // 计算 tile k 的 stage
1509     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1510
1511     // 条件加载: 预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经

```

```

1512 // load了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1513 // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1514 // (row-major, 内维连续的是 K)
1515 if (k + kStages - 1 < NUM_K_TILES) {
1516     int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1517     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1518     int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1519     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1520
1521     uint32_t load_smem_a_ptr =
1522         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1523             load_smem_a_m * BK + load_smem_a_k) *
1524             sizeof(half));
1525     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1526
1527     uint32_t load_smem_b_ptr =
1528         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1529             load_smem_b_n * BK + load_smem_b_k) *
1530             sizeof(half));
1531     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1532     CP_ASYNC.COMMIT_GROUP();
1533 }
1534
1535 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1536 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1537 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1538 uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1539 uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1540
1541 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1542 #pragma unroll
1543 for (int i = 0; i < kValTileM; ++i) {
1544     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1545     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1546     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1547     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1548     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1549     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1550     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1551     uint32_t lane_smem_a_ptr =
1552         (smem_a_base_ptr +
1553             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1554             sizeof(half));
1555     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1556 }
1557
1558 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1559 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1560 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1561 #pragma unroll
1562 for (int j = 0; j < kValTileN; ++j) {
1563     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1564     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1565     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1566     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1567     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1568     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1569     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1570     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1571     uint32_t lane_smem_b_ptr =
1572         (smem_b_base_ptr +
1573             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1574             sizeof(half));

```

```

1575     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1576 }
1577
1578 // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次, 在N方向重复kValTileN(4)次
1579 #pragma unroll
1580 for (int i = 0; i < kValTileM; ++i) {
1581     #pragma unroll
1582     for (int j = 0; j < kValTileN; ++j) {
1583         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1584                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1585                 RB[j][0], RB[j][1], // B fragment
1586                 RC[i][j][0], RC[i][j][1]);
1587     }
1588 }
1589
1590 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1591 if (k + kStages - 1 < NUM_K_TILES) {
1592     CP_ASYNC_WAIT_GROUP(kStages - 2);
1593 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1594     CP_ASYNC_WAIT_GROUP(0);
1595 }
1596 __syncthreads();
1597 }
1598
1599 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1600 {
1601     for (int i = 0; i < kValTileM; ++i) {
1602         // RC[kValTileM][kValTileN][2] 中 RC[...] [...] [2] 中保存了2个 uint32
1603         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...] [...] [0]
1604         // 和RC[...] [...] [1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1605         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1606         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1607         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1608         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1609         // =====
1610         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1611         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1612         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1613         //
1614         //      cols: c0-1    c2-3    c4-5    c6-7
1615         //  r0: T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1616         //  r1: T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1617         //  r2: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1618         //  ...          |
1619         //  r7: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1620         //  r8: T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1621         //  r9: T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1622         // r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1623         //  ...          |
1624         // r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1625         //
1626         // Within a 4-lane group (e.g. T0-T3 for row 0):
1627         //   lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1628         //   lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1629         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1630         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1631         // =====
1632     #pragma unroll
1633     for (int j = 0; j < kValTileN; ++j) {
1634         RC0[j][0] = RC[i][j][0];
1635         RC1[j][0] = RC[i][j][1];
1636         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1637         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);

```

```

1638     RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1639     RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1640     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1641     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1642 }
1643 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1644 // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1645 //
1646 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1647 // |-----|-----|-----|-----|
1648 // | 0~3    | 0        | row 0     | row 8     |
1649 // | 4~7    | 1        | row 1     | row 9     |
1650 // | 8~11   | 2        | row 2     | row 10    |
1651 // | 12~15  | 3        | row 3     | row 11    |
1652 // | 16~19  | 4        | row 4     | row 12    |
1653 // | 20~23  | 5        | row 5     | row 13    |
1654 // | 24~27  | 6        | row 6     | row 14    |
1655 // | 28~31  | 7        | row 7     | row 15    |
1656 //
1657 if (lane_id % 4 == 0) {
1658     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1659     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
1660     // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}, 对应 RC0/RC1 (+8) 的行号, 表示2个8x8矩阵的行号
1661     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
1662 #pragma unroll
1663     for (int j = 0; j < kValTileN; ++j) {
1664         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1665         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
1666         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
1667         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
1668         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
1669         // 128-bit store: 一次写入 8 个 half
1670         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1671             *reinterpret_cast<float4*>(&RC0[j][0]);
1672         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1673             *reinterpret_cast<float4*>(&RC1[j][0]);
1674     }
1675 }
1676 }
1677 }
1678 }
1679
1680 // =====
1681 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1682 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1683 // =====
1684 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1685 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1686 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way) 。
1687 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1688 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1689 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1690 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1691 //   rows 8~11: repeat rows 0~3 pattern
1692 //   rows 12~15: repeat rows 4~7 pattern
1693 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free) 。
1694 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1695 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1696 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1697 //
1698 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1699 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1700

```

```

1701 // ---- Swizzle 辅助函数 ----
1702
1703 // swizzle_permuted_j: 对 smem 列索引 j 做 XOR 置换, 消除 bank conflict.
1704 // i: row index; j: col index.
1705 // e.g kColStride = 16, kStep = 8 -> load 8 half as 128 bits memory issue.
1706 // 公式: ((j / kStep) ^ (i / 4)) % (kColStride / kStep) * kStep
1707 // 用位运算展开 (kStep=8): (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3
1708 // 限制: kColStride ≤ 16 (BK ≤ 16), kStep ∈ {4, 8}, kColStride % kStep == 0
1709 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1710 template <const int kColStride = 16, const int kStep = 8>
1711 static __device__ __forceinline__ int swizzle_permuted_j(int i, int j) {
1712     // for col_stride > 16, we have to permute it using col major ZigZag order.
1713     // e.g, A smem logical layout [Br,d]=[Br,64] -> store layout [4][Br][16].
1714     static_assert(kColStride <= 16, "kColStride must <= 16");
1715     // swizzle: ((int(j / kStep) ^ int(i / 4)) % int(kColStride / kStep)) * kStep;
1716     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1717     static_assert(kColStride % kStep == 0,
1718         "kColStride must be multiple of kStep.");
1719     if constexpr (kStep == 8) {
1720         // j >> 3: 表示8个half(=16 bytes)数据为一个chunk; i >> 2: 表示4行为一组
1721         // kColStride >> 3: 按照kStep=8计算chunk idx, kColStride=16 → {0, 1}
1722         // 最后的 << 3: 将chunk idx转换为实际的列偏移量 (8个half/chunk), {0, 8}
1723         return (((j >> 3) ^ (i >> 2)) % (kColStride >> 3)) << 3;
1724     } else {
1725         static_assert(kStep == 4);
1726         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1727     }
1728 }
1729
1730 // swizzle_A: A 矩阵专用封装 (kMmaAtomK=16, kStep=8)。
1731 // 16 行 (一个 MMA atom 的 M 维) 内的 swizzle pattern:
1732 // 8个half=16 bytes=128 bits=4 banks (32 bits/bank) 组成一个phase,
1733 // 触发一次合并的memory transaction. 这里的col=16的swizzle, 相当于
1734 // TMA中的SWIZZLE_32B pattern.
1735 // -----
1736 // -col 0~16, step 8--
1737 // -----
1738 // | row 0 | (0, 8) |
1739 // | row 1 | (0, 8) |
1740 // | row 2 | (0, 8) |
1741 // | row 3 | (0, 8) |
1742 // -----
1743 // | row 4 | (8, 0) |
1744 // | row 5 | (8, 0) |
1745 // | row 6 | (8, 0) |
1746 // | row 7 | (8, 0) |
1747 // -----
1748 // | row 8 | (0, 8) |
1749 // | row 9 | (0, 8) |
1750 // | row 10 | (0, 8) |
1751 // | row 11 | (0, 8) |
1752 // -----
1753 // | row 12 | (8, 0) |
1754 // | row 13 | (8, 0) |
1755 // | row 14 | (8, 0) |
1756 // | row 15 | (8, 0) |
1757 // -----
1758 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1759 template <const int kMmaAtomK = 16>
1760 static __device__ __forceinline__ int swizzle_A(int i, int j) {
1761     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1762 }
1763

```

```

1764 // swizzle_B: B 矩阵专用封装 (与 A 相同的 pattern) 。
1765 // B^T smem 布局 [BN][BK]=[128][16] 在 BK 维做 swizzle,
1766 // pattern 与 A 完全相同 (kMmaAtomK=16, kStep=8) 。
1767 // source: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1768 template <const int kMmaAtomK = 16>
1769 static __device__ __forceinline__ int swizzle_B(int i, int j) {
1770     return swizzle_permuted_j<kMmaAtomK, 8>(i, j);
1771 }
1772
1773 // =====
1774 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1775 //           + Register Double Buffering (kValTileK=2, BK=32 统一 tile)
1776 // =====
1777 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1778 //   - kValTileK=2: 每个 BK tile 包含 2 个 kMmaK slice (BK = kMmaK * kValTileK = 32)
1779 //   - BK=32 统一 tile: kMmaK=0 和 kMmaK=1 数据连续存放在同一个 BK=32 宽的 smem tile 中
1780 //   - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换
1781 //   - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
1782 //
1783 // ★ smem 布局:
1784 //   s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
1785 //   s_b: 紧接 s_a 之后
1786 //   每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +kMmaK(=16)
1787 //
1788 // ★ 寄存器双缓冲时间线 (每 k 迭代内):
1789 //   Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1790 //   Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1791 //   Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1792 //   Step 4: adaptive wait + __syncthreads
1793 //   Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1794 //
1795 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1796 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1797 // Block: (256, 1, 1), 8 warps
1798 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1799           const int kMmaN = 8,           // MMA atom N dim
1800           const int kMmaK = 16,          // MMA atom K dim
1801           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1802           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1803           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1804           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1805           const int kValTileK = 2,       // MMA_K slices per BK tile, BK = kMmaK*2 = 32
1806           const int kStages = 3,         // cp.async pipeline depth
1807           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
1808 __global__ void __launch_bounds__(256)
1809     hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1810     static_assert(kValTileK == 2, "Only support kValTileK=2 for register double buffering");
1811     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1812     static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1813     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1814     const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1815     const int by = blockIdx.y;
1816     constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1817     constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1818     constexpr int BK = kMmaK * kValTileK;             // 16*2=32
1819
1820     // kStages stages, each with BM×BK for A, BN×BK for B
1821     // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK=32 宽 tile 中
1822     extern __shared__ half smem[];
1823     half *s_a = smem;
1824     half *s_b = smem + kStages * BM * BK;
1825     constexpr int s_a_stage_offset = BM * BK;
1826     constexpr int s_b_stage_offset = BN * BK;

```

```

1827
1828 const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1829 const int warp_id = tid / kWarpSize;
1830 const int lane_id = tid % kWarpSize;
1831 const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1832 const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1833
1834 // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1835 // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1836 // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 虽然BK=16*2=32, 在后续的kValTileK循环中
1837 // 会加上 k_step*kMmaK=0/16, 最终得到 smem 中的列偏移 0/8/16/24, 正好覆盖 BK=32
1838 int load_smem_a_m = tid / 2; // 0~127
1839 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1840 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1841 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1842 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1843 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1844 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1845     return;
1846
1847 // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16, 4*8]=[32, 32].
1848 // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1849 // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128, 128]的C block tile, 这就是Value Tile的作用。
1850 // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNK的概念。
1851 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1852
1853 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1854 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1855
1856 // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1857 #pragma unroll
1858 for (int k = 0; k < (kStages - 1); ++k) {
1859     #pragma unroll
1860     for (int k_step = 0; k_step < kValTileK; ++k_step) {
1861         int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
1862         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1863         int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
1864         int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1865
1866         uint32_t load_smem_a_ptr =
1867             (smem_a_base_ptr +
1868              (k * s_a_stage_offset + load_smem_a_m * BK +
1869               k_step * kMmaK +
1870               swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1871              sizeof(half));
1872         CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1873
1874         uint32_t load_smem_b_ptr =
1875             (smem_b_base_ptr +
1876              (k * s_b_stage_offset + load_smem_b_n * BK +
1877               k_step * kMmaK +
1878               swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1879              sizeof(half));
1880         CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1881     }
1882     CP_ASYNC_COMMIT_GROUP();
1883 }
1884
1885 CP_ASYNC_WAIT_GROUP(kStages - 2);
1886 __syncthreads();
1887
1888 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1889 uint32_t RA[2][kValTileM][4];

```

```

1890 uint32_t RB[2][kValTileN][2];
1891 int reg_st_idx = 0; // write target for ldmatrix
1892 int reg_ld_idx = 1; // read source for MMA
1893
1894 // Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) -> reg[0]
1895 // 此时, reg_st_idx = 0, 对0位置的寄存器buffer做初始化。
1896 #pragma unroll
1897 for (int i = 0; i < kValTileM; ++i) {
1898     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1899     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1900     int lane_smem_a_k = (lane_id / 16) * 8;
1901     uint32_t lane_smem_a_ptr =
1902         (smem_a_base_ptr +
1903          (0 * s_a_stage_offset + lane_smem_a_m * BK +
1904           swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1905           sizeof(half));
1906     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1907                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1908                 lane_smem_a_ptr);
1909 }
1910 #pragma unroll
1911 for (int j = 0; j < kValTileN; ++j) {
1912     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1913     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1914     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1915     uint32_t lane_smem_b_ptr =
1916         (smem_b_base_ptr +
1917          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1918           swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1919           sizeof(half));
1920     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1921                 lane_smem_b_ptr);
1922 }
1923
1924 // 统一循环: k 从 0 开始, 条件 G->S / S->R / wait 处理所有边界情况
1925 // BK = kMmaK * kValTileK = 32, 每个 k 迭代覆盖 BK=32 个 K 元素
1926 const int NUM_K_TILES = div_ceil(K, BK);
1927 for (int k = 0; k < NUM_K_TILES; ++k) {
1928     int smem_sel = k % kStages;
1929     int smem_sel_next = (k + kStages - 1) % kStages;
1930
1931     // G->S: 条件预取 stage(k+kStages-1) 全部 k_step slice
1932     if (k + kStages - 1 < NUM_K_TILES) {
1933 #pragma unroll
1934         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1935             int load_gmem_a_k =
1936                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
1937             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1938             int load_gmem_b_k =
1939                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
1940             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1941
1942             uint32_t load_smem_a_ptr =
1943                 (smem_a_base_ptr +
1944                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1945                   k_step * kMmaK +
1946                  swizzle_A<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1947                  sizeof(half));
1948             CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1949
1950             uint32_t load_smem_b_ptr =
1951                 (smem_b_base_ptr +
1952                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +

```

```

1953         k_step * kMmaK +
1954         swizzle_B<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1955         sizeof(half));
1956     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1957 }
1958 CP_ASYNC_COMMIT_GROUP();
1959 }
1960
1961 // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
1962 // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1963 // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1964
1965 #pragma unroll
1966 for (int k_step = 0; k_step < kValTileK; ++k_step) {
1967     reg_st_idx ^= 1; // 0 -> 1; 1 -> 0
1968     reg_ld_idx ^= 1; // 1 -> 0; 0 -> 1
1969
1970     if (k_step + 1 < kValTileK) {
1971         int smem_k_offset = (k_step + 1) * kMmaK;
1972 #pragma unroll
1973         for (int i = 0; i < kValTileM; ++i) {
1974             int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1975             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1976             int lane_smem_a_k = (lane_id / 16) * 8;
1977             uint32_t lane_smem_a_ptr =
1978                 (smem_a_base_ptr +
1979                  (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
1980                   smem_k_offset +
1981                   swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1982                  sizeof(half));
1983             LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1984                         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1985                         lane_smem_a_ptr);
1986         }
1987 #pragma unroll
1988         for (int j = 0; j < kValTileN; ++j) {
1989             int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1990             int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1991             int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1992             uint32_t lane_smem_b_ptr =
1993                 (smem_b_base_ptr +
1994                  (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
1995                   smem_k_offset +
1996                   swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1997                  sizeof(half));
1998             LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1999                         lane_smem_b_ptr);
2000         }
2001     }
2002
2003 #pragma unroll
2004     for (int i = 0; i < kValTileM; ++i) {
2005 #pragma unroll
2006         for (int j = 0; j < kValTileN; ++j) {
2007             HMMA16816(RC[i][j][0], RC[i][j][1],
2008                       RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
2009                       RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
2010                       RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
2011                       RC[i][j][0], RC[i][j][1]);
2012         }
2013     }
2014 }
2015

```

```

2016 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
2017 if (k + kStages - 1 < NUM_K_TILES) {
2018     CP_ASYNC_WAIT_GROUP(kStages - 2);
2019 } else {
2020     CP_ASYNC_WAIT_GROUP(0);
2021 }
2022 __syncthreads();
2023
2024 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
2025 if (k + 1 < NUM_K_TILES) {
2026     smem_sel = (k + 1) % kStages; // old smem_sel + 1
2027 #pragma unroll
2028     for (int i = 0; i < kValTileM; ++i) {
2029         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2030         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2031         int lane_smem_a_k = (lane_id / 16) * 8;
2032         uint32_t lane_smem_a_ptr =
2033             (smem_a_base_ptr +
2034              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2035               swizzle_A<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2036               sizeof(half));
2037         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2038                     RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2039                     lane_smem_a_ptr);
2040     }
2041 #pragma unroll
2042     for (int j = 0; j < kValTileN; ++j) {
2043         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2044         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2045         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2046         uint32_t lane_smem_b_ptr =
2047             (smem_b_base_ptr +
2048              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2049               swizzle_B<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2050               sizeof(half));
2051         LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2052                     lane_smem_b_ptr);
2053     }
2054 }
2055 }
2056
2057 // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
2058 // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间, RA[2][4][4]正好可以复用
2059 for (int i = 0; i < kValTileM; ++i) {
2060 #pragma unroll
2061     for (int j = 0; j < kValTileN; ++j) {
2062         RA[0][j][0] = RC[i][j][0];
2063         RA[1][j][0] = RC[i][j][1];
2064         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2065         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2066         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2067         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2068         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2069         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2070     }
2071     if (lane_id % 4 == 0) {
2072         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2073         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2074 #pragma unroll
2075         for (int j = 0; j < kValTileN; ++j) {
2076             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2077             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2078             int store_gmem_c_addr_0 =

```

```

2079         store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2080     int store_gmem_c_addr_1 =
2081         (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2082     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2083         *reinterpret_cast<float4*>(&RA[0][j][0]);
2084     *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2085         *reinterpret_cast<float4*>(&RA[1][j][0]);
2086 }
2087 }
2088 }
2089 }
2090
2091 // =====
2092 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2093 // =====
2094 // 面试要点 (CuTe vs 手写 MMA PTX) :
2095 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2096 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2097 // - 与手写 MMA PTX (Phase 7b) 的对比:
2098 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2099 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2100 //     cute::copy / cute::gemm 自动展开为高效指令序列
2101 // - 核心抽象 ("CuTe 五要素") :
2102 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2103 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2104 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2105 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2106 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2107 //
2108 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2109 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2110 //     → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2111 //     → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)
2112 //     → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2113 //
2114 // 调参口诀 (面试速记) :
2115 //   BM/BN/BK 越大 → smem 越大 → occupancy 越低, 但 K 循环更少 → 指令开销更低
2116 //   Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape, 而是把一维 offset
2117 //   重新解释为一个二维逻辑空间, 再把二维坐标映射回 bank-conflict-free 的物理 offset:
2118 //     1. 连续的 2^M 个元素组成二维空间中的一个基本元素; (元素宽度)
2119 //     2. 连续的 2^S 个基本元素组成一行; (列数)
2120 //     3. 二维空间包含 2^B 行; (行数)
2121 //     4. 对二维坐标做列置换: icol' = irow ^ icol;
2122 //     5. 保留基本元素内部的低 M 位, 再将置换后的二维坐标编码回 offset。
2123 //   因此 M 描述基本元素宽度, S 描述二维空间的列数, B 描述二维空间的行数。
2124 //   CuTe 的位级实现等价于:
2125 //     offset' = offset ^ ((offset & YYY) >> S),
2126 //     YYY = ((1 << B) - 1) << (M + S)。
2127 //   对 Swizzle<3,3,3>: 每个基本元素有 2^3=8 个值, 每行有 2^3=8 个基本元素,
2128 //   二维空间有 2^3=8 行; 元素地址位 [6:8] XOR 到 [3:5], 低 3 位保持不变。
2129 //   一个完整 swizzle 周期覆盖 2^(M+S+B)=2^9=512 个元素 (FP16 下为 1024B)。
2130 //   kStage: 2=最低延迟, 3/4=更好隐藏延迟 → 权衡 smem 占用
2131 //   EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局, 决定 TiledMMA 的线程排布
2132 //   与逻辑 tile 组织; MMA atom 越多, 需要的线程数就越多
2133 //
2134 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别:
2135 //   - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2136 //   - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2137 //   - CuTe 优势: 类型安全, 编译器可在编译期推导映射; 布局变更通常只需修改类型定义
2138 //
2139 // ★ 本实现的 Tile 配置:
2140 //   - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2141 //   - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。

```

```

2142 // - TiledMMA 的逻辑 MMA tile 是 32×32×16, G2S/S2R copy 再把它映射到更大的 BM×BN tile;
2143 //   BN=256 不是由“8×2×16”直接推导出的 MMA tile, 而是本 wrapper 选择的 B tile 尺寸。
2144 // - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排, 针对 ldmatrix
2145 //   的访问模式降低 bank conflict; 512 是 swizzle 周期, 不等于 A atom 的逻辑元素数。
2146 // - Threads: size(MMA{}) = 128 (4 warps × 2×2 EU repeat) 。
2147 // - Dynamic smem: Stage=2 时 A[128×32] + B[256×32] 共 49,152 bytes (约 48 KiB) 。
2148 //
2149 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2150 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2151 // =====
2152
2153 #if defined(NOTES_V2_ENABLE_CUTE)
2154
2155 #include <cute/tensor.hpp>
2156
2157 // =====
2158 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2159 // =====
2160 // TN 布局: C[M×N] = A[M×K] × B^T[N×K]
2161 // - A[M][K] row-major, B^T[N][K] row-major (等价 B col-major [K×N])
2162 // - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2163 //
2164 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽, 不是一条完整计算链) :
2165 // 1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async), 填满 stage。
2166 // 2. 主循环 over K tiles:
2167 //   a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2168 //   b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2169 //   c. 在处理当前 tile 的首个 MMA slice 时, 异步提交下一个 tile 的 G→S;
2170 //   d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2171 // 3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store) 。
2172 //
2173 // 面试对比 — 手写 MMA vs CuTe 代码量:
2174 // 手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)
2175 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2176 template <typename T,                                // element type (half)
2177          int BM, int BN, int BK,                      // block tile: BM=128, BN=256, BK=32
2178          int kStage,                                  // cp.async pipeline depth (2)
2179          typename TiledMMA,                           // MMA: SM80_16x8x16_F16F16F16F16_TN, EURepeat=2x2x1
2180          typename G2SCopyA,                           // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2181          typename G2SCopyB,                           // G→S B: same as G2SCopyA
2182          typename SmemLayoutA,                        // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2183          typename SmemLayoutB,                        // smem B: same atom → (BN,BK,kStage)
2184          typename SmemLayoutC,                        // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2185          typename S2RCopyAtomA,                       // S→R A: SM75_U32x4_LDSTM_N (ldmatrix.x4)
2186          typename S2RCopyAtomB,                       // S→R B: same as S2RCopyAtomA
2187          typename R2SCopyAtomC,                       // R→S C: UniversalCopy<int> (32-bit store)
2188          typename S2GCopyAtomC,                       // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2189          typename S2GCopyC,                           // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2190          const int BlockSwizzle = 0>                 // 1 enables 3D grid swizzle for L2 locality
2191 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2192                                           int n, int k) {
2193     using namespace cute;
2194     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2195     static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");
2196
2197     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2198     extern __shared__ T shm_data[];
2199     T *Ashm = shm_data;
2200     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2201
2202     int idx = threadIdx.x;
2203     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2204     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;

```

```

2205 int iy = blockIdx.y; // M 方向 block 索引
2206 // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2207 // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2208 if (iy * BM >= m || ix * BN >= n) return;
2209
2210 // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2211 // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2212 Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2213                          make_stride(k, Int<1>{}));
2214 Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2215                          make_stride(k, Int<1>{}));
2216 Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2217                          make_stride(n, Int<1>{}));
2218
2219 // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2220 // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2221 // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2222 Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2223 Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2224 Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2225
2226 // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2227 auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2228 auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2229
2230 // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2231 TiledMMA tiled_mma;
2232 auto thr_mma = tiled_mma.get_slice(idx);
2233 auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2234 auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2235 auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2236 clear(tCrD); // 累加器清零
2237
2238 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2239 G2SCopyA g2s_tiled_copy_a;
2240 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2241 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2242 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2243
2244 G2SCopyB g2s_tiled_copy_b;
2245 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2246 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2247 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2248
2249 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2250 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2251 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2252 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2253 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2254 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2255
2256 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2257 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2258 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2259 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2260
2261 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2262 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2263 #pragma unroll
2264 for (int istage = 0; istage < kStage - 1; ++istage) {
2265     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2266               tAsA_copy(_, _, _, istage));
2267     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),

```

```

2268         tBsB_copy(_, _, _, istage));
2269     cp_async_fence();
2270     ++itile_to_read;
2271     ++ismem_write;
2272 }
2273 cp_async_wait<kStage - 2>();
2274 __syncthreads();
2275
2276 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2277 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2278 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2279 int num_k_tiles = k / BK;
2280 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2281 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2282 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2283
2284 #pragma unroll 1
2285 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2286     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2287     //
2288     // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2289     //   tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2290     //   - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2291     //     所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2292     //   - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2293     //     同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2294     //     在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2295     //     无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2296     //         for (i=0; i<kValTileM; ++i)
2297     //             for (j=0; j<kValTileN; ++j)
2298     //                 HMMMA16816(RC[i][j][0], RC[i][j][1], ...);
2299     //
2300     // CUTLASS 推导链 (mma_atom.hpp) :
2301     //   make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnK
2302     //   → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2303     //   → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2304     //   做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2305     //   K 被分为 BK/kMmaPK 个 perm-K slice
2306     //   随后按 AtomShape_MnK<2> (MMA_K_atom) 做 zipped_divide
2307     //   每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2308     //   → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2309     // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2310     int num_k_steps = size<2>(tCrA);
2311
2312 #pragma unroll 1
2313 for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2314     int k_step_next = (k_step + 1) % num_k_steps;
2315
2316     if (k_step == num_k_steps - 1) {
2317         // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2318         cp_async_wait<kStage - 2>();
2319         __syncthreads();
2320         ismem_read = (ismem_read + 1) % kStage;
2321     }
2322
2323     // S→R: 加载下一组 A/B fragment 到寄存器。
2324     //
2325     // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2326     //   cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2327     // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**:
2328     //   当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2329     //   让 S→R 延迟与 MMA 计算重叠。
2330     //   如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。

```

```

2331     cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2332               tCrA_view(_, _, k_step_next));
2333     cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2334               tCrB_view(_, _, k_step_next));
2335
2336     if (k_step == 0) {
2337         // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2338         if (itile_to_read < num_k_tiles) {
2339             cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2340                       tAsA_copy(_, _, _, ismem_write));
2341             cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2342                       tBsB_copy(_, _, _, ismem_write));
2343             ++itile_to_read;
2344             ismem_write = (ismem_write + 1) % kStage;
2345         }
2346         cp_async_fence();
2347     }
2348
2349     // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2350     // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2351     cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2352 }
2353 }
2354
2355 // Epilogue: D 寄存器 → shared memory → global memory。
2356 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2357 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2358 //   sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2359 //   只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2360 //   解释这同一段地址。当前 wrapper 的固定配置下:
2361 //   一个 A stage: 128 x 32 = 4096 个 half;
2362 //   C scratchpad: 32 x 32 x 4 = 4096 个 half。
2363 //   两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom
2364 //   是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2365 //   C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2366 //   检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2367 //
2368 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2369 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2370 // 用它写 C, S2G 也用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2371 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2372
2373 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2374 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2375 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2376 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2377 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2378 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2379 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2380 //
2381 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2382 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2383 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2384 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2385 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2386 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2387 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2388 // 角色, 而不是另一次 memory transfer。
2389 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2390 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2391 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2392 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2393 //   MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>

```

```

2394 //      → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2395 //      → make_tiled_copy_C 的 tiler = (32, 32)
2396 //      → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2397 //      → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2398 //      → 结果 shape 的 M/N remainder = _1, _1
2399 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2400 // “thread-level tensor 自动只剩 CPY” 这一条规则。_1 表示该逻辑 mode 的 extent
2401 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2402 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2403
2404 // S2G TiledCopy: shared memory → global memory (128-bit store)
2405 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2406 S2GCopyC s2g_tiled_copy_c;
2407 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2408 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2409 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2410 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2411 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2412 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2413 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2414 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2415
2416 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2417 //   group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2418 //       -> Tensor(CPY, (CPY_M, CPY_N))
2419 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。
2420 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系, 只是现在可以用一个坐标
2421 // i+j 访问这个组合 mode; 因此这里的 group_modes 更准确地说是“合并 mode”,
2422 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2423 // 从 type/shape 结构看, 结果确实是 Tensor(CPY, (CPY_M, CPY_N)); 但 grouped
2424 // mode 的 cardinality 是两个子 mode 的乘积, 因此 size<1>(…) 可作为一个标量
2425 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2426 // 的线性 logical coordinate, 不是在 C++ 中对二维 tuple 做加法, 更不是 raw
2427 // pointer offset; 最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2428 //
2429 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2430 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment, 原始形状为 (CPY, CPY_M, CPY_N);
2431 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置, 形状与源 Tensor 对齐;
2432 //   - 两者使用相同的 [1,3) 合并后, 源/目的 Tensor 仍保持相同的坐标空间,
2433 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2434 //
2435 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode, 因为它们还需要显式保留
2436 // shared-memory C scratchpad 的 pipeline 维度; step=size<3>(tCsC_r2s) 表示
2437 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2438 // 按 step 前进, 内层 j 选择当前 pipeline slot: 寄存器先写入 sC(_,0,0,j),
2439 // 再从同一个 slot 写回 global memory。
2440 // 当前 kSmemLayoutCBatch=4, 因此 step=4。代码未对 i+j 做尾部 predication,
2441 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除; 不应将
2442 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2443 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2444 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2445
2446 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2447 // 双层循环的语义 (注意 step 与 CPY_N 无关):
2448 //   group_modes<1,3> 之后, tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2449 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N, 即该线程需处理的 C fragment 总数。
2450 //   外循环以 step=4 为步长, 内循环 j=0..3 每次处理 1 个 fragment, 将其写入
2451 //   第 j 号 C scratchpad pipe slot, 再读出写回 global。
2452 //
2453 // 这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2454 // TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2455 // 通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2456 // CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。

```

```

2457 // 唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2458 //
2459 // 第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2460 // 各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2461 // 写入和读取命中同一段 shared memory。
2462 #pragma unroll
2463 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2464     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2465     #pragma unroll
2466     for (int j = 0; j < step; ++j) {
2467         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2468         // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2469         // 成输出元素类型 T 的临时 payload。
2470         //
2471         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2472         // dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2473         // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2474         // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2475         //
2476         // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2477         // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2478         // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2479         // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2480         // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2481         // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2482         // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2483         // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2484         //
2485         // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2486         // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2487         // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2488         // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2489         // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例
2490         // 保持同源实现的通用 staging 写法。
2491         auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2492         cute::copy(tCrC_r2sx(_, i + j), t);
2493         // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2494         // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2495         //
2496         // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2497         // r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>, T>
2498         // → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2499         // → src(t) 与 dst(tCsC_r2s(_, 0, 0, j)) 都是 rank-1 → 直接 copy_atom.call
2500         // → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2501         // 走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2502         // → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2503         //
2504         // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2505         // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2506         // - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2507         //   → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2508         // - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2509         //   → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2510         //   计算了当前线程每个 val 在 smem 中的物理 offset
2511         // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2512         // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2513         cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2514     }
2515     // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2516     // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2517     __syncthreads();
2518
2519     // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。

```

```

2520 #pragma unroll
2521 for (int j = 0; j < step; ++j) {
2522     // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2523     // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2524     // (如 st.global.v4.f32 或 st.global.b128)。
2525     //
2526     // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2527     //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>,T>
2528     //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2529     //   → src=tCsC_s2g(_,0,0,j) 与 dst=tCgC_s2gx(_,i+j) 都是 rank-1
2530     //   → 直接 copy_atom.call(src, dst)
2531     //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2532     //     → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2533     //     → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2534     //
2535     // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2536     //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2537     //     用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2538     //     在 sC (shared memory) 中的物理 offset。
2539     //     S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2540     //     = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2541     //     因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2542     //     M/N remainder。
2543     //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2544     //     → slice. gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2545     //     因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2546     //     repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2547     //     两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2548     //     因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2549     cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2550 }
2551 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2552 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。
2553 __syncthreads();
2554 }
2555
2556 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照: 手写版必须显式处理 MMA fragment
2557 // 的物理分布, 使用 RC0/RC1 暂存, 再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2558 // 8 个 half, 并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2559 // CuTe 版没有把这些 lane 映射写死在 kernel 源码中: retile_S 表达 accumulator
2560 // fragment 与 R2S copy 的对应, R2S/sC/S2G 通过 shared-memory scratchpad 完成
2561 // CTA 范围的数据重组, S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2562 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局;
2563 // 区别是手写版直接编排 shuffle/store, CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2564 }
2565
2566 // =====
2567 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2568 // =====
2569 // CuTe 的核心设计理念: "类型即配置" (Type-level configuration)
2570 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**,
2571 // kernel 接收这些类型的实例 (通常为空 struct), 在编译期完成全部映射推导。
2572 //
2573 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法, 形成 1:1 对照。
2574 //
2575 // 面试常问: 「CuTe 为什么比手写 PTX 简洁?」
2576 // 答: 手写需要:
2577 //   1) 手动计算每线程的 smem 地址偏移
2578 //   2) 手动计算 ldmatrix 的 lane 映射
2579 //   3) 手动插入 swizzle XOR 到每个地址计算点
2580 //   4) 手动做 epilogue 的 warp shuffle 编排
2581 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2582 // 在编译期自动完成上述全部推导, 生成与手写等价的 PTX 指令序列。

```

```

2583 // =====
2584 template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
2585 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2586     using namespace cute;
2587
2588     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2589     //
2590     // kernel 用法对照:
2591     //   BM=128:  local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2592     //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2593     //   BN=256:  local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2594     //             local_tile(D, ...) → gD 的列方向
2595     //   BK=32:   num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2596     //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2597     //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2598     auto BM = Int<128>{};
2599     auto BN = Int<256>{};
2600     auto BK = Int<32>{};
2601     auto KStage = Int<Stages>{};
2602     auto kSmemLayoutCBatch = Int<4>{};
2603
2604     // — SmemLayoutA / SmemLayoutB —
2605     // kernel 用法:
2606     // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2607     // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2608     // SmemLayout 由三部分组成: Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage.
2609     //
2610     // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子:
2611     //   - M=3: 连续 2^3=8 个 half 组成一个基本元素, 即 16 bytes;
2612     //   - S=3: 连续 2^3=8 个基本元素组成一行, 即 128 bytes;
2613     //   - B=3: 共有 2^3=8 行。
2614     // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B, 覆盖全部 32 个 shared-memory bank。
2615     // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol,
2616     // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2617     // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2618     // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2619     // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2620     // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2621     // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2622     // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2623     using SmemLayoutAtom = decltype(composition(
2624         Swizzle<3, 3, 3>{},
2625         make_layout(make_shape(Int<8>{}, Int<BK>{}),
2626             make_stride(Int<BK>{}, Int<1>{}))));
2627     using SmemLayoutA = decltype(tile_to_shape(
2628         SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2629     using SmemLayoutB = decltype(tile_to_shape(
2630         SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2631
2632     // — MMA (TiledMMA) —
2633     // kernel 用法:
2634     // TiledMMA tiled_mma;
2635     // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2636     // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2637     // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2638     // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2639     // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2640     // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2641     // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2642     //
2643     // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2644     //   → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2645     //   → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16

```

```

2646 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2647 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2648 using mma_op = SM80_16x8x16_F16F16F16_TN;
2649 using mma_traits = MMA_Traits<mma_op>;
2650 using mma_atom = MMA_Atom<mma_traits>;
2651 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2652
2653 static constexpr int kMmaEURepeatM = 2;
2654 static constexpr int kMmaEURepeatN = 2;
2655 static constexpr int kMmaEURepeatK = 1;
2656 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2657 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2658 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2659 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2660
2661 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2662     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2663 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2664 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2665
2666 // — G2SCopyA / G2SCopyB —
2667 // kernel 用法:
2668 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2669 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2670 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2671 // G→S: 128-bit cp.async. ThrLayout{32,4} × ValLayout{1,8}:
2672 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2673 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2674 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2675 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2676 using G2SCopyA = decltype(make_tiled_copy(
2677     g2s_copy_atom{},
2678     make_layout(make_shape(Int<32>{}, Int<4>{}),
2679         make_stride(Int<4>{}, Int<1>{})),
2680     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2681 using G2SCopyB = G2SCopyA;
2682
2683 // — S2RCopyAtomA / S2RCopyAtomB —
2684 // kernel 用法:
2685 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2686 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2687 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2688 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2689 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2690 // 与 MMA fragment 对齐的 shared-register 映射, 无需手动指定 ThrLayout/ValLayout。
2691 using s2r_copy_op = SM75_U32x4_LDSM_N;
2692 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2693 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2694 using S2RCopyAtomA = s2r_copy_atom;
2695 using S2RCopyAtomB = s2r_copy_atom;
2696
2697 // — SmemLayoutC —
2698 // kernel 用法:
2699 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2700 // 复用当前 A stage 的空间作为 C scratchpad
2701 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2702 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2703 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2704 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2705 // 的 kStage K-tile pipeline。
2706 using SmemLayoutAtomC = decltype(composition(
2707     Swizzle<3, 3, 3>{},
2708     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),

```

```

2709         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2710 using SmemLayoutC = decltype(tile_to_shape(
2711     SmemLayoutAtomC{},
2712     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2713
2714 // — R2SCopyAtomC —
2715 // kernel 用法:
2716 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2717 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2718 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad.
2719 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2720 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2721 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2722
2723 // — S2GCopyC —
2724 // kernel 用法:
2725 // S2GCopyC s2g_tiled_copy_c;
2726 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2727 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared-global wide store.
2728 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2729 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2730 using S2GCopyAtomC = Copy_Atom<UniversalCopy<cute::uint128_t>, T>;
2731 using S2GCopyC = decltype(make_tiled_copy(
2732     S2GCopyAtomC{},
2733     make_layout(make_shape(Int<32>{}, Int<4>{}),
2734         make_stride(Int<4>{}, Int<1>{})),
2735     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2736
2737 // — Grid/Block 配置 —
2738 // kernel 用法:
2739 // int idx = threadIdx.x;          // 0..127
2740 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2741 // int iy = blockIdx.y;           // M 方向 CTA 坐标
2742 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2743 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2744 // M % BM == 0、N % BN == 0、K % BK == 0。
2745 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2746 constexpr int kSwizzleStride = 2048;
2747 int BX = (N + BN - 1) / BN;
2748 int BY = (M + BM - 1) / BM;
2749 int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2750 BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2751 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2752 dim3 grid(BX, BY, BZ);
2753
2754 // — Dynamic Shared Memory 大小 —
2755 // kernel 用法:
2756 // extern __shared__ T shm_data[];
2757 // T *Ashm = shm_data;
2758 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2759 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2760 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2761 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2762 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2763 // 能放进一个 A stage (128*32 = 4096)。
2764 static constexpr int shm_size_AB =
2765     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2766 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2767 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2768     "C shared memory must fit within one A pipe");
2769 static constexpr int kShmSize =
2770     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2771

```

```

2772     cudaFuncSetAttribute(
2773         hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2774             SmemLayoutA, SmemLayoutB, SmemLayoutC,
2775             S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2776             S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
2777         cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2778
2779     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2780         SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2781         S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2782         BlockSwizzle>
2783     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2784 }
2785
2786 #endif /* NOTES_V2_ENABLE_CUTE */
2787
2788 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
2789 // CUDA 13.2 promotes these TMA operations to cuda::ptx. Keep the older
2790 // experimental wrappers so the Hopper path remains buildable on prior toolkits.
2791 __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
2792     #if CUDART_VERSION >= 13020
2793         cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
2794     #else
2795         cuda::device::experimental::fence_proxy_async_shared_cta();
2796     #endif
2797 }
2798
2799 __device__ __forceinline__ void tma_load_2d(
2800     void *dst, const CUtensorMap *tensor_map, int minor_coord, int major_coord,
2801     cuda::barrier<cuda::thread_scope_block> &barrier) {
2802     #if CUDART_VERSION >= 13020
2803         const int32_t coords[] {minor_coord, major_coord};
2804         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2805         cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
2806             cuda::ptx::space_global, dst, tensor_map,
2807             coords, barrier_handle);
2808     #else
2809         cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2810             dst, tensor_map, minor_coord, major_coord, barrier);
2811     #endif
2812 }
2813
2814 __device__ __forceinline__ void tma_arrive_expect_tx(
2815     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
2816     #if CUDART_VERSION >= 13020
2817         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2818         [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
2819             cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
2820             barrier_handle, bytes);
2821     #else
2822         [[maybe_unused]] auto token =
2823             cuda::device::barrier_arrive_tx(barrier, 1, bytes);
2824     #endif
2825 }
2826 #endif
2827
2828 // =====
2829 // Phase 7d: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2830 // =====
2831 // 面试要点 (WGMMMA vs MMA 对比) :
2832 //   - MMA: warp 级 (32 threads) , 同步执行
2833 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
2834 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4x16=64倍)

```

```

2835 // - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2836 // - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier 同步
2837 // - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2838
2839 // ---- WGMMMA 辅助函数 ----
2840 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2841 // - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2842 //   (per-wargroup, 故需 .sync.aligned 确保所有线程同步执行)。
2843 // - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N)。
2844 //   N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3),
2845 //   都是 "wait until only N or fewer groups are pending"。
2846 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2847 #define WGMMMA_COMMIT_GROUP() \
2848     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2849 #define WGMMMA_WAIT_GROUP(n) \
2850     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2851
2852 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMMA descriptor 位域中的 14-bit 值。
2853 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2854 //   encoded = (x & 0x3FFFF) >> 4
2855 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2856 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4), 低 4 bit 恒为 0,
2857 // 可省略以扩大可表示范围。
2858 // 解码: decoded = encoded << 4 (回到原始 byte 值)。
2859 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2860
2861 // make_smem_desc: 构造 WGMMMA (wargroup MMA) 的 64-bit shared memory 矩阵描述符。
2862 // 硬件 WGMMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2863 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2864 //
2865 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2866 // -----
2867 // | 位域          | 范围      | 大小 | 含义
2868 // |-----|-----|-----|-----|
2869 // | start_address  | [0, 14)   | 14   | smem 基址编码
2870 // | (unused)       | [14, 16)  | 2    | -
2871 // | leading_byte_offset | [16, 30) | 14   | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2872 // | (unused)       | [30, 32)  | 2    | -
2873 // | stride_byte_offset | [32, 46) | 14   | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2874 // | (unused)       | [46, 49)  | 3    | -
2875 // | base_offset    | [49, 52)  | 3    | swizzle pattern 偏移
2876 // | (unused)       | [52, 62)  | 10   | -
2877 // | layout_type    | [62, 64)  | 2    | swizzle 模式
2878 // -----
2879 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2880 //
2881 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2882 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2883 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2884 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2885 // - 这是 stride_byte_offset=1024 的来源
2886 //
2887 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1),
2888 //   此处填 16 仅为占位, 编码后为 1。
2889 //
2890 // - base_offset=0 (bits [49,52) 未显式置位), 表示 smem ptr 已 1024B 对齐。
2891 //   若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2892 //
2893 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2894 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2895 __device__ inline uint64_t make_smem_desc(half *ptr) {
2896     // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2897     // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。

```

```

2898 uint32_t addr = static_cast<uint32_t>((__cvta_generic_to_shared(ptr)));
2899
2900 // 从零开始：所有位域初始化为 0。
2901 uint64_t desc = 0x0000000000000000;
2902
2903 // — bits [0, 14): start_address —
2904 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2905 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2906 // 可寻址范围:  $2^{(14+4)} = 2^{18} = 256K$  个 half = 512 KB 共享内存。
2907 desc |= SMEM_DESC_ENCODE(addr);
2908
2909 // — bits [16, 30): leading_byte_offset —
2910 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2911 // << 16 将编码值放到 bits [16, 30)。
2912 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2913 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2914 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2915 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2916 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2917 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2918
2919 // — bits [32, 46): stride_byte_offset —
2920 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2921 // << 32 将编码值放到 bits [32, 46)。
2922 // K-Major 下定义为"从首 8 行到次 8 行的字节偏移" (PTX ISA §9.7.15.5.1.2.1.2) 。
2923 // 1024 的来源 (K-Major + 128B swizzle + half) :
2924 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2925 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2926 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2927 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2928
2929 // — bits [62, 64): layout_type (swizzle mode) —
2930 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2931 // 1 = 128B swizzle mode。
2932 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2933 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2934 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2935 desc |= 1llu << 62;
2936
2937 return desc;
2938 }
2939
2940 // ---- WGMMMA PTX 指令宏 ----
2941 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2942 // m64n128k16: M=64, N=128, K=16。一次 WGMMMA 计算  $D[64 \times 128] += A[64 \times 16] \times B[16 \times 128]$ 。
2943 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2944 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
2945 // 每线程 64 half = 32 uint32。
2946 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2947 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2948 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2949 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2950 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2951 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2952                                     TransB) \
2953 { \
2954     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2955     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2956     asm volatile( \
2957         "{\n" \
2958         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2959         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2960         " %8, %9, %10, %11, %12, %13, %14, %15, " \

```

```

2961     " %16, %17, %18, %19, %20, %21, %22, %23, " \
2962     " %24, %25, %26, %27, %28, %29, %30, %31}, " \
2963     " %32, " \
2964     " %33, " \
2965     " %34, %35, %36, %37, %38;\n" \
2966     "}\n" \
2967     : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2968     "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
2969     "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
2970     "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
2971     "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
2972     "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
2973     "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
2974     "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
2975     : "l"(desc_a), "l"(desc_b), "n"(int32_t(Scaled)), \
2976     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
2977     "n"(int32_t(TransB)); \
2978 }
2979
2980 // ---- TMA Shared Memory Layout ----
2981 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
2982 //
2983 // 每个 stage 包含两块 smem:
2984 //   A tile: [BM, BK] row-major → BK×BM 个 half, 地址连续
2985 //   B tile: TMA 按 [BN, BK] 物理写入 (详见下方 ★), BN×BK 个 half
2986 //
2987 // ★ 关键区别 — WGMMMA 的 B 物理存储 vs 逻辑视图:
2988 //
2989 //   上层数据: 源矩阵 B^T [N, K] row-major (TN 布局, B 的列以行优先形式存储)。
2990 //   物理存储: TMA 将 B^T 的 tile 写入 smem, 物理布局为 [BN, BK] row-major
2991 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
2992 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,
2993 //   TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
2994 //   逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
2995 //   通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
2996 //   实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
2997 //   swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
2998 //
2999 //   stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
3000 //   128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
3001 //   stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
3002 //   = 8 rows × (BK=64) halves × 2 bytes = 1024。
3003 //   这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
3004 //
3005 //   与 MMA(TN) 的对比:
3006 //   MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
3007 //   WGMMMA: smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
3008 //   简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
3009 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
3010     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
3011     // B tile 标记:
3012     //   - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
3013     //   - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
3014     //       以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
3015     //   - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
3016     //       但写 BN×BK 更能体现物理存储形态
3017     alignas(128) half B[BN * BK * QSIZE];
3018 };
3019
3020 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
3021 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化) :
3022 //
3023 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",

```

```

3024 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
3025 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
3026 //
3027 //   WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
3028 //   将 A/B tile 从 HBM 搬到 shared memory。
3029 //   WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
3030 //
3031 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
3032 //   - full[stage]: Producer 发信号表示 stage stage 的数据已就绪, 可被使用
3033 //   - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
3034 //
3035 // 本节重点理解:
3036 //   1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
3037 //      即可搬运整个 2D tile, 无需所有线程参与。
3038 //   2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
3039 //   3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
3040 //      Producer 可以搬运 stage (stage+1), 隐藏 HBM→SMEM 延迟。
3041 //
3042 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
3043 //   WGMMMA Atom:      m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
3044 //   K Tile:           BK=64, 每个 K tile 包含 BK/kWmmaK=4 个 WGMMMA atom
3045 //   M Tile:           BM=128, 每个 M tile 包含 BM/kWmmaM=2 个 WGMMMA atom
3046 //   N Tile:           BN=128, 单个 kWmmaN=128 即可覆盖, 无需在 N 方向分块
3047 //   Block Tile:       C[128,128] = A[128,64] × B[64,128]
3048 //   Threads:          256 = 2 warpgroups × 128 threads/warpgroup
3049 //   Producer(WG0):    128 threads (仅 thread 0 做 TMA 提交)
3050 //   Consumer(WG1):    128 threads = 4 warps (全部参与 WGMMMA 和写回)
3051 //
3052 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
3053 // 的延迟被计算完全掩盖。
3054 //
3055 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
3056 //   - grid.z = S 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
3057 // Block: (256, 1, 1), 2 warpgroups
3058 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
3059 template <const int kWmmaM = 64,           // WGMMMA atom M dim (m64n128k16)
3060           const int kWmmaN = 128,         // WGMMMA atom N dim
3061           const int kWmmaK = 16,          // WGMMMA atom K dim
3062           const int BM = 128,             // block tile M, 2 × kWmmaM
3063           const int BN = 128,             // block tile N, 1 × kWmmaN
3064           const int BK = 64,              // block tile K, 4 × kWmmaK
3065           const int kNumThreads = 256,    // 2 warpgroups × 128 threads
3066           const int kStages = 3,          // TMA pipeline depth (full/empty barriers)
3067           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
3068 __global__ void __launch_bounds__(kNumThreads)
3069     hgemm_wgmma_stages_tn(
3070         int M, int N, int K, half *C,
3071         const CUtensorMap *__restrict__ tensorMapA,
3072         const CUtensorMap *__restrict__ tensorMapB) {
3073     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3074     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3075     // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3076     // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3077     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3078     // 不展开宿主侧 create_tensor_map 细节。
3079
3080     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3081     // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3082     const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3083     const int by = blockIdx.y;
3084     constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
3085     // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3086     // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer

```

```

3087 constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
3088 // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
3089 // 当只有一个 consumer 时, 它负责全部 BM=128 行
3090 constexpr int kWarpgroupM = BM / kNumConsumers; // 128
3091
3092 // 边界检查: 确保当前 block 不超出 M/N 范围
3093 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3094     return;
3095
3096 // ---- Shared Memory 分配 ----
3097 // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3098 // TMA + WGMMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
3099 //   make_smem_desc() 构造 WGMMMA descriptor 时, start_address 字段
3100 //   完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
3101 //   硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
3102 //   descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
3103 //   1024B 边界, WGMMMA 硬件也能正确读取 TMA 写入的数据。
3104 // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle
3105 //   函数 tma_swizzle_128B(), 该函数无法感知 smem 绝对地址, 硬编码
3106 //   了 phase=0 的假设, 因此必须 __align__(1024) 来保证 phase 确实为 0。
3107 //   从健壮性角度看本 kernel 也应该用 __align__(1024); 这里保留 128 是
3108 //   为了突出两种消费者的特点与对比。
3109 extern __shared__ __align__(128) uint8_t smem_tma_wgmma_ws[];
3110 WgmmaSMem<BM, BN, BK, kStages> &s =
3111     *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages> *>(smem_tma_wgmma_ws);
3112 half *s_a = s.A;
3113 half *s_b = s.B;
3114
3115 // ---- cuda::barrier 初始化 ----
3116 //
3117 // ★ Barrier 机制速查 (arrive / wait 核心语义):
3118 //
3119 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3120 //
3121 //   init(bar, N): 设置 arrive_count = N。
3122 //   含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3123 //
3124 //   arrive(): 线程声明"我已到达此 barrier 点"。
3125 //   - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3126 //   - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3127 //
3128 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3129 //   - 即: 等待 phase 翻转。
3130 //   - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3131 //                     (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3132 //
3133 //   典型用法: b.wait(b.arrive())
3134 //   - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3135 //   - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3136 //   - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3137 //   - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3138 //
3139 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129):
3140 //
3141 //   Producer (1 thread)           Consumer (128 threads)
3142 //   -----
3143 //
3144 //   C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3145 //   ┌ for each k_tile: ─┐           ┌ for each k_tile: ─┐
3146 //   │ P1: arrive+wait(empty[q]) │   │ C1: arrive+wait(full[q]) │
3147 //   │   ↓ 等 stage q 被消费完   │   │   ↓ 等 TMA 数据就绪   │
3148 //   │ P2: TMA(A[q]) + TMA(B[q]) │   │ C2: WGMMMA 计算   │
3149 //   │   ↓ 异步拷贝             │   │ C3: wait WGMMMA 完成   │
3150 //   │ P3: arrive_tx(full[q])    │   │ C4: arrive(empty[q])    │

```

```

3150 // |      ↑ 通知: stage q 数据就绪      |      ↑ 通知: stage q 已消费完
3151 // |_____|                                |_____|
3152 //
3153 // 关键不变式 (invariant) :
3154 //   - Producer 不会覆盖 Consumer 正在读的 stage
3155 //   - Consumer 不会读 Producer 还没写完的 stage
3156 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
3157 //
3158 // 每个 stage 有两个 barrier:
3159 //   full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3160 //   empty[stage]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3161 //
3162 //   arrive_count = 128 (consumer) + 1 (producer) = 129:
3163 //   每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3164 #pragma nv_diag_suppress static_var_with_dynamic_init
3165   __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3166   __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3167 #pragma nv_diag_default static_var_with_dynamic_init
3168
3169 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3170 const int NUM_K_TILES = div_ceil(K, BK);
3171 const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
3172 const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup
3173
3174 // 初始化 barriers (仅 thread 0 执行)
3175 if (threadIdx.x == 0) {
3176     for (int i = 0; i < kStages; ++i) {
3177         // arrive_count = 129: 128 consumer + 1 producer
3178         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3179         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3180         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3181         init(&full[i], kConsumerThreads + 1); // 128 consumer + 1 producer
3182         init(&empty[i], kConsumerThreads + 1); // same
3183     }
3184     // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3185     // 对 async proxy (TMA/WGMMA) 可见。
3186     tma_fence_proxy_async_shared_cta();
3187 }
3188 __syncthreads();
3189
3190 // =====
3191 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3192 // =====
3193 //
3194 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工, 不是先后顺序:
3195 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支, 做 Producer。
3196 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支, 做 Consumer。
3197 //
3198 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp, 两者**同时**推进:
3199 //   Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 } ← 自己的循环
3200 //   Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
3201 //
3202 // 两个 warpgroups 各自独立迭代 K-tiles, 通过 full[] / empty[] barrier 同步:
3203 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞, 等 Consumer 消费完
3204 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞, 等 TMA 搬运完
3205 //
3206 // 这是生产者-消费者模型的 GPU 实现: 不需要共享循环变量, barrier 的 phase
3207 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3208 // =====
3209 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3210 // 职责: 提交 TMA 2D 拷贝, 将 A/B tile 从 HBM 异步搬运到 SMEM。
3211 // 只有 wg_tid==0 执行实际拷贝提交, 其余 127 个线程空闲。
3212 // =====

```

```

3213 if (wg_idx == 0) {
3214     if (wg_tid == 0) {
3215         // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3216         int stage = 0;
3217         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3218             if (stage == kStages)
3219                 stage = 0;
3220
3221             // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)
3222             //
3223             // 模式 empty[stage].wait(empty[stage].arrive()):
3224             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达,
3225             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3226             //     128 次 arrive (来自上一轮迭代或 C0 初始化), 则加上这次共 129 次
3227             //     → phase 立即翻转。
3228             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3229             //     由于 arrive() 是第 129 次到达, phase 在 arrive 时已翻转,
3230             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3231             //
3232             // 语义: Producer 说"我准备好写 stage stage 了, Consumer 用完没有?"
3233             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3234             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3235             empty[stage].wait(empty[stage].arrive());
3236
3237             // Step P2: 提交 TMA 2D 拷贝指令
3238             // cp_async_bulk_tensor_2d_global_to_shared:
3239             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3240             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3241             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3242             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3243             //
3244             // TMA 是硬件 DMA 引擎: 一次指令提交即可搬运整个 2D tile,
3245             // 无需线程逐元素搬运, 零寄存器开销。
3246             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3247             tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
3248                 full[stage]);
3249
3250             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3251             tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
3252                 full[stage]);
3253
3254             // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
3255             //
3256             // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3257             //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3258             //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3259             //   - Phase 翻转条件:
3260             //     (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3261             //     (b) 所有声明的 async 字节已写入 smem
3262             //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3263             //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3264             tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
3265         }
3266     }
3267 }
3268 // =====
3269 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3270 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3271 // 所有 128 个线程 (4 warps) 全部参与。
3272 // =====
3273 else {
3274     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3275     //

```

```

3276 // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3277 // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[stage].wait()
3278 // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129) 。
3279 //
3280 // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3281 // Producer 后续的 empty[stage].arrive() 作为第 129 次, 触发 phase 翻转。
3282 for (int i = 0; i < kStages; ++i) {
3283     [[maybe_unused]] auto token = empty[i].arrive();
3284 }
3285
3286 // 累加器寄存器声明
3287 // d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4]:
3288 //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3289 //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3290 //   - d[*][g][*]: N 方向第 g 组 16 列
3291 //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3292 // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3293 // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3294 uint32_t d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4] = {};
3295
3296 int stage = 0;
3297 // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3298 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3299     if (stage == kStages)
3300         stage = 0;
3301
3302     // Step C1: 等待 TMA 数据就绪 (full 信号)
3303     //
3304     // 模式 full[stage].wait(full[stage].arrive()):
3305     //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3306     //   - 当前 phase 累积: 128/129, phase 尚未翻转。
3307     //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)
3308     //     贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3309     //
3310     // 语义: Consumer 说"我准备好读 stage stage 了, 数据到了没有? "
3311     full[stage].wait(full[stage].arrive());
3312
3313     // Step C2: 发射 WGMMMA 指令序列
3314     //
3315     // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3316     // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3317     //
3318     // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3319     //   - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3320     //   - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3321     //   - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3322     //
3323     // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3324     //   D[64x128] += A[64x16] × B[16x128]
3325     //   A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3326     //   ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都 ScaleD=1。
3327     WGMMMA_FENCE();
3328
3329     // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3330     // 每个 atom 处理 64 行 M 方向数据。
3331 #pragma unroll
3332     for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3333         // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3334         // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3335         half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;
3336
3337         // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3338         // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。

```

```

3339 #pragma unroll
3340 for (int k_step = 0; k_step < BK / kWmmaK; ++k_step) {
3341     // 第 k_step 次 WGMMMA:
3342     //   A: wmma_sA + k_step * kWmmaK (A 的 K 维起始位置)
3343     //   B: s_b + stage * BK * BN + k_step * kWmmaK (B 的 K 维起始位置)
3344     //   注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3345     //   第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3346     //   ScaleD=1: 累加 (K 维迭代需要累积结果)
3347     WGMMMA_M64N128K16_F16F16F16(d[m], wmma_sA + k_step * kWmmaK,
3348                                   s_b + stage * BK * BN + k_step * kWmmaK,
3349                                   1, // ScaleD=1: accumulate (不清零)
3350                                   1, 1, 0, 0);
3351 }
3352 }
3353
3354 // Step C3: 提交并等待 WGMMMA 完成
3355 //
3356 // WGMMMA_COMMIT_GROUP (wmma.commit_group.sync.aligned):
3357 //   将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3358 //   类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3359 //
3360 // WGMMMA_WAIT_GROUP(0) (wmma.wait_group.sync.aligned 0):
3361 //   阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3362 //   * 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3363 //   两者都是 "wait until only N or fewer of the most recent groups are pending"。
3364 //   此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3365 //   必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3366 WGMMMA_COMMIT_GROUP();
3367 WGMMMA_WAIT_GROUP(0);
3368
3369 // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
3370 //
3371 // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
3372 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3373 //
3374 // 语义: Consumer 说 "stage stage 我已经读完了, 你可以放心覆盖了。"
3375 [[maybe_unused]] auto token = empty[stage].arrive();
3376 }
3377
3378 // =====
3379 // Epilogue: 将寄存器中的累加结果写回 global memory C
3380 // =====
3381 //
3382 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3383 //
3384 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3385 // 这些 half 分布在 d[2][8][4] 数组中:
3386 //   d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3387 //   d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3388 //   d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3389 //   d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3390 //
3391 // 其中:
3392 //   row = warp * 16 + lane / 4   (0~63, 4 warps * 16 rows)
3393 //   col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3394 //
3395 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3396 //   +-----+-----+
3397 //   | reg[0] | reg[2] | <- rows [row, row+8)
3398 //   |(col,+2)|(col+8)|
3399 //   +-----+-----+
3400 //   | reg[1] | reg[3] | <- rows [row+8, row+16)
3401 //   |(col,+2)|(col+8)|

```

```

3402 // +-----+-----+
3403 // 每个 reg 包含 2 个连续 half (col 和 col+2) , 用 uint32 存储。
3404 //
3405 // 三维遍历:
3406 // m_it=0: rows 0~63, m_it=1: rows 64~127
3407 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3408 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3409 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3410
3411 const int lane = wg_tid % 32;
3412 const int warp = wg_tid / 32;
3413 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3414 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3415 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3416 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分) 。
3417 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3418 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0) 。
3419 const int row = warp * 16 + lane / 4;
3420 // block_C: 当前 C tile 在 global memory 中的起始地址
3421 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3422 half *block_C = C + by * BM * N + bx * BN;
3423
3424 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3425 // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
3426 #pragma unroll
3427 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3428     int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
3429 #pragma unroll
3430 for (int g = 0; g < kWgmmaN / 16; ++g) {
3431     int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3432     // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3433     // 左上象限: (row, col)
3434     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) = d[m][g][0];
3435     // 左下象限: (row+8, col)
3436     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
3437     // 右上象限: (row, col+8)
3438     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
3439     // 右下象限: (row+8, col+8)
3440     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
3441 }
3442 }
3443 }
3444 }
3445
3446 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3447 // SM120 不支持 WGMMMA, 但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3448 // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
3449 template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
3450     static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
3451     half A[BM * BK * QSIZE];
3452     half B[BN * BK * QSIZE];
3453 };
3454
3455 // tma_swizzle_128B: 计算 128B TMA swizzle 下 smem 中 (row, k) 的物理 K 偏移。
3456 //
3457 // TMA descriptor 使用 CU_TENSOR_MAP_SWIZZLE_128B 时, 硬件会将写入 smem 的
3458 // 数据按 128B 粒度做 XOR 重映射以消除 bank conflict。消费者在 ldmatrix 前必须
3459 // 对地址施加相同的 swizzle 变换, 否则读到的数据与 TMA 写入的物理位置不一致。
3460 //
3461 // 位级公式与 CuTe 对照:
3462 // swizzle_k = (((k >> 3) ^ (row & 7)) << 3) | (k & 7)
3463 //
3464 // 等价于 CuTe Swizzle<3,4,3>:

```

```

3465 // M=3 → 基本元素宽 2^3=8 个 half (16 bytes)
3466 // S=4 → 每行 2^4=16 个基本元素 (32 个 half) ...不, BK=64...
3467 //
3468 // 实际让我们从函数出发:
3469 // - k & 7: 低 3 位保持不变。即 8 个 half (16 bytes) 为一组, 组内顺序不变。
3470 // - k >> 3: K 被划分为 8 个 chunk (BK=64, 每 chunk 8 个 half) 。
3471 // - row & 7: 取行号的低 3 位 (swizzle 周期为 8 行) 。
3472 // - XOR: 将 chunk 索引与行号的低 3 位做异或。
3473 //
3474 // ★ swizzle 映射图 (BK=64, 每 row 8 chunk à 8 half; 周期 = 8 rows × 512 half = 1024B) :
3475 //
3476 // 下图表示同一 chunk 索引在各行被映射到的物理 chunk 位置。例如 chunk 0 在
3477 // row 0 位于物理 chunk 0, 在 row 1 被 XOR 到物理 chunk 1, 以此类推。
3478 //
3479 // chunk idx:      0      1      2      3      4      5      6      7
3480 // -----
3481 // row 0 (0^c):    0      1      2      3      4      5      6      7      ← 恒等
3482 // row 1 (1^c):    1      0      3      2      5      4      7      6      ← 相邻对交换
3483 // row 2 (2^c):    2      3      0      1      6      7      4      5
3484 // row 3 (3^c):    3      2      1      0      7      6      5      4
3485 // row 4 (4^c):    4      5      6      7      0      1      2      3
3486 // row 5 (5^c):    5      4      7      6      1      0      3      2
3487 // row 6 (6^c):    6      7      4      5      2      3      0      1
3488 // row 7 (7^c):    7      6      5      4      3      2      1      0
3489 //
3490 // — 8 行一周期 —
3491 // row 8 (0^c): 同 row 0 映射; row 9 同 row 1; 依此类推。
3492 //
3493 // 例子——ldmatrix 读取 row 1、逻辑 k=0 处的数据:
3494 // swizzle_k = (((0 >> 3) ^ (1 & 7)) << 3) | (0 & 7) = ((0 ^ 1) << 3) | 0 = 8
3495 // 即 row 1 的 chunk 0 被 TMA 写到了物理 K=8 的位置, ldmatrix 地址需从 K=8 读。
3496 //
3497 // 例子——ldmatrix 读取 row 3、逻辑 k=16 处的数据:
3498 // swizzle_k = (((16 >> 3) ^ (3 & 7)) << 3) | (16 & 7)
3499 //              = ((2 ^ 3) << 3) | 0 = (1 << 3) | 0 = 8
3500 // 即 row 3 的 chunk 2 被映射到物理 chunk 1 (K=8) 。
3501 //
3502 // 关键结论: TMA 和 ldmatrix 两侧必须使用相同的 swizzle; 任何一侧不用或用了
3503 // 不同的 pattern, 整个 tile 的输出都会被破坏。128B = 128 BYTES = 64 half
3504 // = 8 chunk × 8 half/chunk = 8 rows × 512 half/row。
3505 __device__ __forceinline__ int tma_swizzle_128B(int row, int k) {
3506     return (((k >> 3) ^ (row & 7)) << 3) | (k & 7);
3507 }
3508
3509 // 消费者 MMA 层级映射参考 hgemmmma_stages_tn_swizzle。与那个
3510 // 八 warp kernel 不同, 本 warp-specialized 消费者仅拥有四个 warp。
3511 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
3512           const int kMmaN = 8,           // MMA atom N dim
3513           const int kMmaK = 16,          // MMA atom K dim
3514           const int kMmaTileM = 2,        // consumer warps along M, warp tile M = 32
3515           const int kMmaTileN = 2,        // consumer warps along N, warp tile N = 16
3516           const int kValTileM = 4,        // value-repeat along M, BM = 16*2*4 = 128
3517           const int kValTileN = 8,        // value-repeat along N, BN = 8*2*8 = 128
3518           const int kValTileK = 4,        // MMA_K slices per BK tile, BK = 16*4 = 64
3519           const int kStages = 2,          // TMA full/empty pipeline depth
3520           const int kNumThreads = 256,    // 128 producer + 128 consumer threads
3521           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
3522 __global__ void __launch_bounds__(kNumThreads)
3523 hgemm_tma_mma_ws_tn(
3524     int M, int N, int K, half *C,
3525     const CUtensorMap *__restrict__ tensorMapA,
3526     const CUtensorMap *__restrict__ tensorMapB) {
3527

```

```

3528 constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3529 constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3530 constexpr int BK = kMmaK * kValTileK;
3531 static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3532 static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3533 static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3534 static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3535 static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3536
3537 const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3538 const int by = blockIdx.y;
3539 constexpr int kConsumerThreads = kNumThreads / 2;
3540 static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3541             "Consumer threads must cover the MMA warp grid");
3542
3543 if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3544     return;
3545
3546 // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐,
3547 // 以确保硬件 swizzle phase 从零开始。消费者 tma_swizzle_128B()
3548 // 假设零 phase; 其他对齐 (如 128) 会导致 phase 偏移, 产生不匹配的
3549 // 物理地址, 破坏整个输出。
3550 //
3551 // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128), 而本 kernel
3552 // 必须 __align__(1024)? 核心区别在于消费者如何感知 swizzle phase:
3553 //
3554 // WGMMMA 消费者: 通过 make_smem_desc() 构造 64-bit WGMMMA descriptor,
3555 // 其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3556 // 1024B 边界的偏移量 ((addr >> 7) & 0x7)。硬件在读取 smem 时会
3557 // 用这个 base_offset 自动补偿 swizzle phase, 因此 WGMMMA 路径不
3558 // 需要 1024 字节对齐也能得到正确数据。
3559 //
3560 // 本 kernel 消费者: 使用 ldmatrix + 手写 tma_swizzle_128B() 来
3561 // 计算 smem 物理地址。这个函数是纯软件公式, 没有任何 base_offset
3562 // 补偿机制。它假设 smem 基址恰好落在 1024B 边界上 (phase=0)。
3563 // 如果基址只满足 128B 对齐, TMA 硬件会以非零 phase 写入数据,
3564 // 而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3565 //
3566 // 简言之: WGMMMA 用硬件 descriptor base_offset 自动解决 phase 偏移;
3567 // ldmatrix 路径没有等价机制, 必须靠 1024B 对齐来保证 phase=0。
3568 extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3569 TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3570     *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages>*>(smem_tma_mma_ws);
3571 half *s_a = s.A;
3572 half *s_b = s.B;
3573
3574 #pragma nv_diag_suppress static_var_with_dynamic_init
3575 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3576 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3577 #pragma nv_diag_default static_var_with_dynamic_init
3578
3579 const int NUM_K_TILES = div_ceil(K, BK);
3580 const int wg_idx = threadIdx.x / kConsumerThreads;
3581 const int wg_tid = threadIdx.x % kConsumerThreads;
3582
3583 if (threadIdx.x == 0) {
3584     for (int stage = 0; stage < kStages; ++stage) {
3585         init(&full[stage], kConsumerThreads + 1);
3586         init(&empty[stage], kConsumerThreads + 1);
3587     }
3588     tma_fence_proxy_async_shared_cta();
3589 }
3590 __syncthreads();

```

```

3591
3592 if (wg_idx == 0) {
3593     // 生产者 Warpgroup (wg_idx==0)
3594     if (wg_tid == 0) {
3595         int stage = 0;
3596         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3597             if (stage == kStages)
3598                 stage = 0;
3599
3600             empty[stage].wait(empty[stage].arrive());
3601
3602             tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
3603                 full[stage]);
3604             tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
3605                 full[stage]);
3606             tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
3607         }
3608     }
3609 } else {
3610     // 消费者 Warpgroup (wg_idx==1)
3611     // 初始化: 标记所有 stage 为"空" (可被 Producer 写入)
3612     for (int stage = 0; stage < kStages; ++stage) {
3613         [[maybe_unused]] auto token = empty[stage].arrive();
3614     }
3615
3616     const int warp_id = wg_tid / kWarpSize;
3617     const int lane_id = wg_tid % kWarpSize;
3618     // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
3619     const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
3620     const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}
3621     uint32_t RA[kValTileM][4];
3622     uint32_t RB[kValTileN][2];
3623     uint32_t RC[kValTileM][kValTileN][2] = {0};
3624
3625     const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
3626     const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
3627     int stage = 0;
3628     for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3629         if (stage == kStages)
3630             stage = 0;
3631
3632         full[stage].wait(full[stage].arrive());
3633         tma_fence_proxy_async_shared_cta();
3634
3635         // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
3636         // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
3637         // 同步机制与 warp specialization 的线程分工:
3638         //
3639         // 1. 生产者-消费者解耦: TMA 拷贝由 producer (wg_idx==0 的单个线程)
3640         //    通过 tma_load_2d + tma_arrive_expect_tx 提交, 而非所有 256 个线程
3641         //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
3642         //    拷贝都完成后进入计算; 而 TMA 路径用 cuda::barrier 的 full/empty
3643         //    协议完成 CTA 级同步——full[stage].wait() 返回时, producer 已通过
3644         //    expect_tx 声明了完整的传输字节数, 且硬件保证这些字节已全部写入 smem。
3645         //
3646         // 2. async proxy fence 替代 __syncthreads: fence_proxy_async(space_shared)
3647         //    在 async proxy (TMA 写入通道) 和后续 smem load (ldmatrix 读取通道)
3648         //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作 (TMA)
3649         //    对 fence 之后的所有 smem 读操作 (ldmatrix) 可见。这是一个轻量级
3650         //    proxy 间 ordering, 不需要阻塞线程, 也不会让 warp 等待其他 warp。
3651         //
3652         // 3. 消费者内部 warp 间无依赖: WG1 内的 4 个 warp (2x2 grid) 各自独立
3653         //    读取 smem 中互不重叠的 A/B 区域 (warp_m 和 warp_n 分别索引不同的

```

```

3654 // 行区间)。它们之间不需要读写共享数据,因此不需要 __syncthreads 来
3655 // 对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令,保证同一
3656 // warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。
3657
3658 // 注意:这里已经没有NUM_K_TILES循环了,因为K loop load已经被WG0生产者处理了
3659 #pragma unroll
3660 for (int k_step = 0; k_step < kValTileK; ++k_step) {
3661
3662 #pragma unroll
3663 for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
3664 // kMmaM * kValTileM = 16*4 = 64, 每个消费者 warp 在 M 方向覆盖 64 行。
3665 const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3666 const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
3667 // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
3668 // lane_smem_a_k 中传给 tma_swizzle_128B(), 而非像 swizzle kernel
3669 // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
3670 // tma_swizzle_128B 是 128B TMA swizzle, 作用于完整 BK=64,
3671 // swizzle 周期覆盖全部 64 列, k_step=0 和 k_step=1 的 chunk 会
3672 // 被 XOR 交叉混合 → k_step 偏移必须在 swizzle 内部参与 chunk 计算。
3673 // swizzle_A<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
3674 // swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
3675 const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
3676 const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
3677 (stage * BM * BK + lane_smem_a_m * BK +
3678 tma_swizzle_128B(lane_smem_a_m, lane_smem_a_k)) *
3679 sizeof(half);
3680 LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
3681 }
3682
3683 #pragma unroll
3684 for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
3685 // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
3686 const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3687 const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
3688 const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
3689 const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
3690 (stage * BN * BK + lane_smem_b_n * BK +
3691 tma_swizzle_128B(lane_smem_b_n, lane_smem_b_k)) *
3692 sizeof(half);
3693 LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
3694 }
3695
3696 #pragma unroll
3697 for (int i = 0; i < kValTileM; ++i) {
3698 #pragma unroll
3699 for (int j = 0; j < kValTileN; ++j) {
3700 HMM16816(RC[i][j][0], RC[i][j][1],
3701 RA[i][0], RA[i][1], RA[i][2], RA[i][3],
3702 RB[j][0], RB[j][1],
3703 RC[i][j][0], RC[i][j][1]);
3704 }
3705 }
3706 }
3707 [[maybe_unused]] auto token = empty[stage].arrive();
3708 }
3709
3710 // Epilogue: 复用 RA[0] 和 RA[1] (已在寄存器中) 作为 shuffle 缓冲,
3711 // 与 hgemm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
3712 // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4,
3713 // 由 lane 0 发出对齐的 128-bit store。
3714 #pragma unroll
3715 for (int i = 0; i < kValTileM; ++i) {
3716 const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;

```

```

3717     const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
3718 #pragma unroll
3719     for (int j = 0; j < kValTileN; ++j) {
3720         const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3721         RA[0][0] = RC[i][j][0];
3722         RA[1][0] = RC[i][j][1];
3723         RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
3724         RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
3725         RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
3726         RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
3727         RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
3728         RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
3729         if (lane_id % 4 == 0) {
3730             const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
3731             const int store_gmem_c_addr_0 =
3732                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
3733             const int store_gmem_c_addr_1 =
3734                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
3735             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
3736                 *reinterpret_cast<float4*>(&RA[0][0]);
3737             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
3738                 *reinterpret_cast<float4*>(&RA[1][0]);
3739         }
3740     }
3741 }
3742 }
3743 }
3744 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
3745
3746 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3747 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3748 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled):
3749 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3750 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3751 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3752 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3753 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3754 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3755 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3756 //
3757 // * gmem_prob_stride + 1 的含义:
3758 //   语法层面 — 数组名退化 + 指针算术:
3759 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3760 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3761 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3762 //   语义层面 — 跳过隐式的最内维 stride:
3763 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3764 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3765 //     固定等于 sizeof(element_type), 不需要显式传入。
3766 //     对于 tensorRank=2 的 FP16 矩阵:
3767 //       dim 0 (内维, K): stride = sizeof(half) ← 隐式, API 不需要
3768 //       dim 1 (外维, M): stride = sizeof(half)*K ← 需要传给 API
3769 //     gmem_prob_stride 数组的布局是内维在前:
3770 //       [0] = sizeof(half) ← 内维, 不传
3771 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3772 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3773 //
3774 // * smem_box_stride 为什么不需要 +1?
3775 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3776 //   最内维 stride 也必须显式传入 (不隐式)。
3777 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3778 //   都是连续存放的, 维度间没有 padding/stride。
3779 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),

```

```

3780 // 即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3781 // 所以这里直接传 smem_box_stride (即 &smem_box_stride[0]) , API 会读到全部
3782 // 两个 stride 值, 不需要跳过任何一个。
3783 //
3784 // 参考: CUDA Programming Guide $TMA, PTX ISA $9.7.15.5, CUTLASS GmmaDescriptor
3785
3786 template <int BlockMajorSize, int BlockMinorSize>
3787 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3788                                             half *gmem_ptr,
3789                                             int blocks_height,
3790                                             int blocks_width) {
3791     void *gmem_address = (void *)gmem_ptr;
3792     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3793                                     (uint64_t)BlockMajorSize * blocks_height,
3794                                     1, 1, 1};
3795     uint64_t gmem_prob_stride[5] = {
3796         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3797     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3798                                   uint32_t(BlockMajorSize), 1, 1, 1};
3799     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3800     CUresult result = cuTensorMapEncodeTiled(
3801         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3802         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3803         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3804         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3805     if (result != CUDA_SUCCESS)
3806         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3807 }
3808
3809 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(
3810     half *src, int blocks_height, int blocks_width) {
3811     CUtensorMap *tma_map_d;
3812     cudaMalloc(&tma_map_d, sizeof(CUtensorMap));
3813     CUtensorMap tma_map_host;
3814     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3815     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUtensorMap),
3816               cudaMemcpyHostToDevice);
3817     return tma_map_d;
3818 }
3819 #endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */
3820
3821 // =====
3822 // Phase 8: FlashAttention-2 (Split-Q + MMA m16n8k16)
3823 // =====
3824 // 面试要点 (FlashAttention 算法):
3825 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3826 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
3827 // 2. FA 三板斧:
3828 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3829 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3830 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3831 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3832 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3833 //    - 优点: 减少 warp 间通信和 shuffle
3834 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691):
3835 //    for each K,V tile:
3836 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3837 //        m_new = max(m_old, row_max(S_cur * scale))
3838 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3839 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3840 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3841 //        O_final = O_new / l_final
3842 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?

```

```

3843 // - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3844 // - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3845 // 后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3846 // - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3847 // 都天然同构”的通用结论
3848 //
3849 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3850 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3851 // Grid: ((QKV_seqlen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
3852 // Block: (128, 1, 1), kNumThreads=kWarpSize*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
3853 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3854
3855 // ---- 寄存器填充辅助函数 ----
3856 template <typename T, int M, const int N, const int K = 2>
3857 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3858 #pragma unroll
3859     for (int i = 0; i < M; ++i)
3860 #pragma unroll
3861         for (int j = 0; j < N; ++j)
3862 #pragma unroll
3863             for (int k = 0; k < K; ++k)
3864                 R[i][j][k] = val;
3865 }
3866
3867 template <typename T, int M, const int N = 2>
3868 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3869 #pragma unroll
3870     for (int i = 0; i < M; ++i)
3871 #pragma unroll
3872         for (int j = 0; j < N; ++j)
3873             R[i][j] = val;
3874 }
3875
3876 // =====
3877 // FlashAttention-2 Split-Q Kernel (完整实现)
3878 // =====
3879 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3880 //
3881 // Tile 设计 (以 kHeadDim=64 为例) :
3882 // Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3883 // Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3884 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3885 //
3886 // 执行流程:
3887 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3888 // 2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3889 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3890 // 4) 3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
3891 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3892 // → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3893 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3894 // → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3895 // 7) 3e: Online rescaling —  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$ 
3896 // 8) 最终 rescale:  $O_{final} = (1/l_{final}) * O_{final}$ 
3897 // 9) Epilogue: warp shuffle + 128-bit collective store
3898
3899 template <
3900     const int kHeadDim,          // head dim: 32, 64, 128
3901     const int kMmaAtomM,         // 16 (MMA instruction M dimension)
3902     const int kMmaAtomN,         // 8 (MMA instruction N dimension)
3903     const int kMmaAtomK,         // 16 (MMA instruction K dimension)
3904     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
3905     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8

```

```

3906     const int kMmaTileSeqLenP, // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3907     const int kMmaTileHeadDimV, // MMA tiles for P@V N dim (head dim direction)
3908     const int kValTileSeqLenQ, // value tiles along Q's M, 1 → Br per warp=16
3909     const int kValTileSeqLenK, // value tiles along K's N, 8 → Bc_warp=8*8=64
3910     const int kValTileSeqLenP, // value tiles for P@V M dim, 1
3911     const int kValTileHeadDimV, // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3912     const int kStage, // pipeline stages for K: >= 1
3913     const int kPad> // padding for bank conflict avoidance
3914 __global__ void __launch_bounds__(kWarpSize *kMmaTileSeqLenQ *kMmaTileSeqLenK)
3915     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
3916                                   int QKV_seqlen, int QKV_head) {
3917     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3918     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3919     constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3920     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
3921     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3922     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3923     const float scale = 1.0f / sqrtf((float)kHeadDim);
3924
3925     // Block indexing
3926     const int QKV_batch_id = blockIdx.y / QKV_head;
3927     const int QKV_head_id = blockIdx.y % QKV_head;
3928     const int Q_tile_id = blockIdx.x;
3929     const int tid = threadIdx.x;
3930     const int warp_id = tid / kWarpSize;
3931     const int lane_id = tid % kWarpSize;
3932     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3933     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3934
3935     // Global memory base offsets for this (batch, head)
3936     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3937     const int Q_gmem_offset =
3938         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
3939         kHeadDim;
3940     const int K_gmem_offset = Q_gmem_offset;
3941     const int V_gmem_offset = Q_gmem_offset;
3942     const int O_gmem_offset = Q_gmem_offset;
3943
3944     // Thread-to-smem mapping for cooperative load
3945     int load_smem_Q_Br = tid / (kNumThreads / Br);
3946     int load_smem_Q_d =
3947         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3948     int load_smem_K_Bc = tid / (kNumThreads / Bc);
3949     int load_smem_K_d =
3950         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3951     int load_smem_V_Bc = tid / (kNumThreads / Bc);
3952     int load_smem_V_d =
3953         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3954
3955     int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
3956     if (load_gmem_Q_Br >= QKV_seqlen)
3957         return;
3958
3959     // ---- Shared memory layout ----
3960     extern __shared__ half smem[];
3961     constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
3962     constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
3963     half *Q_tile_smem = smem;
3964     half *K_tile_smem = Q_tile_smem + Q_tile_size;
3965     half *V_tile_smem = K_tile_smem + kStage * KV_tile_size; // kStage copies of K
3966     // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
3967     // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
3968

```

```

3969 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
3970 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
3971 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
3972
3973 // ---- Online Softmax persistent state ----
3974 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
3975 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
3976 float lane_block_row_max_old[kValTileSeqLenQ][2];
3977 float lane_block_row_sum_old[kValTileSeqLenQ][2];
3978 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
3979 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
3980
3981 // ---- Register allocation ----
3982 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
3983 uint32_t R_K[kValTileSeqLenK][2]; // K regs
3984 uint32_t R_V[kValTileHeadDimV][2]; // V regs
3985 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
3986 // 对单个 m16n8k16 tile 而言:
3987 //   - reg[0] 持有该 tile 前 8 行里的两个 half 值
3988 //   - reg[1] 持有该 tile 后 8 行里的两个 half 值
3989 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
3990 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
3991 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
3992 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
3993           [2]; // O accumulator (final output)
3994
3995 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
3996 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
3997
3998 // =====
3999 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
4000 // =====
4001 {
4002     int load_gmem_Q_addr =
4003         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
4004     uint32_t load_smem_Q_ptr =
4005         smem_Q_base_ptr +
4006         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
4007 #pragma unroll
4008     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4009         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
4010     }
4011     CP_ASYNC_COMMIT_GROUP();
4012 }
4013
4014 // =====
4015 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
4016 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
4017 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
4018 // =====
4019 if constexpr (kStage > 1) {
4020 #pragma unroll
4021     for (int stage = 0; stage < (kStage - 1); ++stage) {
4022         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
4023         int load_gmem_K_addr =
4024             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4025         uint32_t load_smem_K_ptr =
4026             smem_K_base_ptr +
4027             (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
4028              load_smem_K_d) *
4029             sizeof(half);
4030 #pragma unroll
4031         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {

```

```

4032     CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4033 }
4034 CP_ASYNC_COMMIT_GROUP();
4035 }
4036 CP_ASYNC_WAIT_GROUP(kStage - 2);
4037 __syncthreads();
4038 }
4039
4040 // =====
4041 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
4042 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
4043 // =====
4044 #pragma unroll 1
4045 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
4046     int smem_sel = tile_K_seqlen % kStage;
4047     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
4048
4049     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
4050     if constexpr (kStage > 1) {
4051         // 只有 kStage>1 才能真正做 K 的 pipeline:
4052         // smem_sel 负责“当前正在计算”的 K tile, smem_sel_next 负责“下一轮预取”的 K tile。
4053         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
4054         // Load current V tile (no pipeline for V — one stage is enough)
4055         {
4056             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
4057             int load_gmem_V_addr =
4058                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
4059             uint32_t load_smem_V_ptr =
4060                 smem_V_base_ptr +
4061                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
4062 #pragma unroll
4063             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4064                 CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
4065             }
4066             CP_ASYNC_COMMIT_GROUP();
4067         }
4068
4069         // Prefetch next K tile (pipelined)
4070         if ((tile_K_seqlen + 1) < Tc) {
4071             int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
4072             int load_gmem_K_addr =
4073                 K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4074             uint32_t load_smem_K_ptr =
4075                 smem_K_base_ptr +
4076                 (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
4077                  load_smem_K_d) *
4078                 sizeof(half);
4079 #pragma unroll
4080             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4081                 CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4082             }
4083             CP_ASYNC_COMMIT_GROUP();
4084         }
4085     }
4086
4087     // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
4088     fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
4089 #pragma unroll
4090     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
4091         // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
4092 #pragma unroll
4093         for (int i = 0; i < kValTileSeqLenQ; ++i) {
4094             int warp_smem_Q_Br =

```

```

4095     warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
4096     int lane_smem_Q_Br =
4097         warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
4098     int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
4099     uint32_t lane_smem_Q_ptr =
4100         smem_Q_base_ptr +
4101         (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
4102     LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4103         lane_smem_Q_ptr);
4104 }
4105
4106 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
4107 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
4108 #pragma unroll
4109 for (int j = 0; j < kValTileSeqLenK; ++j) {
4110     int warp_smem_K_Bc =
4111         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
4112     int lane_smem_K_Bc =
4113         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
4114     int lane_smem_K_d =
4115         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
4116     uint32_t lane_smem_K_ptr =
4117         smem_K_base_ptr +
4118         (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
4119             lane_smem_K_d) *
4120             sizeof(half);
4121     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
4122 }
4123
4124 // MMA: S[tile] += Q[tile] @ K^T[tile]
4125 #pragma unroll
4126 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4127     #pragma unroll
4128     for (int j = 0; j < kValTileSeqLenK; ++j) {
4129         HMMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
4130             R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
4131             R_S[i][j][1]);
4132     }
4133 }
4134 } // end loop over d
4135 __syncthreads();
4136
4137 // =====
4138 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
4139 // =====
4140 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
4141 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
4142 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
4143 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
4144 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
4145 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
4146 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
4147 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
4148 // kValTileSeqLenK tiles.
4149
4150 float lane_row_max_new[kValTileSeqLenQ][2];
4151 float lane_row_sum_new[kValTileSeqLenQ][2];
4152 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
4153 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
4154
4155 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
4156 #pragma unroll
4157 for (int i = 0; i < kValTileSeqLenQ; ++i) {

```

```

4158 #pragma unroll
4159 for (int j = 0; j < kValTileSeqLenK; ++j) {
4160     // Extract half values from R_S registers (C matrix fragment layout)
4161     float2 t_reg_S_0 =
4162         __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
4163     float2 t_reg_S_1 =
4164         __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
4165     // S = (Q@K^T) * scale
4166     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
4167     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
4168     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
4169     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
4170 }
4171 // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
4172 // {0,4,8,...,28} hold valid data)
4173 lane_row_max_new[i][0] =
4174     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
4175 lane_row_max_new[i][1] =
4176     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
4177 }
4178
4179 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
4180 // 面试关键点: 这里将 P 写回 R_S 寄存器!
4181 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
4182 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
4183 #pragma unroll
4184 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4185     // m_new = max(m_old, m_cur)
4186     float block_row_max_new_0 =
4187         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
4188     float block_row_max_new_1 =
4189         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
4190
4191     #pragma unroll
4192     for (int j = 0; j < kValTileSeqLenK; ++j) {
4193         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
4194         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
4195
4196         // P = exp(S * scale - m_new), 用 fma 保证精度
4197         t_reg_S_0.x =
4198             __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
4199         t_reg_S_0.y =
4200             __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
4201         t_reg_S_1.x =
4202             __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
4203         t_reg_S_1.y =
4204             __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
4205
4206         // Accumulate row sums
4207         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
4208         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
4209
4210         // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
4211         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
4212         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
4213     }
4214
4215     // Warp-level reduce sum (kWarpWidth = 4, same as max)
4216     lane_row_sum_new[i][0] =
4217         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4218     lane_row_sum_new[i][1] =
4219         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4220 }

```

```

4221 __syncthreads();
4222
4223 // =====
4224 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = 0[Br,d]
4225 // =====
4226 // Wait for V to be ready before computing P@V
4227 if constexpr (kStage > 1) {
4228     if ((tile_K_seqlen + 1) < Tc) {
4229         CP_ASYNC_WAIT_GROUP(1);
4230     } else {
4231         CP_ASYNC_WAIT_GROUP(0);
4232     }
4233 } else {
4234     CP_ASYNC_WAIT_GROUP(0);
4235 }
4236 __syncthreads();
4237
4238 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
4239
4240 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4241 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4242 #pragma unroll
4243 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4244     // ldmatrix.x2.trans: load V[Bc,d] with transposition
4245     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4246     // transposed ldmatrix
4247 #pragma unroll
4248 for (int j = 0; j < kValTileHeadDimV; ++j) {
4249     int warp_smem_V_d =
4250         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4251     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4252     int lane_smem_V_d = warp_smem_V_d;
4253     uint32_t lane_smem_V_ptr =
4254         smem_V_base_ptr +
4255         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
4256     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4257 }
4258
4259 // P matrix layout in R_S[i][j][2]:
4260 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4261 // | 64x64 | warp_KV 0 |
4262 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4263 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4264 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4265 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4266 // tile_V_Bc selects which 16-column slice of P to use:
4267 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
4268 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
4269 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
4270 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
4271 // 对应的 MMA A fragment 布局可以这样记:
4272 // - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
4273 // - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
4274 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
4275 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
4276 // fragment 都无条件成立的通用结论。layout转换逻辑:
4277 // C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
4278 int w = tile_V_Bc * 2;
4279 #pragma unroll
4280 for (int i = 0; i < kValTileSeqLenP; ++i) {
4281 #pragma unroll
4282 for (int j = 0; j < kValTileHeadDimV; ++j) {
4283     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output

```

```

4284         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
4285         R_V[j][0], R_V[j][1], // B fragment = V
4286         R_0[i][j][0], R_0[i][j][1]); // C fragment output
4287     }
4288 }
4289 } // end for tile_V_Bc
4290 __syncthreads();
4291
4292 // =====
4293 // 3e: Online rescaling —  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$ 
4294 // =====
4295 // 公式来源: FA2 paper Eq.(7-8), 使用  $\exp(m_{old} - m_{new})$  做 0 与 1 的 rescale
4296 #pragma unroll
4297 for (int i = 0; i < kValTileSeqLenP; ++i) {
4298     float block_row_max_new_0 = lane_row_max_new[i][0];
4299     float block_row_max_new_1 = lane_row_max_new[i][1];
4300     float block_row_sum_new_0 = lane_row_sum_new[i][0];
4301     float block_row_sum_new_1 = lane_row_sum_new[i][1];
4302
4303     float block_row_max_old_0 = lane_block_row_max_old[i][0];
4304     float block_row_max_old_1 = lane_block_row_max_old[i][1];
4305
4306     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
4307     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
4308
4309     // Handle first iteration:  $m_{old} = -inf$ , need to use  $m_{new}$  directly
4310     block_row_max_old_0 =
4311         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
4312     block_row_max_old_1 =
4313         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
4314
4315     float rescale_o_factor_0 =
4316         __expf(block_row_max_old_0 - block_row_max_new_0);
4317     float rescale_o_factor_1 =
4318         __expf(block_row_max_old_1 - block_row_max_new_1);
4319
4320     // Rescale  $O_{old}$  + Add  $P@V$  in one fused step
4321 #pragma unroll
4322     for (int j = 0; j < kValTileHeadDimV; ++j) {
4323         //  $R_0 / R_D$  与前面的  $R_S$  一样, 都按 MMA C fragment 布局解释:
4324         //   reg[0] -> rows 0~7 的 {c0,c1}
4325         //   reg[1] -> rows 8~15 的 {c2,c3}
4326         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
4327         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
4328         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4329         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4330
4331         //  $O_{new} = \exp(m_{old} - m_{new}) * O_{old} + P@V$  (fused multiply-add)
4332         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
4333         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
4334         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
4335         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
4336
4337         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4338         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4339     }
4340
4341     // Update  $l$ :  $l_{new} = \exp(m_{old} - m_{new}) * l_{old} + row\_sum(P)$ 
4342     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
4343     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
4344     lane_block_row_sum_old[i][0] = __fmaf_rn(
4345         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
4346     lane_block_row_sum_old[i][1] = __fmaf_rn(

```

```

4347         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
4348
4349         // Update m
4350         lane_block_row_max_old[i][0] = block_row_max_new_0;
4351         lane_block_row_max_old[i][1] = block_row_max_new_1;
4352     }
4353
4354     // Wait for next K tile to be ready in smem before next iteration
4355     if constexpr (kStage > 1) {
4356         if ((tile_K_seqlen + 1) < Tc) {
4357             CP_ASYNC_WAIT_GROUP(0);
4358         }
4359         __syncthreads();
4360     }
4361 } // end outer loop over K seqlen
4362 __syncthreads();
4363
4364 // =====
4365 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
4366 // =====
4367 #pragma unroll
4368 for (int i = 0; i < kValTileSeqLenP; ++i) {
4369     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4370     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4371     #pragma unroll
4372     for (int j = 0; j < kValTileHeadDimV; ++j) {
4373         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4374         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4375         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
4376         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4377         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4378         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
4379         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4380         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4381     }
4382 }
4383
4384 // =====
4385 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4386 // =====
4387 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4388 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4389 #pragma unroll
4390 for (int i = 0; i < kValTileSeqLenP; ++i) {
4391     #pragma unroll
4392     for (int j = 0; j < kValTileHeadDimV; ++j) {
4393         uint32_t R_Z[2][4];
4394         R_Z[0][0] = R_D[i][j][0];
4395         R_Z[1][0] = R_D[i][j][1];
4396         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4397         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4398         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4399         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4400         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
4401         R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
4402
4403         // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4404         if (lane_id % 4 == 0) {
4405             int store_warp_regs_0_Br =
4406                 warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4407             int store_lane_gmem_0_Br =
4408                 Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4409             int store_warp_regs_0_d =

```

```

4410     warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4411     int store_lane_gmem_0_d = store_warp_regs_0_d;
4412     int store_gmem_0_addr_0 = 0_gmem_offset +
4413         (store_lane_gmem_0_Br + 0) * kHeadDim +
4414         store_lane_gmem_0_d;
4415     int store_gmem_0_addr_1 = 0_gmem_offset +
4416         (store_lane_gmem_0_Br + 8) * kHeadDim +
4417         store_lane_gmem_0_d;
4418     // LDST128BITS = reinterpret_cast<float4*>
4419     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4420         *reinterpret_cast<float4*>(&R_Z[0][0]);
4421     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4422         *reinterpret_cast<float4*>(&R_Z[1][0]);
4423 }
4424 }
4425 }
4426 }
4427
4428 // =====
4429 // 以下是测试代码, 验证 Phase 1 - Phase 8 的kernel的正确性, 不评估性能。
4430 // =====
4431
4432 static inline void check(cudaError_t err, const char *msg) {
4433     if (err != cudaSuccess) {
4434         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
4435         exit(EXIT_FAILURE);
4436     }
4437 }
4438
4439
4440 static void test_block_reduce(int N) {
4441
4442     srand(42);
4443     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4444     for (int i = 0; i < N; i++)
4445         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4446
4447     // CPU reference: sum of all elements
4448     double ref = 0.0;
4449     for (int i = 0; i < N; i++) ref += (double)h_a[i];
4450
4451     float *d_a, *d_y;
4452     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4453     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4454
4455     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
4456     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4457
4458     dim3 block(128);
4459     dim3 grid((N + 127) / 128);
4460     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4461     check(cudaGetLastError(), "blockreduce launch");
4462     check(cudaDeviceSynchronize(), "blockreduce sync");
4463
4464     float result;
4465     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4466
4467     float err = fabsf(result - (float)ref);
4468     printf("| %-42s | %.6e | %-4s | %-8s |\n", "BlockReduce", err,
4469         err < 1e-2f ? "PASS" : "FAIL", "None");
4470
4471     free(h_a);
4472     cudaFree(d_a); cudaFree(d_y);

```

```

4473 }
4474
4475
4476 static void test_dot(int N) {
4477
4478     srand(42);
4479     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4480     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4481     for (int i = 0; i < N; i++) {
4482         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4483         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4484     }
4485
4486     // CPU reference
4487     double ref = 0.0;
4488     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4489
4490     float *d_a, *d_b, *d_y;
4491     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4492     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4493     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4494
4495     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4496     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4497     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4498
4499     dim3 block(128);
4500     dim3 grid((N + 127) / 128);
4501     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4502     check(cudaGetLastError(), "dot launch");
4503     check(cudaDeviceSynchronize(), "dot sync");
4504
4505     float result;
4506     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4507
4508     float err = fabsf(result - (float)ref);
4509     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot", err,
4510         err < 1e-2f ? "PASS" : "FAIL", "None");
4511
4512     // ---- Dot Vec4 ----
4513     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4514     dim3 block_v4(32);
4515     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4516     check(cudaGetLastError(), "dot_vec4 launch");
4517     check(cudaDeviceSynchronize(), "dot_vec4 sync");
4518
4519     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4520     float err_v4 = fabsf(result - (float)ref);
4521     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL", "None");
4522
4523     free(h_a); free(h_b);
4524     cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4525 }
4526
4527
4528 static void test_relu(int N) {
4529     srand(42);
4530     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4531     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4532     for (int i = 0; i < N; i++)
4533         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4534
4535     float *d_x, *d_y;

```

```

4536 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4537 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4538 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4539
4540 for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4541 dim3 block256(256);
4542 dim3 grid256((N + 255) / 256);
4543 relu<<<grid256, block256>>>(d_x, d_y, N);
4544 check(cudaGetLastError(), "relu launch");
4545 check(cudaDeviceSynchronize(), "relu sync");
4546 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4547 float max_err = 0.0f;
4548 for (int i = 0; i < N; i++) {
4549     float expected = fmaxf(0.0f, h_x[i]);
4550     float err = fabsf(h_y[i] - expected);
4551     if (err > max_err) max_err = err;
4552 }
4553 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4554
4555 dim3 block64(64);
4556 relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4557 check(cudaGetLastError(), "relu_vec4 launch");
4558 check(cudaDeviceSynchronize(), "relu_vec4 sync");
4559 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4560 max_err = 0.0f;
4561 for (int i = 0; i < N; i++) {
4562     float expected = fmaxf(0.0f, h_x[i]);
4563     float err = fabsf(h_y[i] - expected);
4564     if (err > max_err) max_err = err;
4565 }
4566 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4567 );
4568 free(h_x); free(h_y);
4569 cudaFree(d_x); cudaFree(d_y);
4570 }
4571
4572
4573 static void test_elementwise(int N) {
4574     srand(42);
4575     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4576     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4577     for (int i = 0; i < N; i++) {
4578         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4579         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4580     }
4581
4582     float *d_a, *d_b, *d_c;
4583     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4584     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4585     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4586     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4587     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4588
4589     dim3 block256(256);
4590     dim3 grid256((N + 255) / 256);
4591     elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4592     check(cudaGetLastError(), "eadd launch");
4593     check(cudaDeviceSynchronize(), "eadd sync");
4594     float *h_c = (float *)malloc((size_t)N * sizeof(float));
4595     check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4596     float max_err = 0.0f;
4597     for (int i = 0; i < N; i++) {

```

```

4598     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4599     if (err > max_err) max_err = err;
4600 }
4601 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4602
4603 dim3 block64(64);
4604 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4605 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4606 check(cudaGetLastError(), "eadd_vec4 launch");
4607 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4608 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4609 max_err = 0.0f;
4610 for (int i = 0; i < N; i++) {
4611     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4612     if (err > max_err) max_err = err;
4613 }
4614 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
    "None");
4615
4616 free(h_a); free(h_b); free(h_c);
4617 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4618 }
4619
4620
4621 static void test_histogram(int N) {
4622     srand(42);
4623     int BINS = 16;
4624     int *h_hist = (int *)calloc(BINS, sizeof(int));
4625     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4626     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4627     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4628     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4629
4630     int *d_idx, *d_hist;
4631     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4632     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4633     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4634     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4635
4636     dim3 block256(256);
4637     dim3 grid256((N + 255) / 256);
4638     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4639     check(cudaGetLastError(), "histogram launch");
4640     check(cudaDeviceSynchronize(), "histogram sync");
4641     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4642
4643     float max_err = 0.0f;
4644     for (int i = 0; i < BINS; i++) {
4645         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4646         if (err > max_err) max_err = err;
4647     }
4648     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
    );
4649
4650     free(h_hist); free(h_hist_ref); free(h_idx);
4651     cudaFree(d_idx); cudaFree(d_hist);
4652 }
4653
4654
4655 static void test_merge_attn_states(int num_tokens, int num_heads,
4656                                   int head_size) {
4657

```

```

4658 size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4659 size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4660
4661 float *h_prefix_out = (float *)malloc(out_size);
4662 float *h_suffix_out = (float *)malloc(out_size);
4663 float *h_prefix_lse = (float *)malloc(lse_size);
4664 float *h_suffix_lse = (float *)malloc(lse_size);
4665 float *h_output_ref = (float *)malloc(out_size);
4666
4667 srand(42);
4668 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4669     h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4670     h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4671 }
4672 for (int i = 0; i < num_heads * num_tokens; i++) {
4673     h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4674     h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4675 }
4676
4677 // CPU reference: 与 kernel 完全相同的浮点计算
4678 for (int t = 0; t < num_tokens; t++) {
4679     for (int h = 0; h < num_heads; h++) {
4680         float p_lse = h_prefix_lse[h * num_tokens + t];
4681         float s_lse = h_suffix_lse[h * num_tokens + t];
4682         p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4683         s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4684
4685         float max_lse = fmaxf(p_lse, s_lse);
4686         p_lse -= max_lse;
4687         s_lse -= max_lse;
4688         float p_se = expf(p_lse);
4689         float s_se = expf(s_lse);
4690         float p_scale = p_se / (p_se + s_se);
4691         float s_scale = s_se / (p_se + s_se);
4692
4693         int head_off = t * num_heads * head_size + h * head_size;
4694         for (int d = 0; d < head_size; d++) {
4695             h_output_ref[head_off + d] =
4696                 h_prefix_out[head_off + d] * p_scale +
4697                 h_suffix_out[head_off + d] * s_scale;
4698         }
4699     }
4700 }
4701
4702 float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4703 check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4704 check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4705 check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4706 check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4707 check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4708
4709 check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4710                 cudaMemcpyHostToDevice),
4711        "merge_attn H2D p_out");
4712 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4713                 cudaMemcpyHostToDevice),
4714        "merge_attn H2D s_out");
4715 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4716                 cudaMemcpyHostToDevice),
4717        "merge_attn H2D p_lse");
4718 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4719                 cudaMemcpyHostToDevice),
4720        "merge_attn H2D s_lse");

```

```

4721
4722 int threads_per_head = head_size / 4;
4723 int total_threads = num_tokens * num_heads * threads_per_head;
4724 dim3 block(128);
4725 dim3 grid((total_threads + 127) / 128);
4726 merge_attn_states<<<grid, block>>>(  

4727     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,  

4728     num_tokens, num_heads, head_size);  

4729 check(cudaGetLastError(), "merge_attn launch");  

4730 check(cudaDeviceSynchronize(), "merge_attn sync");  

4731  

4732 float *h_output = (float *)malloc(out_size);  

4733 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),  

4734     "merge_attn D2H");  

4735  

4736 float max_err = 0.0f;  

4737 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {  

4738     float err = fabsf(h_output[i] - h_output_ref[i]);  

4739     if (err > max_err) max_err = err;  

4740 }  

4741 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates", max_err,  

4742     max_err < 1e-4f ? "PASS" : "FAIL", "None");  

4743  

4744 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)  

4745 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf  

4746 h_suffix_lse[0] = 0.0f;  

4747 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,  

4748     cudaMemcpyHostToDevice),  

4749     "merge_attn H2D inf lse");  

4750 merge_attn_states<<<grid, block>>>(  

4751     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,  

4752     num_tokens, num_heads, head_size);  

4753 check(cudaGetLastError(), "merge_attn inf launch");  

4754 check(cudaDeviceSynchronize(), "merge_attn inf sync");  

4755 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),  

4756     "merge_attn inf D2H");  

4757  

4758 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )  

4759 float inf_err = 0.0f;  

4760 for (int d = 0; d < head_size; d++) {  

4761     float err = fabsf(h_output[d] - h_suffix_out[d]);  

4762     if (err > inf_err) inf_err = err;  

4763 }  

4764 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates-inf", inf_err,  

4765     inf_err < 1e-4f ? "PASS" : "FAIL", "None");  

4766  

4767 free(h_prefix_out);  

4768 free(h_suffix_out);  

4769 free(h_prefix_lse);  

4770 free(h_suffix_lse);  

4771 free(h_output_ref);  

4772 free(h_output);  

4773 cudaFree(d_prefix_out);  

4774 cudaFree(d_suffix_out);  

4775 cudaFree(d_prefix_lse);  

4776 cudaFree(d_suffix_lse);  

4777 cudaFree(d_output);  

4778 }  

4779  

4780  

4781 static void test_softmax(int N) {  

4782     // Use N = blockDim.x so one block processes all elements as one token.  

4783     constexpr int kNumThreads = 256;  


```

```

4784 if (N != kNumThreads) {
4785     printf("    OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
4786     return;
4787 }
4788
4789 srand(42);
4790 float *h_x = (float *)malloc((size_t)N * sizeof(float));
4791 float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4792 for (int i = 0; i < N; i++)
4793     h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4794
4795 // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4796 float max_val = -FLT_MAX;
4797 for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4798 double sum_exp = 0.0;
4799 for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4800 for (int i = 0; i < N; i++)
4801     h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4802
4803 float *d_x, *d_y;
4804 check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4805 check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4806
4807 check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4808
4809 dim3 block(256);
4810 dim3 grid(1); // one token covering all N elements
4811 online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4812 check(cudaGetLastError(), "softmax launch");
4813 check(cudaDeviceSynchronize(), "softmax sync");
4814
4815 float *h_y = (float *)malloc((size_t)N * sizeof(float));
4816 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4817
4818 float max_err = 0.0f;
4819 for (int i = 0; i < N; i++) {
4820     float err = fabsf(h_y[i] - h_y_ref[i]);
4821     if (err > max_err) max_err = err;
4822 }
4823 printf("| %-42s | %.6e | %-4s | %-8s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
4824     , "None");
4825
4826 // ---- Safe Softmax ----
4827 safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4828 check(cudaGetLastError(), "safe_softmax launch");
4829 check(cudaDeviceSynchronize(), "safe_softmax sync");
4830 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4831 max_err = 0.0f;
4832 for (int i = 0; i < N; i++) {
4833     float err = fabsf(h_y[i] - h_y_ref[i]);
4834     if (err > max_err) max_err = err;
4835 }
4836 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
4837     None");
4838
4839 // ---- Naive Softmax ----
4840 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4841 check(cudaGetLastError(), "naive_softmax launch");
4842 check(cudaDeviceSynchronize(), "naive_softmax sync");
4843 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4844 max_err = 0.0f;
4845 for (int i = 0; i < N; i++) {
4846     float err = fabsf(h_y[i] - h_y_ref[i]);

```

```

4845     if (err > max_err) max_err = err;
4846 }
4847 printf("| %-42s | %.6e | %-4s | %-8s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
4848     None");
4849 free(h_x); free(h_y); free(h_y_ref);
4850 cudaFree(d_x); cudaFree(d_y);
4851 }
4852
4853
4854 static void test_rms_norm(int N, int K) {
4855
4856     srand(42);
4857     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4858     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4859     for (int i = 0; i < N * K; i++)
4860         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4861     float g = 1.5f; // gain
4862
4863     // CPU reference: y = (x / rms(x)) * g
4864     float epsilon = 1e-5f;
4865     for (int n = 0; n < N; n++) {
4866         double sum_sq = 0.0;
4867         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
4868         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
4869         for (int k = 0; k < K; k++)
4870             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
4871     }
4872
4873     float *d_x, *d_y;
4874     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
4875     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
4876     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
4877
4878     dim3 block(128);
4879     dim3 grid(N);
4880     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
4881     check(cudaGetLastError(), "rmsnorm launch");
4882     check(cudaDeviceSynchronize(), "rmsnorm sync");
4883
4884     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4885     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
4886
4887     float max_err = 0.0f;
4888     for (int i = 0; i < N * K; i++) {
4889         float err = fabsf(h_y[i] - h_y_ref[i]);
4890         if (err > max_err) max_err = err;
4891     }
4892     printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4893
4894     // ---- RMS Norm Vec4 ----
4895     dim3 block_rv4(32);
4896     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
4897     check(cudaGetLastError(), "rmsnorm_vec4 launch");
4898     check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
4899     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
4900     max_err = 0.0f;
4901     for (int i = 0; i < N * K; i++) {
4902         float err = fabsf(h_y[i] - h_y_ref[i]);
4903         if (err > max_err) max_err = err;
4904     }
4905     printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

```

```

4906     free(h_x); free(h_y); free(h_y_ref);
4907     cudaFree(d_x); cudaFree(d_y);
4908 }
4909
4910
4911
4912 static void test_layer_norm(int N, int K) {
4913
4914     srand(42);
4915     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4916     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4917     for (int i = 0; i < N * K; i++)
4918         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4919     float g = 1.5f, b = 0.3f; // gain and bias
4920
4921     // CPU reference: y = ((x - mean) / std) * g + b
4922     float epsilon = 1e-5f;
4923     for (int n = 0; n < N; n++) {
4924         double sum = 0.0;
4925         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
4926         float mean = (float)sum / (float)K;
4927         double sum_sq = 0.0;
4928         for (int k = 0; k < K; k++) {
4929             float diff = h_x[n * K + k] - mean;
4930             sum_sq += (double)diff * (double)diff;
4931         }
4932         float std = sqrtf((float)sum_sq / (float)K + epsilon);
4933         for (int k = 0; k < K; k++)
4934             h_y_ref[n * K + k] = (h_x[n * K + k] - mean) / std * g + b;
4935     }
4936
4937     float *d_x, *d_y;
4938     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
4939     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
4940     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
4941
4942     dim3 block(128);
4943     dim3 grid(N);
4944     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
4945     check(cudaGetLastError(), "layernorm launch");
4946     check(cudaDeviceSynchronize(), "layernorm sync");
4947
4948     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
4949     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
4950
4951     float max_err = 0.0f;
4952     for (int i = 0; i < N * K; i++) {
4953         float err = fabsf(h_y[i] - h_y_ref[i]);
4954         if (err > max_err) max_err = err;
4955     }
4956     printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
4957         );
4958
4959     // ---- Layer Norm Vec4 ----
4960     dim3 block_lv4(32);
4961     layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
4962     check(cudaGetLastError(), "layernorm_vec4 launch");
4963     check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
4964     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
4965     max_err = 0.0f;
4966     for (int i = 0; i < N * K; i++) {
4967         float err = fabsf(h_y[i] - h_y_ref[i]);
4968         if (err > max_err) max_err = err;
4969     }

```

```

4968 }
4969 printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4970
4971 free(h_x); free(h_y); free(h_y_ref);
4972 cudaFree(d_x); cudaFree(d_y);
4973 }
4974
4975
4976 static void test_rope(int seq_len, int N) {
4977
4978     int total_pairs = seq_len * N;
4979     int total_elems = total_pairs * 2;
4980     size_t size = (size_t)total_elems * sizeof(float);
4981
4982     float *h_x = (float *)malloc(size);
4983     float *h_y_ref = (float *)malloc(size);
4984
4985     srand(42);
4986     for (int i = 0; i < total_elems; i++)
4987         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4988
4989     // CPU reference: 2D rotation for each pair
4990     for (int idx = 0; idx < total_pairs; idx++) {
4991         int token_pos = idx / N;
4992         int token_idx = idx % N;
4993         float x1 = h_x[idx * 2];
4994         float x2 = h_x[idx * 2 + 1];
4995         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
4996         float angle = (float)token_pos * theta;
4997         float cos_v = cosf(angle);
4998         float sin_v = sinf(angle);
4999         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
5000         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
5001     }
5002
5003     float *d_x, *d_y;
5004     check(cudaMalloc(&d_x, size), "rope alloc X");
5005     check(cudaMalloc(&d_y, size), "rope alloc Y");
5006     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
5007
5008     dim3 block(256);
5009     dim3 grid((total_pairs + 255) / 256);
5010     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
5011     check(cudaGetLastError(), "rope launch");
5012     check(cudaDeviceSynchronize(), "rope sync");
5013
5014     float *h_y = (float *)malloc(size);
5015     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
5016
5017     float max_err = 0.0f;
5018     for (int i = 0; i < total_elems; i++) {
5019         float err = fabsf(h_y[i] - h_y_ref[i]);
5020         if (err > max_err) max_err = err;
5021     }
5022     printf("| %-42s | %.6e | %-4s | %-8s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5023
5024     free(h_x);
5025     free(h_y);
5026     free(h_y_ref);
5027     cudaFree(d_x);
5028     cudaFree(d_y);
5029 }

```

```

5030
5031
5032 static void test_mat_transpose(int row, int col) {
5033
5034     size_t size_in = (size_t)row * col * sizeof(float);
5035     size_t size_out = (size_t)col * row * sizeof(float);
5036
5037     float *h_x = (float *)malloc(size_in);
5038     float *h_y_ref = (float *)malloc(size_out);
5039
5040     srand(42);
5041     for (int i = 0; i < row * col; i++)
5042         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5043
5044     // CPU reference: y[j][i] = x[i][j] (row-major)
5045     for (int i = 0; i < row; i++)
5046         for (int j = 0; j < col; j++)
5047             h_y_ref[j * row + i] = h_x[i * col + j];
5048
5049     float *d_x, *d_y;
5050     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
5051     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
5052     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
5053
5054     dim3 block(16, 16);
5055     dim3 grid((col + 15) / 16, (row + 15) / 16);
5056     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
5057     check(cudaGetLastError(), "mattrans launch");
5058     check(cudaDeviceSynchronize(), "mattrans sync");
5059
5060     float *h_y = (float *)malloc(size_out);
5061     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
5062
5063     float max_err = 0.0f;
5064     for (int i = 0; i < col * row; i++) {
5065         float err = fabsf(h_y[i] - h_y_ref[i]);
5066         if (err > max_err) max_err = err;
5067     }
5068     printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "
5069         None");
5070
5071     free(h_x);
5072     free(h_y);
5073     free(h_y_ref);
5074     cudaFree(d_x);
5075     cudaFree(d_y);
5076 }
5077
5078 static void test_mat_transpose_padded(int row, int col) {
5079
5080     size_t size_in = (size_t)row * col * sizeof(float);
5081     size_t size_out = (size_t)col * row * sizeof(float);
5082
5083     float *h_x = (float *)malloc(size_in);
5084     float *h_y_ref = (float *)malloc(size_out);
5085
5086     srand(42);
5087     for (int i = 0; i < row * col; i++)
5088         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5089
5090     // CPU reference: y[j][i] = x[i][j] (row-major)
5091     for (int i = 0; i < row; i++)

```

```

5092     for (int j = 0; j < col; j++)
5093         h_y_ref[j * row + i] = h_x[i * col + j];
5094
5095     float *d_x, *d_y;
5096     check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
5097     check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
5098     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
5099
5100     dim3 block(16, 16);
5101     dim3 grid((col + 15) / 16, (row + 63) / 64);
5102     mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
5103     check(cudaGetLastError(), "mattrans_padded launch");
5104     check(cudaDeviceSynchronize(), "mattrans_padded sync");
5105
5106     float *h_y = (float *)malloc(size_out);
5107     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
5108
5109     float max_err = 0.0f;
5110     for (int i = 0; i < col * row; i++) {
5111         float err = fabsf(h_y[i] - h_y_ref[i]);
5112         if (err > max_err) max_err = err;
5113     }
5114     printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "None");
5115
5116     free(h_x);
5117     free(h_y);
5118     free(h_y_ref);
5119     cudaFree(d_x);
5120     cudaFree(d_y);
5121 }
5122
5123
5124 static void test_sgemv(int M, int K) {
5125
5126     srand(42);
5127     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
5128     float *h_x = (float *)malloc((size_t)K * sizeof(float));
5129     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5130     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5131     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5132
5133     // CPU reference: y[m] = sum_k A[m,k] * x[k]
5134     for (int m = 0; m < M; m++) {
5135         double sum = 0.0;
5136         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
5137         h_y_ref[m] = (float)sum;
5138     }
5139
5140     float *d_a, *d_x, *d_y;
5141     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
5142     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
5143     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
5144
5145     check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
5146     check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
5147
5148     dim3 block(32, 4);
5149     dim3 grid((M + 3) / 4);
5150     sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
5151     check(cudaGetLastError(), "sgemv launch");
5152     check(cudaDeviceSynchronize(), "sgemv sync");
5153

```

```

5154 float *h_y = (float *)malloc((size_t)M * sizeof(float));
5155 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
5156
5157 float max_err = 0.0f;
5158 for (int m = 0; m < M; m++) {
5159     float err = fabsf(h_y[m] - h_y_ref[m]);
5160     if (err > max_err) max_err = err;
5161 }
5162 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
");
5163
5164 // ---- SGEMV K32 ----
5165 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
5166 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
5167 check(cudaGetLastError(), "sgemv_k32 launch");
5168 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
5169
5170 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
5171
5172 max_err = 0.0f;
5173 for (int m = 0; m < M; m++) {
5174     float err = fabsf(h_y[m] - h_y_ref[m]);
5175     if (err > max_err) max_err = err;
5176 }
5177 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
");
5178
5179 // ---- SGEMV K16 ----
5180 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5181 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5182
5183 int K16 = 16;
5184 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
5185 h_x = (float *)malloc((size_t)K16 * sizeof(float));
5186 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5187 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5188 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5189
5190 for (int m = 0; m < M; m++) {
5191     double sum = 0.0;
5192     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
5193     h_y_ref[m] = (float)sum;
5194 }
5195
5196 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
5197 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
5198 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
5199
5200 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
5201 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
5202
5203 dim3 grid_k16((M + 7) / 8);
5204 sgemv_k16<<<2>>><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
5205 check(cudaGetLastError(), "sgemv_k16 launch");
5206 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
5207
5208 h_y = (float *)malloc((size_t)M * sizeof(float));
5209 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
5210
5211 max_err = 0.0f;
5212 for (int m = 0; m < M; m++) {
5213     float err = fabsf(h_y[m] - h_y_ref[m]);
5214     if (err > max_err) max_err = err;

```

```

5215 }
5216 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None"
5217 );
5218 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5219 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5220 }
5221
5222
5223 static void test_sgemm(int M, int N, int K) {
5224
5225     size_t size_a = (size_t)M * K * sizeof(float);
5226     size_t size_b = (size_t)K * N * sizeof(float);
5227     size_t size_c = (size_t)M * N * sizeof(float);
5228
5229     float *h_a = (float *)malloc(size_a);
5230     float *h_b = (float *)malloc(size_b);
5231     float *h_c_ref = (float *)malloc(size_c);
5232
5233     srand(42);
5234     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5235     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5236
5237     float *d_a, *d_b, *d_c;
5238     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
5239     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
5240     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
5241
5242     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
5243     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
5244
5245     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
5246     cublasHandle_t handle;
5247     cublasCreate(&handle);
5248     float alpha = 1.0f, beta = 0.0f;
5249     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5250                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
5251     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
5252
5253     // Kernel
5254     cudaEvent_t start, stop;
5255     cudaEventCreate(&start);
5256     cudaEventCreate(&stop);
5257
5258     dim3 block(32, 32);
5259     dim3 grid((N + 31) / 32, (M + 31) / 32);
5260     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
5261     check(cudaGetLastError(), "sgemm launch");
5262     check(cudaDeviceSynchronize(), "sgemm sync");
5263
5264     float *h_c = (float *)malloc(size_c);
5265     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
5266
5267     // Verify
5268     float max_err = 0.0f;
5269     for (int i = 0; i < M * N; i++) {
5270         float err = fabsf(h_c[i] - h_c_ref[i]);
5271         if (err > max_err) max_err = err;
5272     }
5273     printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5274
5275     // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
5276

```

```

5277 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
5278 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
5279 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
5280 check(cudaGetLastError(), "sgemm_vec4 launch");
5281 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
5282
5283 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
5284
5285 max_err = 0.0f;
5286 for (int i = 0; i < M * N; i++) {
5287     float err = fabsf(h_c[i] - h_c_ref[i]);
5288     if (err > max_err) max_err = err;
5289 }
5290 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None
    ");
5291
5292 free(h_a); free(h_b); free(h_c); free(h_c_ref);
5293 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
5294 cublasDestroy(handle);
5295 cudaEventDestroy(start);
5296 cudaEventDestroy(stop);
5297 }
5298
5299
5300 static void test_hgemm_mma(int M, int N, int K) {
5301
5302     size_t size_a = (size_t)M * K * sizeof(half);
5303     size_t size_b = (size_t)K * N * sizeof(half);
5304     size_t size_c = (size_t)M * N * sizeof(half);
5305
5306     half *h_a = (half *)malloc(size_a);
5307     half *h_b = (half *)malloc(size_b);
5308     half *h_c_ref = (half *)malloc(size_c);
5309
5310     srand(42);
5311     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5312     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5313
5314     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
5315     // We store h_b as B [K×N] row-major for cuBLAS, then create B^T for kernel.
5316     size_t size_b_t = (size_t)N * K * sizeof(half);
5317     half *h_b_t = (half *)malloc(size_b_t);
5318     for (int n = 0; n < N; n++)
5319         for (int k = 0; k < K; k++)
5320             h_b_t[n * K + k] = h_b[k * N + n];
5321
5322     half *d_a, *d_b, *d_b_t, *d_c;
5323     check(cudaMalloc(&d_a, size_a), "hgemm alloc A");
5324     check(cudaMalloc(&d_b, size_b), "hgemm alloc B (cuBLAS)");
5325     check(cudaMalloc(&d_b_t, size_b_t), "hgemm alloc B_t (kernel)");
5326     check(cudaMalloc(&d_c, size_c), "hgemm alloc C");
5327
5328     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm H2D A");
5329     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm H2D B (cuBLAS)");
5330     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
5331
5332     // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5333     // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
5334     cublasHandle_t handle;
5335     cublasCreate(&handle);
5336     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5337     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5338                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,

```

```

5339         &beta_h, d_c, CUDA_R_16F, N,
5340         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5341 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
5342
5343 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
5344 cudaEvent_t start, stop;
5345 cudaEventCreate(&start);
5346 cudaEventCreate(&stop);
5347
5348 constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
5349 size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
5350 dim3 block(256);
5351 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5352 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5353 check(cudaGetLastError(), "hgemm launch");
5354 check(cudaDeviceSynchronize(), "hgemm sync");
5355
5356 half *h_c = (half *)malloc(size_c);
5357 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
5358
5359 // Verify
5360 float max_err = 0.0f;
5361 for (int i = 0; i < M * N; i++) {
5362     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5363     if (err > max_err) max_err = err;
5364 }
5365 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL", "None")
5366 ;
5367
5368 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5369 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5370 cublasDestroy(handle);
5371 cudaEventDestroy(start);
5372 cudaEventDestroy(stop);
5373 }
5374
5375 static void test_hgemm_swizzle(int M, int N, int K) {
5376     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
5377     // + Register Double Buffering (kValTileK=2, BK=32)
5378     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5379     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
5380     // (kValTileK=2, kStages=3, BK=32)
5381     // smem: kStages × (BM×BK + BN×BK) halves
5382
5383     size_t size_a = (size_t)M * K * sizeof(half);
5384     size_t size_b = (size_t)K * N * sizeof(half);
5385     size_t size_c = (size_t)M * N * sizeof(half);
5386
5387     half *h_a = (half *)malloc(size_a);
5388     half *h_b = (half *)malloc(size_b);
5389     half *h_c_ref = (half *)malloc(size_c);
5390
5391     srand(42);
5392     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5393     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5394
5395     // Kernel expects B^T [N×K] row-major layout (TN layout convention).
5396     size_t size_b_t = (size_t)N * K * sizeof(half);
5397     half *h_b_t = (half *)malloc(size_b_t);
5398     for (int n = 0; n < N; n++)
5399         for (int k = 0; k < K; k++)
5400             h_b_t[n * K + k] = h_b[k * N + n];

```

```

5401
5402     half *d_a, *d_b, *d_b_t, *d_c;
5403     check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
5404     check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
5405     check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
5406     check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
5407
5408     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
5409     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
5410     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
5411
5412     // cuBLAS FP16 reference
5413     cublasHandle_t handle;
5414     cublasCreate(&handle);
5415     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5416     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5417                 &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5418                 &beta_h, d_c, CUDA_R_16F, N,
5419                 CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5420     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
5421
5422     // MMA swizzle kernel (default params: kStages=3, kValTileK=2, BK=32)
5423     constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;
5424     size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
5425     dim3 block(256);
5426     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5427     hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5428     check(cudaGetLastError(), "hgemm_swizzle launch");
5429     check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
5430
5431     half *h_c = (half *)malloc(size_c);
5432     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
5433
5434     // Verify
5435     float max_err = 0.0f;
5436     for (int i = 0; i < M * N; i++) {
5437         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5438         if (err > max_err) max_err = err;
5439     }
5440     printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "
5441         FAIL", "None");
5442
5443     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5444     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5445     cublasDestroy(handle);
5446 }
5447
5448 #if defined(NOTES_V2_ENABLE_CUTE)
5449 static void test_hgemm_cute(int M, int N, int K) {
5450     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3,3> + kStage=2
5451     // TN layout: C[MxN] = A[MxK] x B^T[NxK]
5452     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5453     // Tile: BM=128, BN=256, BK=32, 128 threads/block
5454
5455     size_t size_a = (size_t)M * K * sizeof(half);
5456     size_t size_b = (size_t)K * N * sizeof(half);
5457     size_t size_c = (size_t)M * N * sizeof(half);
5458
5459     half *h_a = (half *)malloc(size_a);
5460     half *h_b = (half *)malloc(size_b);
5461     half *h_c_ref = (half *)malloc(size_c);
5462

```

```

5463 srand(42);
5464 for (int i = 0; i < M * K; i++)
5465     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5466 for (int i = 0; i < K * N; i++)
5467     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5468
5469 // CuTe kernel expects B^T [N×K] row-major (TN layout).
5470 size_t size_b_t = (size_t)N * K * sizeof(half);
5471 half *h_b_t = (half *)malloc(size_b_t);
5472 for (int n = 0; n < N; n++)
5473     for (int k = 0; k < K; k++)
5474         h_b_t[n * K + k] = h_b[k * N + n];
5475
5476 half *d_a, *d_b, *d_b_t, *d_c;
5477 check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5478 check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5479 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5480 check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5481
5482 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5483 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5484 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5485
5486 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5487 cublasHandle_t handle;
5488 cublasCreate(&handle);
5489 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5490 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5491             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5492             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5493 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5494
5495 // CuTe kernel (Stages=2, TN layout)
5496 launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
5497 check(cudaGetLastError(), "hgemm_cute launch");
5498 check(cudaDeviceSynchronize(), "hgemm_cute sync");
5499
5500 half *h_c = (half *)malloc(size_c);
5501 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5502
5503 // Verify
5504 float max_err = 0.0f;
5505 for (int i = 0; i < M * N; i++) {
5506     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5507     if (err > max_err) max_err = err;
5508 }
5509 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM CuTe Swizzle + Reg2x",
5510        max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
5511
5512 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5513 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5514 cublasDestroy(handle);
5515 }
5516 #endif /* NOTES_V2_ENABLE_CUTE */
5517
5518
5519 #if defined(NOTES_V2_ENABLE_WGMMA)
5520 static void test_hgemm_wgmma(int M, int N, int K) {
5521     // HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
5522     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5523     // Kernel: hgemm_wgmma_stages_tn with default template params
5524
5525     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;

```

```

5526 // M, K must be divisible by tile dims
5527 if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5528     printf("| %-42s | %-12s | %-4s | %-8s |\n",
5529         "HGEMM WGMMA", "SKIP", "SKIP");
5530     return;
5531 }
5532
5533 size_t size_a = (size_t)M * K * sizeof(half);
5534 size_t size_b = (size_t)K * N * sizeof(half);
5535 size_t size_c = (size_t)M * N * sizeof(half);
5536
5537 half *h_a = (half *)malloc(size_a);
5538 half *h_b = (half *)malloc(size_b);
5539 half *h_c_ref = (half *)malloc(size_c);
5540
5541 srand(42);
5542 for (int i = 0; i < M * K; i++)
5543     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5544 for (int i = 0; i < K * N; i++)
5545     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5546
5547 // B^T [N×K] row-major for TN layout (same as hgemm_mma kernel)
5548 size_t size_b_t = (size_t)N * K * sizeof(half);
5549 half *h_b_t = (half *)malloc(size_b_t);
5550 for (int n = 0; n < N; n++)
5551     for (int k = 0; k < K; k++)
5552         h_b_t[n * K + k] = h_b[k * N + n];
5553
5554 half *d_a, *d_b, *d_b_t, *d_c;
5555 check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5556 check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5557 check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5558 check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5559
5560 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5561 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5562     "wgmma H2D B (cuBLAS)");
5563 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5564     "wgmma H2D B_t (kernel)");
5565
5566 // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5567 cublasHandle_t handle;
5568 cublasCreate(&handle);
5569 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5570 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5571     CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5572     CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5573 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5574     "wgmma D2H ref");
5575
5576 // Create TMA tensor maps for A and B^T
5577 // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5578 // B^T[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)
5579 CUTensorMap *tma_a =
5580     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5581 CUTensorMap *tma_b =
5582     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5583
5584 // Launch WGMMA kernel
5585 // kBlockSwizzle=false → 2D grid, no swizzle
5586 size_t smem_bytes =
5587     kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB

```

```

5589     cudaFuncSetAttribute(
5590         hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
5591         false>,
5592         cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5593
5594     dim3 block(kNumThreads);
5595     dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5596     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
5597         <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5598     check(cudaGetLastError(), "wgmma launch");
5599     check(cudaDeviceSynchronize(), "wgmma sync");
5600
5601     half *h_c = (half *)malloc(size_c);
5602     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5603
5604     // Verify
5605     float max_err = 0.0f;
5606     for (int i = 0; i < M * N; i++) {
5607         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5608         if (err > max_err)
5609             max_err = err;
5610     }
5611     printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM TMA WGMMMA WS (3-stage)", max_err,
5612           max_err < 1.0f ? "PASS" : "FAIL", "None");
5613
5614     free(h_a);
5615     free(h_b);
5616     free(h_b_t);
5617     free(h_c);
5618     free(h_c_ref);
5619     cudaFree(d_a);
5620     cudaFree(d_b);
5621     cudaFree(d_b_t);
5622     cudaFree(d_c);
5623     cudaFree(tma_a);
5624     cudaFree(tma_b);
5625     cublasDestroy(handle);
5626 }
5627 #endif /* NOTES_V2_ENABLE_WGMMMA */
5628
5629 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5630 template <int kStages, int kBlockSwizzle = 0>
5631 static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
5632                                     half *d_b_t, half *d_c,
5633                                     CUTensorMap *tma_a, CUTensorMap *tma_b) {
5634     constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
5635     constexpr int kMmaTileM = 2, kMmaTileN = 2;
5636     constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
5637     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
5638     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
5639     constexpr int BK = kMmaK * kValTileK;
5640     constexpr int kNumThreads = 256;
5641     constexpr size_t payload_bytes =
5642         kStages * (BM * BK + BN * BK) * sizeof(half);
5643     constexpr size_t smem_bytes = payload_bytes;
5644     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
5645                             const CUTensorMap *);
5646     Kernel kernel = hgemm_tma_mma_ws_tn<
5647         kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
5648         kValTileK, kStages, kNumThreads, kBlockSwizzle>;
5649
5650     int device = 0;
5651     int max_smem = 0;

```

```

5652 cudaFuncAttributes attributes{};
5653 check(cudaGetDevice(&device), "tma_mma_ws get device");
5654 check(cudaDeviceGetAttribute(&max_smem,
5655                             cudaDevAttrMaxSharedMemoryPerBlockOptin,
5656                             device),
5657        "tma_mma_ws max shared memory");
5658 check(cudaFuncGetAttributes(&attributes, kernel),
5659        "tma_mma_ws function attributes");
5660 if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
5661     printf("| %-42s | %-12s | %-4s | %-8s |\n",
5662           kStages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5663                       : "HGEMM TMA MMA WS (S=3, SW=0)",
5664           "SMEM SKIP", "SKIP", "None");
5665     return false;
5666 }
5667
5668 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
5669                             smem_bytes),
5670        "tma_mma_ws set dynamic shared memory");
5671 dim3 block(kNumThreads);
5672 constexpr int kSwizzleN = 16;
5673 const int n_tiles = N / BN;
5674 const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
5675 const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
5676 dim3 grid(grid_x, M / BM, grid_z);
5677 kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5678 check(cudaGetLastError(), "tma_mma_ws launch");
5679 check(cudaDeviceSynchronize(), "tma_mma_ws sync");
5680 return true;
5681 }
5682
5683 static void test_hgemm_tma_mma_ws(int M, int N, int K) {
5684     constexpr int BM = 128, BN = 128, BK = 64;
5685     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5686         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS", "SKIP",
5687             "SKIP", "None");
5688         return;
5689     }
5690
5691     const size_t size_a = size_t(M) * K * sizeof(half);
5692     const size_t size_b = size_t(K) * N * sizeof(half);
5693     const size_t size_b_t = size_t(N) * K * sizeof(half);
5694     const size_t size_c = size_t(M) * N * sizeof(half);
5695     half *h_a = static_cast<half *>(malloc(size_a));
5696     half *h_b = static_cast<half *>(malloc(size_b));
5697     half *h_b_t = static_cast<half *>(malloc(size_b_t));
5698     half *h_c = static_cast<half *>(malloc(size_c));
5699     half *h_c_ref = static_cast<half *>(malloc(size_c));
5700     srand(42);
5701     for (int i = 0; i < M * K; ++i)
5702         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5703     for (int i = 0; i < K * N; ++i)
5704         h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5705     for (int n = 0; n < N; ++n)
5706         for (int k = 0; k < K; ++k)
5707             h_b_t[n * K + k] = h_b[k * N + n];
5708
5709     half *d_a, *d_b, *d_b_t, *d_c;
5710     check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
5711     check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
5712     check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
5713     check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
5714     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");

```

```

5715 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
5716 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5717         "tma_mma_ws H2D B_t");
5718
5719 cublasHandle_t handle;
5720 cublasCreate(&handle);
5721 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
5722 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
5723             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
5724             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5725 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5726         "tma_mma_ws D2H reference");
5727
5728 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5729 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5730 for (int block_swizzle : {0, 1}) {
5731     for (int stages : {2, 3}) {
5732         const bool launched = stages == 2
5733             ? (block_swizzle
5734                 ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
5735                     tma_a, tma_b)
5736                 : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
5737                     tma_a, tma_b))
5738             : (block_swizzle
5739                 ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
5740                     tma_a, tma_b)
5741                 : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
5742                     tma_a, tma_b));
5743         if (!launched)
5744             continue;
5745         check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
5746             "tma_mma_ws D2H");
5747         float max_err = 0.0f;
5748         for (int i = 0; i < M * N; ++i)
5749             max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
5750                 __half2float(h_c_ref[i])));
5751         printf("| %-42s | %.6e | %-4s | %-8s |\n",
5752             block_swizzle
5753                 ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=1)"
5754                     : "HGEMM TMA MMA WS (S=3, SW=1)")
5755                 : (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5756                     : "HGEMM TMA MMA WS (S=3, SW=0)"),
5757             max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
5758     }
5759 }
5760
5761 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5762 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5763 cudaFree(tma_a); cudaFree(tma_b);
5764 cublasDestroy(handle);
5765 }
5766 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
5767
5768
5769 static void test_flash_attn(int seqlen, int head_dim) {
5770     // FlashAttention-2 with split-Q, MMA m16n8k16
5771     int B = 1, H = 8;
5772
5773     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5774
5775     srand(42);
5776     half *h_q = (half *)malloc(sz);
5777     half *h_k = (half *)malloc(sz);

```

```

5778 half *h_v = (half *)malloc(sz);
5779 for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5780     h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5781     h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5782     h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5783 }
5784
5785 // CPU reference in FP32: O = softmax(Q @ K^T / sqrt(d)) @ V
5786 float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5787 float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5788 float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5789 float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5790 int count = B * H * seqlen * head_dim;
5791 for (int i = 0; i < count; i++) {
5792     ref_q[i] = __half2float(h_q[i]);
5793     ref_k[i] = __half2float(h_k[i]);
5794     ref_v[i] = __half2float(h_v[i]);
5795 }
5796
5797 float scale = 1.0f / sqrtf((float)head_dim);
5798 for (int bi = 0; bi < B * H; bi++) {
5799     for (int qi = 0; qi < seqlen; qi++) {
5800         // S[qi, kj] = Q[qi,:] @ K[kj,:]^T * scale
5801         float smax = -INFINITY;
5802         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5803         for (int kj = 0; kj < seqlen; kj++) {
5804             float s = 0.0f;
5805             for (int d = 0; d < head_dim; d++)
5806                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5807                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5808             S[kj] = s * scale;
5809             if (S[kj] > smax) smax = S[kj];
5810         }
5811         // softmax
5812         double sum_exp = 0.0;
5813         for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5814         float inv_sum = 1.0f / (float)sum_exp;
5815         // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5816         for (int d = 0; d < head_dim; d++) {
5817             double o_acc = 0.0;
5818             for (int kj = 0; kj < seqlen; kj++)
5819                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5820                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5821             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5822         }
5823         free(S);
5824     }
5825 }
5826
5827 half *d_q, *d_k, *d_v, *d_o;
5828 check(cudaMalloc(&d_q, sz), "fa alloc Q");
5829 check(cudaMalloc(&d_k, sz), "fa alloc K");
5830 check(cudaMalloc(&d_v, sz), "fa alloc V");
5831 check(cudaMalloc(&d_o, sz), "fa alloc O");
5832 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5833 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5834 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5835
5836 // Template params for kHeadDim=64, kStage=2
5837 constexpr int kHeadDim = 64, kStage = 2, kPad = 8;
5838 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
5839 constexpr int kMmaTileSeqLenQ = 8;
5840 constexpr int kMmaTileSeqLenK = 1;

```

```

5841 constexpr int kMmaTileSeqLenP = 8;
5842 constexpr int kMmaTileHeadDimV = 1;
5843 constexpr int kValTileSeqLenQ = 1;
5844 constexpr int kValTileSeqLenK = 8;
5845 constexpr int kValTileSeqLenP = 1;
5846 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5847
5848 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5849 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5850 size_t smem_bytes = (Br * (kHeadDim + kPad) +
5851                     kStage * Bc * (kHeadDim + kPad) +
5852                     Bc * (kHeadDim + kPad)) * sizeof(half);
5853
5854 dim3 block(256);
5855 dim3 grid((seqlen + Br - 1) / Br, B * H);
5856
5857 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
5858 FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
5859      kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
5860      kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
5861      kValTileHeadDimV, kStage, kPad>;
5862 cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5863 fa_k<<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
5864 check(cudaGetLastError(), "fa launch");
5865 check(cudaDeviceSynchronize(), "fa sync");
5866
5867 half *h_o = (half *)malloc(sz);
5868 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
5869
5870 float max_err = 0.0f;
5871 for (int i = 0; i < count; i++) {
5872     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
5873     if (err > max_err) max_err = err;
5874 }
5875 printf("| %-42s | %.6e | %-4s | %-8s |\n", "FlashAttn-SplitQ", max_err, max_err < 5e-1f ? "PASS" : "FAIL",
5876        "None");
5877
5878 free(h_q); free(h_k); free(h_v); free(h_o);
5879 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
5880 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
5881 }
5882
5883 // Bench mode globals and helpers
5884 static bool g_bench_mode = false;
5885 static bool g_bench_fa = false;
5886 static int g_bench_M = 4096, g_bench_N = 4096, g_bench_K = 4096;
5887 static int g_bench_B = 1, g_bench_H = 32, g_bench_Nfa = 4096, g_bench_D = 64;
5888
5889 static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
5890     double flops = 2.0 * M * N * K;
5891     return (float)(flops / (double)time_ms / 1e9);
5892 }
5893
5894 static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
5895     double flops = 4.0 * B * H * N * N * D;
5896     return (float)(flops / (double)time_ms / 1e9);
5897 }
5898
5899 static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
5900     int device = 0;
5901     int max_smem = 0;
5902     cudaFuncAttributes attrs{};

```

```

5903     cudaGetDevice(&device);
5904     cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
5905     cudaFuncGetAttributes(&attrs, kernel_func);
5906     return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
5907 }
5908
5909 // =====
5910 // Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
5911 // =====
5912 template <int kStages, int kBlockSwizzle>
5913 static bool launch_timed_hgemm_mma(half *d_a, half *d_b_t, half *d_c, half *h_c,
5914     half *h_c_ref, int M, int N, int K, size_t size_c,
5915     cudaEvent_t start, cudaEvent_t stop, float &max_err,
5916     float &time_ms) {
5917     constexpr int BM = 128, BN = 128, BK = 16;
5918     using Kernel = void (*)(half *, half *, half *, int, int, int);
5919     Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
5920     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
5921     if (!check_smem_feasible((const void *)k, smem)) return false;
5922     dim3 block(256);
5923     const int tiles_n = (N + BN - 1) / BN;
5924     constexpr int kSwizzleN = 16;
5925     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
5926     int gz = kBlockSwizzle ? kSwizzleN : 1;
5927     dim3 grid(gx, (M + BM - 1) / BM, gz);
5928     cudaEventRecord(start);
5929     k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
5930     cudaEventRecord(stop);
5931     cudaEventSynchronize(stop);
5932     cudaEventElapsedTime(&time_ms, start, stop);
5933     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
5934     max_err = 0.0f;
5935     for (int i = 0; i < M * N; i++) {
5936         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5937         if (err > max_err) max_err = err;
5938     }
5939     return true;
5940 }
5941
5942 static void bench_hgemm_mma(int M, int N, int K) {
5943     size_t size_a = (size_t)M * K * sizeof(half);
5944     size_t size_b = (size_t)K * N * sizeof(half);
5945     size_t size_c = (size_t)M * N * sizeof(half);
5946     half *h_a = (half *)malloc(size_a);
5947     half *h_b = (half *)malloc(size_b);
5948     half *h_c_ref = (half *)malloc(size_c);
5949     srand(42);
5950     for (int i = 0; i < M * K; i++)
5951         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5952     for (int i = 0; i < K * N; i++)
5953         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5954     size_t size_b_t = (size_t)N * K * sizeof(half);
5955     half *h_b_t = (half *)malloc(size_b_t);
5956     for (int n = 0; n < N; n++)
5957         for (int k = 0; k < K; k++)
5958             h_b_t[n * K + k] = h_b[k * N + n];
5959     half *d_a, *d_b, *d_b_t, *d_c;
5960     check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
5961     check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
5962     check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
5963     check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
5964     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
5965     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");

```

```

5966 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
5967 cublasHandle_t handle;
5968 cublasCreate(&handle);
5969 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5970 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
5971             d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
5972             CUBLAS_GEMM_DEFAULT);
5973 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
5974 half *h_c = (half *)malloc(size_c);
5975 cudaEvent_t start, stop;
5976 cudaEventCreate(&start);
5977 cudaEventCreate(&stop);
5978 for (int stages : {2, 3}) {
5979     for (int swizzle : {0, 1}) {
5980         float max_err = 0, time_ms = 0;
5981         bool ok = false;
5982         if (stages == 2)
5983             ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
5984                 K, size_c, start, stop, max_err, time_ms)
5985                 : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
5986                 K, size_c, start, stop, max_err, time_ms);
5987         else
5988             ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
5989                 K, size_c, start, stop, max_err, time_ms)
5990                 : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
5991                 K, size_c, start, stop, max_err, time_ms);
5992         char label[64];
5993         snprintf(label, sizeof(label), "HGEMM MMA (S=%d, SW=%d)", stages, swizzle);
5994         if (!ok) {
5995             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
5996         } else {
5997             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
5998             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
5999                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6000         }
6001     }
6002 }
6003 cudaEventDestroy(start);
6004 cudaEventDestroy(stop);
6005 free(h_a);
6006 free(h_b);
6007 free(h_b_t);
6008 free(h_c);
6009 free(h_c_ref);
6010 cudaFree(d_a);
6011 cudaFree(d_b);
6012 cudaFree(d_b_t);
6013 cudaFree(d_c);
6014 cublasDestroy(handle);
6015 }
6016
6017 // =====
6018 // Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=2, BK=32)
6019 // =====
6020 template <int kStages, int kBlockSwizzle>
6021 static bool launch_timed_hgemm_swizzle(half *d_a, half *d_b_t, half *d_c, half *h_c,
6022             half *h_c_ref, int M, int N, int K, size_t size_c,
6023             cudaEvent_t start, cudaEvent_t stop, float &max_err,
6024             float &time_ms) {
6025     constexpr int BM = 128, BN = 128, BK = 32;
6026     using Kernel = void (*)(half *, half *, half *, int, int, int);
6027     Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 2, kStages, kBlockSwizzle>;
6028     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);

```

```

6029 if (!check_smem_feasible((const void *)k, smem)) return false;
6030 dim3 block(256);
6031 const int tiles_n = (N + BN - 1) / BN;
6032 constexpr int kSwizzleN = 16;
6033 int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6034 int gz = kBlockSwizzle ? kSwizzleN : 1;
6035 dim3 grid(gx, (M + BM - 1) / BM, gz);
6036 cudaEventRecord(start);
6037 k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6038 cudaEventRecord(stop);
6039 cudaEventSynchronize(stop);
6040 cudaEventElapsedTime(&time_ms, start, stop);
6041 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
6042 max_err = 0.0f;
6043 for (int i = 0; i < M * N; i++) {
6044     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6045     if (err > max_err) max_err = err;
6046 }
6047 return true;
6048 }
6049
6050 static void bench_hgemm_swizzle(int M, int N, int K) {
6051     size_t size_a = (size_t)M * K * sizeof(half);
6052     size_t size_b = (size_t)K * N * sizeof(half);
6053     size_t size_c = (size_t)M * N * sizeof(half);
6054     half *h_a = (half *)malloc(size_a);
6055     half *h_b = (half *)malloc(size_b);
6056     half *h_c_ref = (half *)malloc(size_c);
6057     srand(42);
6058     for (int i = 0; i < M * K; i++)
6059         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6060     for (int i = 0; i < K * N; i++)
6061         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6062     size_t size_b_t = (size_t)N * K * sizeof(half);
6063     half *h_b_t = (half *)malloc(size_b_t);
6064     for (int n = 0; n < N; n++)
6065         for (int k = 0; k < K; k++)
6066             h_b_t[n * K + k] = h_b[k * N + n];
6067     half *d_a, *d_b, *d_b_t, *d_c;
6068     check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
6069     check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
6070     check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
6071     check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
6072     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
6073     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
6074     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
6075     cublasHandle_t handle;
6076     cublasCreate(&handle);
6077     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6078     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6079         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6080         CUBLAS_GEMM_DEFAULT);
6081     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
6082     half *h_c = (half *)malloc(size_c);
6083     cudaEvent_t start, stop;
6084     cudaEventCreate(&start);
6085     cudaEventCreate(&stop);
6086     for (int stages : {2, 3}) {
6087         for (int swizzle : {0, 1}) {
6088             float max_err = 0, time_ms = 0;
6089             bool ok = false;
6090             if (stages == 2)
6091                 ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,

```

```

6092         N, K, size_c, start, stop, max_err, time_ms)
6093     : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6094         N, K, size_c, start, stop, max_err, time_ms);
6095     else
6096         ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6097             N, K, size_c, start, stop, max_err, time_ms)
6098         : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6099             N, K, size_c, start, stop, max_err, time_ms);
6100     char label[64];
6101     snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, SW=%d)", stages, swizzle);
6102     if (!ok) {
6103         printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6104     } else {
6105         float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6106         printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6107             max_err < 1.0f ? "PASS" : "FAIL", tflops);
6108     }
6109 }
6110 }
6111 cudaEventDestroy(start);
6112 cudaEventDestroy(stop);
6113 free(h_a);
6114 free(h_b);
6115 free(h_b_t);
6116 free(h_c);
6117 free(h_c_ref);
6118 cudaFree(d_a);
6119 cudaFree(d_b);
6120 cudaFree(d_b_t);
6121 cudaFree(d_c);
6122 cublasDestroy(handle);
6123 }
6124 #if defined(NOTES_V2_ENABLE_CUTE)
6125 // =====
6126 // Bench: HGEMM CuTe
6127 // =====
6128 static void bench_hgemm_cute(int M, int N, int K) {
6129     size_t size_a = (size_t)M * K * sizeof(half);
6130     size_t size_b = (size_t)K * N * sizeof(half);
6131     size_t size_c = (size_t)M * N * sizeof(half);
6132     half *h_a = (half *)malloc(size_a);
6133     half *h_b = (half *)malloc(size_b);
6134     half *h_c_ref = (half *)malloc(size_c);
6135     srand(42);
6136     for (int i = 0; i < M * K; i++)
6137         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6138     for (int i = 0; i < K * N; i++)
6139         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6140     size_t size_b_t = (size_t)N * K * sizeof(half);
6141     half *h_b_t = (half *)malloc(size_b_t);
6142     for (int n = 0; n < N; n++)
6143         for (int k = 0; k < K; k++)
6144             h_b_t[n * K + k] = h_b[k * N + n];
6145     half *d_a, *d_b, *d_b_t, *d_c;
6146     check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
6147     check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
6148     check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
6149     check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
6150     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
6151     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");
6152     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
6153     cublasHandle_t handle;
6154     cublasCreate(&handle);

```

```

6155 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6156 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6157 d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6158 CUBLAS_GEMM_DEFAULT);
6159 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
6160 half *h_c = (half *)malloc(size_c);
6161 cudaEvent_t start, stop;
6162 cudaEventCreate(&start);
6163 cudaEventCreate(&stop);
6164 for (int stages : {2, 3}) {
6165     for (int swizzle : {0, 1}) {
6166         char label[64];
6167         snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, SW=%d)", stages, swizzle);
6168         bool ok = false;
6169         float time_ms = 0, max_err = 0;
6170         if (stages == 2) {
6171             if (swizzle) {
6172                 cudaEventRecord(start);
6173                 launch_hgemm_mma_stages_tn_cute<half, 2, 1>(d_a, d_b_t, d_c, M, N, K);
6174                 cudaEventRecord(stop);
6175                 cudaEventSynchronize(stop);
6176                 cudaEventElapsedTime(&time_ms, start, stop);
6177                 ok = true;
6178             } else {
6179                 cudaEventRecord(start);
6180                 launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
6181                 cudaEventRecord(stop);
6182                 cudaEventSynchronize(stop);
6183                 cudaEventElapsedTime(&time_ms, start, stop);
6184                 ok = true;
6185             }
6186         } else {
6187             if (swizzle) {
6188                 cudaEventRecord(start);
6189                 launch_hgemm_mma_stages_tn_cute<half, 3, 1>(d_a, d_b_t, d_c, M, N, K);
6190                 cudaEventRecord(stop);
6191                 cudaEventSynchronize(stop);
6192                 cudaEventElapsedTime(&time_ms, start, stop);
6193                 ok = true;
6194             } else {
6195                 cudaEventRecord(start);
6196                 launch_hgemm_mma_stages_tn_cute<half, 3, 0>(d_a, d_b_t, d_c, M, N, K);
6197                 cudaEventRecord(stop);
6198                 cudaEventSynchronize(stop);
6199                 cudaEventElapsedTime(&time_ms, start, stop);
6200                 ok = true;
6201             }
6202         }
6203         if (ok) {
6204             check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
6205             max_err = 0.0f;
6206             for (int i = 0; i < M * N; i++) {
6207                 float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6208                 if (err > max_err) max_err = err;
6209             }
6210             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6211             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6212                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6213         } else {
6214             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "LAUNCH ERR", "FAIL", "None");
6215         }
6216     }
6217 }

```

```

6218     cudaEventDestroy(start);
6219     cudaEventDestroy(stop);
6220     free(h_a);
6221     free(h_b);
6222     free(h_b_t);
6223     free(h_c);
6224     free(h_c_ref);
6225     cudaFree(d_a);
6226     cudaFree(d_b);
6227     cudaFree(d_b_t);
6228     cudaFree(d_c);
6229     cublasDestroy(handle);
6230 }
6231 #endif
6232
6233 #if defined(NOTES_V2_ENABLE_WGMMMA)
6234 // =====
6235 // Bench: HGEMM WGMMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
6236 // =====
6237 template <int kStages, int kBlockSwizzle>
6238 static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
6239                                     CUTensorMap *tma_a, CUTensorMap *tma_b,
6240                                     int M, int N, int K, size_t size_c,
6241                                     cudaEvent_t start, cudaEvent_t stop,
6242                                     float &max_err, float &time_ms) {
6243     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
6244     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
6245                             const CUTensorMap *);
6246     Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
6247                kBlockSwizzle>;
6248     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6249     if (!check_smem_feasible((const void *)k, smem)) return false;
6250     cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
6251     dim3 block(kNumThreads);
6252     const int tiles_n = (N + BN - 1) / BN;
6253     constexpr int kSwizzleN = 16;
6254     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6255     int gz = kBlockSwizzle ? kSwizzleN : 1;
6256     dim3 grid(gx, (M + BM - 1) / BM, gz);
6257     cudaEventRecord(start);
6258     k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
6259     cudaEventRecord(stop);
6260     cudaEventSynchronize(stop);
6261     cudaEventElapsedTime(&time_ms, start, stop);
6262     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
6263     max_err = 0.0f;
6264     for (int i = 0; i < M * N; i++) {
6265         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6266         if (err > max_err) max_err = err;
6267     }
6268     return true;
6269 }
6270
6271 static void bench_hgemm_wgmma(int M, int N, int K) {
6272     constexpr int BM = 128, BN = 128, BK = 64;
6273     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6274         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM WGMMMA (unaligned)", "SKIP", "SKIP",
6275             "None");
6276         return;
6277     }
6278     size_t size_a = (size_t)M * K * sizeof(half);
6279     size_t size_b = (size_t)K * N * sizeof(half);
6280     size_t size_c = (size_t)M * N * sizeof(half);

```

```

6281 half *h_a = (half *)malloc(size_a);
6282 half *h_b = (half *)malloc(size_b);
6283 half *h_c_ref = (half *)malloc(size_c);
6284 srand(42);
6285 for (int i = 0; i < M * K; i++)
6286     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6287 for (int i = 0; i < K * N; i++)
6288     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6289 size_t size_b_t = (size_t)N * K * sizeof(half);
6290 half *h_b_t = (half *)malloc(size_b_t);
6291 for (int n = 0; n < N; n++)
6292     for (int k = 0; k < K; k++)
6293         h_b_t[n * K + k] = h_b[k * N + n];
6294 half *d_a, *d_b, *d_b_t, *d_c;
6295 check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
6296 check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
6297 check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");
6298 check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");
6299 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
6300 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
6301 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
6302 cublasHandle_t handle;
6303 cublasCreate(&handle);
6304 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6305 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6306     d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6307     CUBLAS_GEMM_DEFAULT);
6308 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
6309 CUtensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6310 CUtensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6311 half *h_c = (half *)malloc(size_c);
6312 cudaEvent_t start, stop;
6313 cudaEventCreate(&start);
6314 cudaEventCreate(&stop);
6315 for (int stages : {1, 2, 3}) {
6316     for (int swizzle : {0, 1}) {
6317         float max_err = 0, time_ms = 0;
6318         bool ok = false;
6319         if (stages == 2)
6320             ok = swizzle
6321                 ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
6322                     K, size_c, start, stop, max_err, time_ms)
6323                 : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
6324                     size_c, start, stop, max_err, time_ms);
6325         else
6326             ok = swizzle
6327                 ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
6328                     K, size_c, start, stop, max_err, time_ms)
6329                 : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
6330                     size_c, start, stop, max_err, time_ms);
6331         char label[64];
6332         snprintf(label, sizeof(label), "HGEMM TMA WGMMA WS (S=%d, SW=%d)", stages, swizzle);
6333         if (!ok) {
6334             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6335         } else {
6336             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6337             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6338                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6339         }
6340     }
6341 }
6342 cudaEventDestroy(start);
6343 cudaEventDestroy(stop);

```

```

6344 free(h_a);
6345 free(h_b);
6346 free(h_b_t);
6347 free(h_c);
6348 free(h_c_ref);
6349 cudaFree(d_a);
6350 cudaFree(d_b);
6351 cudaFree(d_b_t);
6352 cudaFree(d_c);
6353 cudaFree(tma_a);
6354 cudaFree(tma_b);
6355 cublasDestroy(handle);
6356 }
6357 #endif
6358
6359 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6360 // =====
6361 // Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
6362 // =====
6363 template <int kStages, int kBlockSwizzle>
6364 static bool bench_launch_tma_mma_ws(int M, int N, int K, half *d_a, half *d_b_t,
6365     half *d_c, half *h_c, half *h_c_ref, size_t size_c,
6366     CUTensorMap *tma_a, CUTensorMap *tma_b,
6367     cudaEvent_t start, cudaEvent_t stop,
6368     float &max_err, float &time_ms) {
6369     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
6370     constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
6371     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
6372         const CUTensorMap *);
6373     Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
6374         kNumThreads, kBlockSwizzle>;
6375     if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
6376     cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
6377         payload_bytes);
6378     dim3 block(kNumThreads);
6379     constexpr int kSwizzleN = 16;
6380     const int n_tiles = N / BN;
6381     const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
6382     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
6383     dim3 grid(grid_x, M / BM, grid_z);
6384     cudaEventRecord(start);
6385     kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6386     cudaEventRecord(stop);
6387     cudaEventSynchronize(stop);
6388     cudaEventElapsedTime(&time_ms, start, stop);
6389     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
6390     max_err = 0.0f;
6391     for (int i = 0; i < M * N; ++i)
6392         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
6393     return true;
6394 }
6395
6396 static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
6397     constexpr int BM = 128, BN = 128, BK = 64;
6398     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6399         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP",
6400             "SKIP", "None");
6401         return;
6402     }
6403     const size_t size_a = size_t(M) * K * sizeof(half);
6404     const size_t size_b = size_t(K) * N * sizeof(half);
6405     const size_t size_b_t = size_t(N) * K * sizeof(half);
6406     const size_t size_c = size_t(M) * N * sizeof(half);

```

```

6407 half *h_a = static_cast<half *>(malloc(size_a));
6408 half *h_b = static_cast<half *>(malloc(size_b));
6409 half *h_b_t = static_cast<half *>(malloc(size_b_t));
6410 half *h_c = static_cast<half *>(malloc(size_c));
6411 half *h_c_ref = static_cast<half *>(malloc(size_c));
6412 srand(42);
6413 for (int i = 0; i < M * K; ++i)
6414     h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6415 for (int i = 0; i < K * N; ++i)
6416     h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6417 for (int n = 0; n < N; ++n)
6418     for (int k = 0; k < K; ++k)
6419         h_b_t[n * K + k] = h_b[k * N + n];
6420 half *d_a, *d_b, *d_b_t, *d_c;
6421 check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
6422 check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
6423 check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
6424 check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
6425 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
6426 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
6427 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
6428         "bench tma mma ws H2D B_t");
6429 cublasHandle_t handle;
6430 cublasCreate(&handle);
6431 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
6432 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
6433             d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6434             CUBLAS_GEMM_DEFAULT);
6435 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
6436         "bench tma mma ws D2H reference");
6437 CUsensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6438 CUsensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6439 cudaEvent_t start, stop;
6440 cudaEventCreate(&start);
6441 cudaEventCreate(&stop);
6442 for (int stages : {1, 2, 3}) {
6443     for (int swizzle : {0, 1}) {
6444         float max_err = 0, time_ms = 0;
6445         bool ok = false;
6446         if (stages == 2)
6447             ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
6448                 h_c_ref, size_c, tma_a, tma_b,
6449                 start, stop, max_err, time_ms)
6450             : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
6451                 h_c_ref, size_c, tma_a, tma_b,
6452                 start, stop, max_err, time_ms);
6453         else
6454             ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
6455                 h_c_ref, size_c, tma_a, tma_b,
6456                 start, stop, max_err, time_ms)
6457             : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
6458                 h_c_ref, size_c, tma_a, tma_b,
6459                 start, stop, max_err, time_ms);
6460         char label[64];
6461         snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, SW=%d)", stages, swizzle);
6462         if (!ok) {
6463             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6464         } else {
6465             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6466             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6467                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6468         }
6469     }

```

```

6470 }
6471 cudaEventDestroy(start);
6472 cudaEventDestroy(stop);
6473 free(h_a);
6474 free(h_b);
6475 free(h_b_t);
6476 free(h_c);
6477 free(h_c_ref);
6478 cudaFree(d_a);
6479 cudaFree(d_b);
6480 cudaFree(d_b_t);
6481 cudaFree(d_c);
6482 cudaFree(tma_a);
6483 cudaFree(tma_b);
6484 cublasDestroy(handle);
6485 }
6486 #endif
6487
6488 // =====
6489 // Bench: FlashAttention Split-Q
6490 // =====
6491 static void bench_flash_attn(int B, int H, int N, int D) {
6492     int seqlen = N, head_dim = D;
6493     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
6494
6495     srand(42);
6496     half *h_q = (half *)malloc(sz);
6497     half *h_k = (half *)malloc(sz);
6498     half *h_v = (half *)malloc(sz);
6499     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
6500         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6501         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6502         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6503     }
6504
6505     #if defined(NOTES_V2_ENABLE_CUDNN)
6506     half *d_q, *d_k, *d_v, *d_o, *d_o_ref;
6507     check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
6508     check(cudaMalloc(&d_k, sz), "bench fa alloc K");
6509     check(cudaMalloc(&d_v, sz), "bench fa alloc V");
6510     check(cudaMalloc(&d_o, sz), "bench fa alloc O");
6511     check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
6512     check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
6513     check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
6514     check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");
6515
6516     // cuDNN SDPA reference
6517     cudnnHandle_t cudnn_handle;
6518     cudnnCreate(&cudnn_handle);
6519     {
6520         auto graph = std::make_shared<fe::graph::Graph>();
6521         graph->set_io_data_type(fe::DataType_t::HALF)
6522             .set_intermediate_data_type(fe::DataType_t::FLOAT)
6523             .set_compute_data_type(fe::DataType_t::FLOAT);
6524
6525         auto Q = graph->tensor(fe::graph::Tensor_attributes()
6526             .set_uid(1)
6527             .set_dim({B, H, seqlen, head_dim})
6528             .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6529         auto K = graph->tensor(fe::graph::Tensor_attributes()
6530             .set_uid(2)
6531             .set_dim({B, H, seqlen, head_dim})
6532             .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));

```

```

6533     auto V = graph->tensor(fe::graph::Tensor_attributes()
6534         .set_uid(3)
6535         .set_dim({B, H, seqlen, head_dim})
6536         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6537
6538     auto [O, Stats] = graph->sdpa(Q, K, V,
6539         fe::graph::SDPA_attributes()
6540         .set_name("sdpa_ref")
6541         .set_attn_scale(1.0f / sqrtf((float)head_dim)));
6542
6543     O->set_output(true).set_uid(4)
6544         .set_dim({B, H, seqlen, head_dim})
6545         .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
6546
6547     { auto _e = graph->build(cudnn_handle, {fe::HeurMode_t::A}); if (!_e.is_good()) { fprintf(stderr, "cudnn
6548         build failed: %s\n", _e.get_message().c_str()); exit(1); } }
6549
6550     std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> variant_pack = {
6551         {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}
6552     };
6553
6554     int64_t workspace_size = 0;
6555     { auto _e = graph->get_workspace_size(workspace_size); if (!_e.is_good()) { fprintf(stderr, "cudnn
6556         workspace size failed\n"); exit(1); } }
6557     int8_t *d_workspace = nullptr;
6558     if (workspace_size > 0) check(cudaMalloc(&d_workspace, workspace_size), "bench fa workspace");
6559
6560     { auto _e = graph->execute(cudnn_handle, variant_pack, d_workspace); if (!_e.is_good()) { fprintf(stderr,
6561         "cudnn execute failed\n"); exit(1); } }
6562     check(cudaDeviceSynchronize(), "bench fa cudnn sync");
6563
6564     if (d_workspace) cudaFree(d_workspace);
6565 }
6566 cudnnDestroy(cudnn_handle);
6567
6568     half *h_o_ref = (half *)malloc(sz);
6569     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
6570 #else
6571     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
6572     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
6573     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
6574     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
6575     int count = B * H * seqlen * head_dim;
6576     for (int i = 0; i < count; i++) {
6577         ref_q[i] = __half2float(h_q[i]);
6578         ref_k[i] = __half2float(h_k[i]);
6579         ref_v[i] = __half2float(h_v[i]);
6580     }
6581
6582     float scale = 1.0f / sqrtf((float)head_dim);
6583     for (int bi = 0; bi < B * H; bi++) {
6584         for (int qi = 0; qi < seqlen; qi++) {
6585             float smax = -INFINITY;
6586             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
6587             for (int kj = 0; kj < seqlen; kj++) {
6588                 float s = 0.0f;
6589                 for (int d = 0; d < head_dim; d++)
6590                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
6591                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
6592                 S[kj] = s * scale;
6593                 if (S[kj] > smax) smax = S[kj];
6594             }
6595             double sum_exp = 0.0;

```

```

6593     for (int kj = 0; kj < seqlen; kj++)
6594         sum_exp += (double)expf(S[kj] - smax);
6595     float inv_sum = 1.0f / (float)sum_exp;
6596     for (int d = 0; d < head_dim; d++) {
6597         double o_acc = 0.0;
6598         for (int kj = 0; kj < seqlen; kj++)
6599             o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
6600                 ref_v[bi * seqlen * head_dim + kj * head_dim + d];
6601         ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
6602     }
6603     free(S);
6604 }
6605 }
6606
6607 half *d_q, *d_k, *d_v, *d_o;
6608 check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
6609 check(cudaMalloc(&d_k, sz), "bench fa alloc K");
6610 check(cudaMalloc(&d_v, sz), "bench fa alloc V");
6611 check(cudaMalloc(&d_o, sz), "bench fa alloc O");
6612 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
6613 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
6614 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");
6615 #endif
6616
6617 constexpr int kHeadDim = 64, kStage = 2, kPad = 8;
6618 constexpr int kMmaAtomM = 16, kMmaAtomN = 8, kMmaAtomK = 16;
6619 constexpr int kMmaTileSeqLenQ = 8, kMmaTileSeqLenK = 1, kMmaTileSeqLenP = 8;
6620 constexpr int kMmaTileHeadDimV = 1, kValTileSeqLenQ = 1, kValTileSeqLenK = 8;
6621 constexpr int kValTileSeqLenP = 1;
6622 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
6623 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
6624 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
6625 size_t smem_bytes =
6626     (Br * (kHeadDim + kPad) + kStage * Bc * (kHeadDim + kPad) +
6627      Bc * (kHeadDim + kPad)) *
6628     sizeof(half);
6629
6630 dim3 block(256);
6631 dim3 grid((seqlen + Br - 1) / Br, B * H);
6632
6633 cudaStream_t timing_stream;
6634 cudaStreamCreate(&timing_stream);
6635 cudaEvent_t start, stop;
6636 cudaEventCreate(&start);
6637 cudaEventCreate(&stop);
6638
6639 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
6640 FAKernel fa_k = flash_attn_mma_stages_split_q<
6641     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK,
6642     kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
6643     kValTileSeqLenP, kValTileHeadDimV, kStage, kPad>;
6644 bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
6645 if (smem_ok) {
6646     cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6647 }
6648
6649 cudaEventRecord(start, timing_stream);
6650 if (smem_ok) {
6651     fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, head_dim);
6652 }
6653 cudaEventRecord(stop, timing_stream);
6654 cudaEventSynchronize(stop);
6655

```

```

6656 float time_ms = 0;
6657 cudaEventElapsedTime(&time_ms, start, stop);
6658
6659 half *h_o = (half *)malloc(sz);
6660 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");
6661
6662 int count = B * H * seqlen * head_dim;
6663 float max_err = 0.0f;
6664 if (smem_ok) {
6665 #if defined(NOTES_V2_ENABLE_CUDNN)
6666     for (int i = 0; i < count; i++) {
6667         float err = fabsf(__half2float(h_o[i]) - __half2float(h_o_ref[i]));
6668         if (err > max_err) max_err = err;
6669     }
6670 #else
6671     for (int i = 0; i < count; i++) {
6672         float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
6673         if (err > max_err) max_err = err;
6674     }
6675 #endif
6676 }
6677 if (smem_ok) {
6678     float tflops = bench_fa_tflops(B, H, N, D, time_ms);
6679     printf("| %-42s | %.6e | %-4s | %-8.1f |\n", "FlashAttn-SplitQ (kStage=2)", max_err,
6680           max_err < 5e-1f ? "PASS" : "FAIL", tflops);
6681 } else {
6682     printf("| %-42s | %-12s | %-4s | %-8s |\n", "FlashAttn-SplitQ (kStage=2)", "SMEM too large", "SKIP", "
6683     None");
6684 }
6685
6686 cudaEventDestroy(start);
6687 cudaEventDestroy(stop);
6688 cudaStreamDestroy(timing_stream);
6689 free(h_q);
6690 free(h_k);
6691 free(h_v);
6692 free(h_o);
6693 #if defined(NOTES_V2_ENABLE_CUDNN)
6694 free(h_o_ref);
6695 #endif
6696 cudaFree(d_o_ref);
6697 #else
6698 free(ref_q);
6699 free(ref_k);
6700 free(ref_v);
6701 free(ref_o);
6702 #endif
6703 #endif
6704 cudaFree(d_q);
6705 cudaFree(d_k);
6706 cudaFree(d_v);
6707 cudaFree(d_o);
6708 }
6709
6710 int main(int argc, char *argv[]) {
6711 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6712     cuInit(0);
6713 #endif
6714
6715 for (int i = 1; i < argc; i++) {
6716     if (strcmp(argv[i], "--bench") == 0) {
6717         g_bench_mode = true;
6718     } else if (strcmp(argv[i], "--bench-fa") == 0) {
6719         g_bench_fa = true;

```

```

6718 } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
6719     sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
6720 } else if (strcmp(argv[i], "--bnhd") == 0 && i + 1 < argc) {
6721     sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_Nfa, &g_bench_H, &g_bench_D);
6722 }
6723 }
6724
6725 if (g_bench_mode) {
6726     printf("=== notes-v2.cu bench mode ===\n");
6727     printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
6728         g_bench_M, g_bench_N, g_bench_K,
6729         g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
6730     printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
6731     printf("|-----|-----|-----|-----|\n");
6732
6733     test_block_reduce(g_bench_N);
6734     test_dot(g_bench_N);
6735     test_relu(1024);
6736     test_elementwise(1024);
6737     test_histogram(1024);
6738     test_merge_attn_states(512, 16, 128);
6739     test_softmax(256);
6740     test_rms_norm(8, 128);
6741     test_layer_norm(8, 128);
6742     test_rope(8, 128);
6743     test_mat_transpose(256, 256);
6744     test_mat_transpose_padded(256, 256);
6745     test_sgemv(256, 128);
6746     test_sgemm(g_bench_M, g_bench_N, g_bench_K);
6747
6748     bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
6749     bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
6750 #if defined(NOTES_V2_ENABLE_CUTE)
6751     bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
6752 #endif
6753 #if defined(NOTES_V2_ENABLE_WGMMMA)
6754     bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
6755 #endif
6756 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6757     bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
6758 #endif
6759     if (g_bench_fa)
6760         bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
6761
6762     printf("=== Bench done ===\n");
6763     return 0;
6764 }
6765
6766 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6767 if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
6768     int M = 128, N = 128, K = 64;
6769     if (argc > 4) {
6770         M = atoi(argv[2]);
6771         N = atoi(argv[3]);
6772         K = atoi(argv[4]);
6773     }
6774     printf("=== SM120 TMA MMA WS validation ===\n");
6775     printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
6776     printf("|-----|-----|-----|-----|\n");
6777     test_hgemm_tma_mma_ws(M, N, K);
6778     return 0;
6779 }
6780 #endif

```

```

6781 int M = 1024, N = 1024, K = 1024;
6782 if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
6783
6784 printf("=== notes-v2.cu verification harness ===\n");
6785 printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
6786 printf("|-----|-----|-----|-----|\n");
6787
6788 test_block_reduce(N);
6789 test_dot(N);
6790 test_relu(1024);
6791 test_elementwise(1024);
6792 test_histogram(1024);
6793 test_merge_attn_states(512, 16, 128);
6794 test_softmax(256);
6795 test_rms_norm(8, 128);
6796 test_layer_norm(8, 128);
6797 test_rope(8, 128);
6798 test_mat_transpose(256, 256);
6799 test_mat_transpose_padded(256, 256);
6800 test_sgemv(256, 128);
6801 test_sgemm(M, N, K);
6802 test_hgemm_mma(M, N, K);
6803 test_hgemm_swizzle(M, N, K);
6804 #if defined(NOTES_V2_ENABLE_CUTE)
6805     test_hgemm_cute(M, N, K);
6806 #endif
6807 #if defined(NOTES_V2_ENABLE_WGMMA)
6808     test_hgemm_wgmma(M, N, K);
6809 #endif
6810 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6811     test_hgemm_tma_mma_ws(M, N, K);
6812 #endif
6813 test_flash_attn(1024, 64);
6814
6815 printf("=== All tests done ===\n");
6816 return 0;
6817 }
6818
6819 // =====
6820 // Quick build & run reference
6821 // =====
6822 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMA) :
6823 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
6824 // -I ../../third-party/cutlass/include \
6825 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
6826 //
6827 // # sm_89 单独编译 + 运行:
6828 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
6829 //
6830 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
6831 // 项目内置 third-party/cutlass) :
6832 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
6833 // -I ../../third-party/cutlass/include \
6834 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
6835 //
6836 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
6837 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
6838 // -DNOTES_V2_ENABLE_WGMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
6839 //
6840 // # sm_90a + CuTe + WGMMA:
6841 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
6842 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMA \
6843 // -I ../../third-party/cutlass/include \

```

```

6844 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
6845 //
6846 // # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
6847 // RTX PRO 5000 / RTX 5090):
6848 // nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
6849 // -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
6850 // ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
6851 //
6852 // # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
6853 // requires cudnn-frontend submodule: git submodule update --init):
6854 // nvcc -std=c++20 -O2 -arch=sm_120a \
6855 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
6856 // -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
6857 // -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
6858 // -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
6859 // # Quick bench: ./notes_v2_cute_ws_sm120a.bin --bench --mnk 512,512,512 --bnhd 1,512,8,64
6860 // # Full bench: ./notes_v2_cute_ws_sm120a.bin --bench

```